

Programação Funcional

Unidade 3 – Programação Funcional com a Linguagem LISP



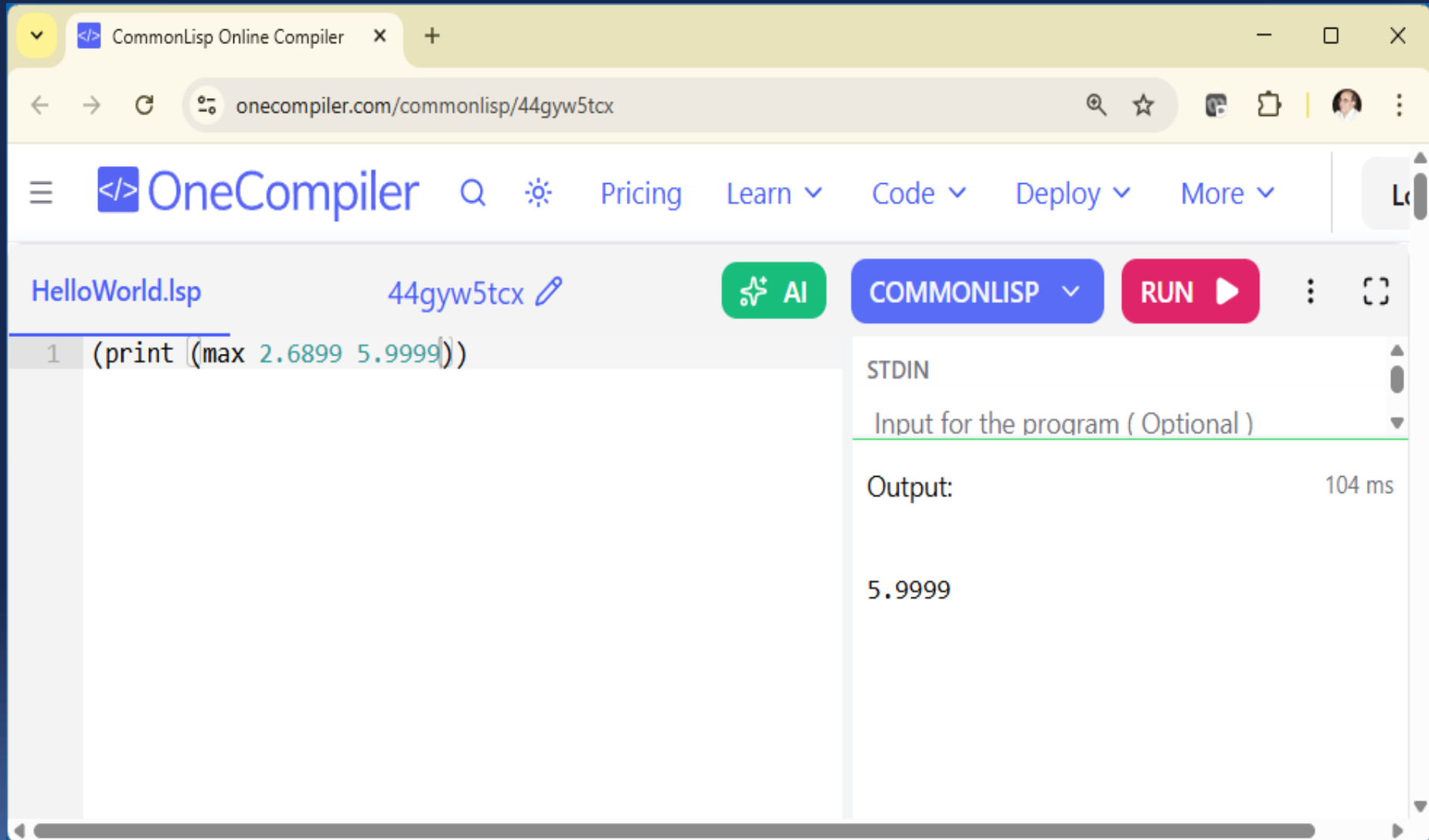
Prof. Aparecido V. de Freitas
Doutor em Engenharia
da Computação pela EPUSP
aparecido.freitas@maua.br
aparecidovfreitas@gmail.com

LISP

- ✓ **Lisp** é uma das linguagens de programação mais antigas (criada em 1958) e ainda muito influente na computação;
- ✓ Baseada em listas (**List Processing**);
- ✓ Sintaxe simples e uniforme (uso de parênteses);
- ✓ Código e dados possuem a mesma estrutura;
- ✓ Suporte forte à recursão;
- ✓ Uso intenso de funções (paradigma funcional).



Compiladores Online



The screenshot shows a web browser window with the OneCompiler website. The browser's address bar displays the URL `onecompiler.com/commonlisp/44gyw5tcx`. The OneCompiler logo is visible in the top left of the page, followed by navigation links: Pricing, Learn, Code, Deploy, and More. The main interface features a code editor on the left with a file named `HelloWorld.lsp` and a snippet identifier `44gyw5tcx`. The code in the editor is `(print (max 2.6899 5.9999))`. To the right of the editor are buttons for AI, a language dropdown set to COMMONLISP, and a RUN button. On the far right, there are icons for settings and a full-screen toggle. On the right side of the interface, there is a panel for STDIN with a text input field labeled "Input for the program (Optional)". Below this is an "Output:" section showing the result `5.9999` and a execution time of `104 ms`.

<http://www.newlisp.org/index.cgi?Documentation>

newlisp.org/index.cgi?Documentation

Drive  GDrive  R  T  g   Ms  Prime  aws  Oracle  ibm  Foto  libgen  FStack  Y  F  ML



puts the fun

back in Lisp

[Home](#) · [About](#) · [Downloads](#) · [Docs](#) · [Modules](#) · [Links](#) · [Tips&Tricks](#)

Documentation

Main documentation and man pages

- newLISP Manual and Reference in [HTML](#) format, no frames.
newLISP Manual and Reference in [HTML](#) format, with index frame.

newLisp



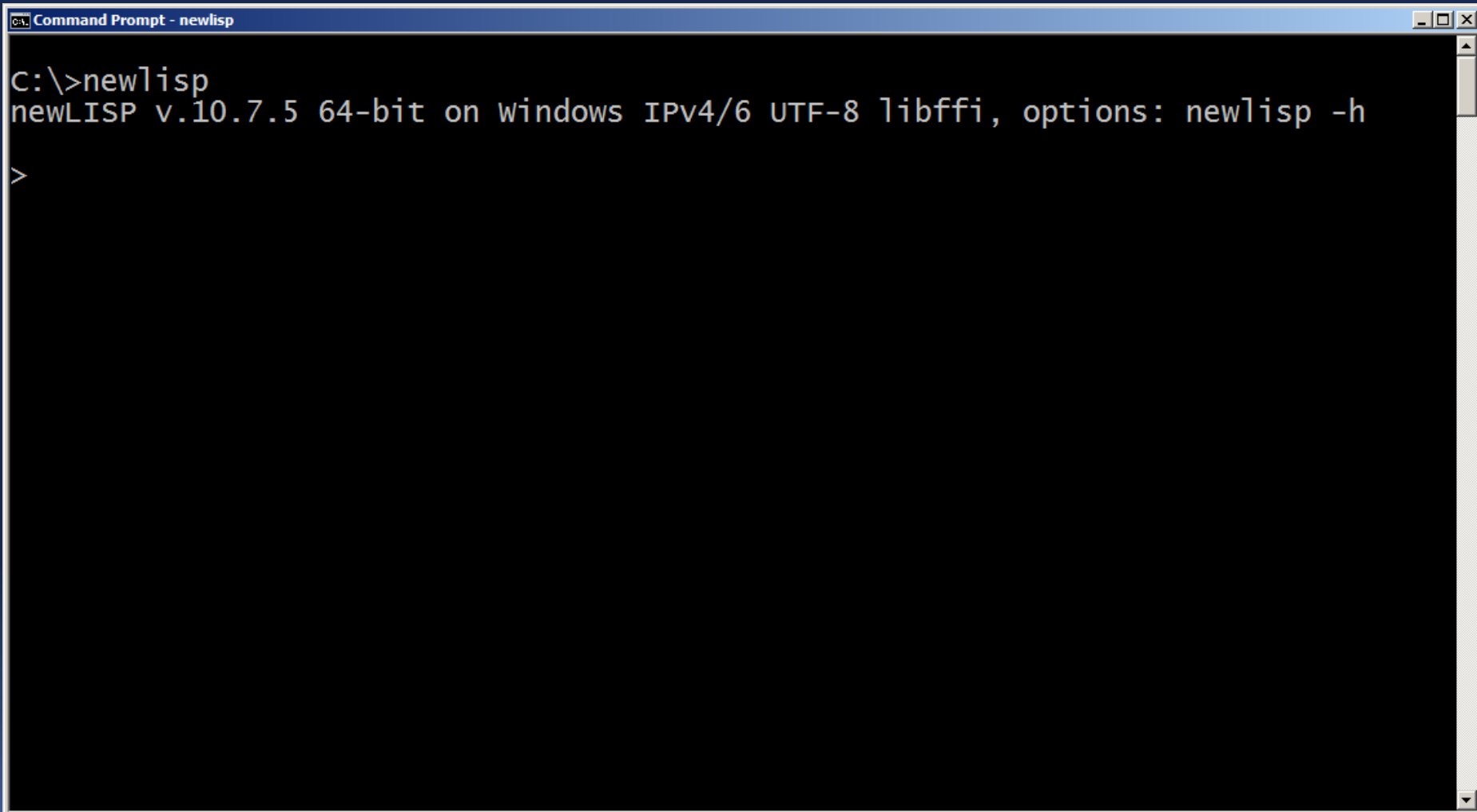
NewLisp
Programming
Language

Downloading e Instalação

- ✓ O download do **newLISP** pode ser feito em <http://www.newlisp.org/>
- ✓ Disponível em diversas plataformas;
- ✓ O interpretador poderá ser chamado diretamente pela linha de comandos, em uma console;
- ✓ O código fonte pode ser gerado a partir de um editor de programas, como por exemplo o **Notepad++**.

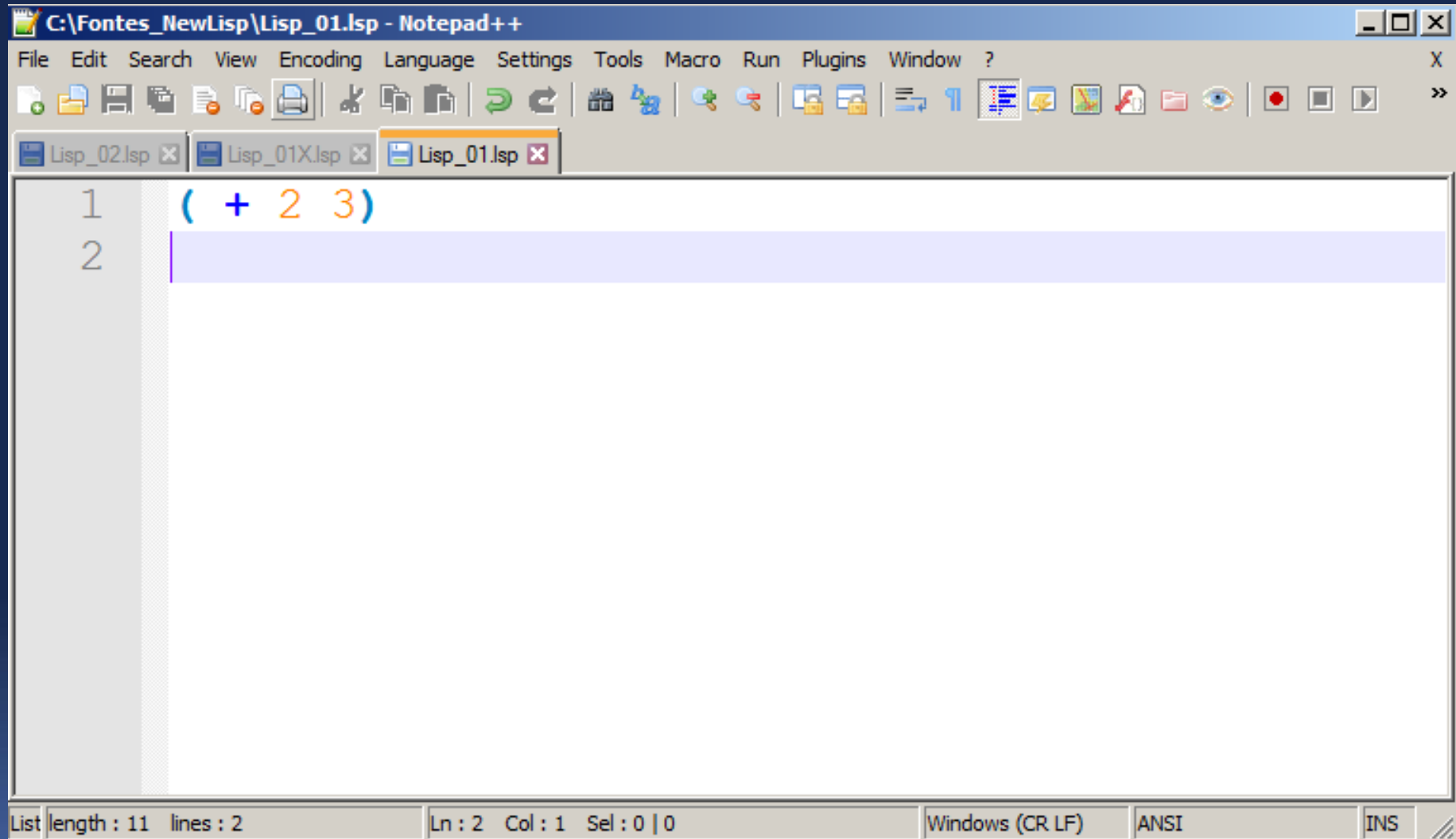


Introdução



```
Command Prompt - newlisp
C:\>newlisp
newLISP v.10.7.5 64-bit on windows IPV4/6 UTF-8 libffi, options: newlisp -h
>
```

Exemplo – Lisp_01.lsp



C:\Fontes_NewLisp\Lisp_01.lsp - Notepad++

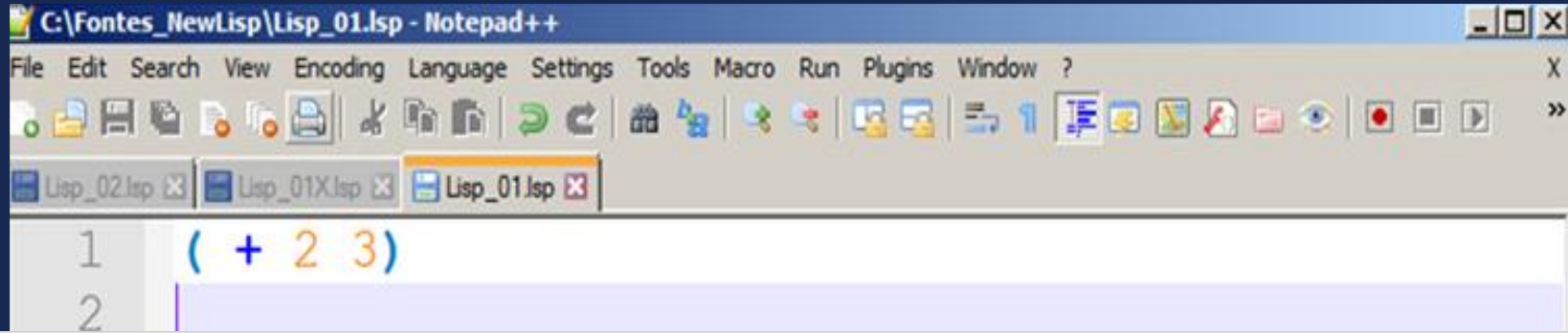
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?

Lisp_02.lsp x Lisp_01X.lsp x Lisp_01.lsp x

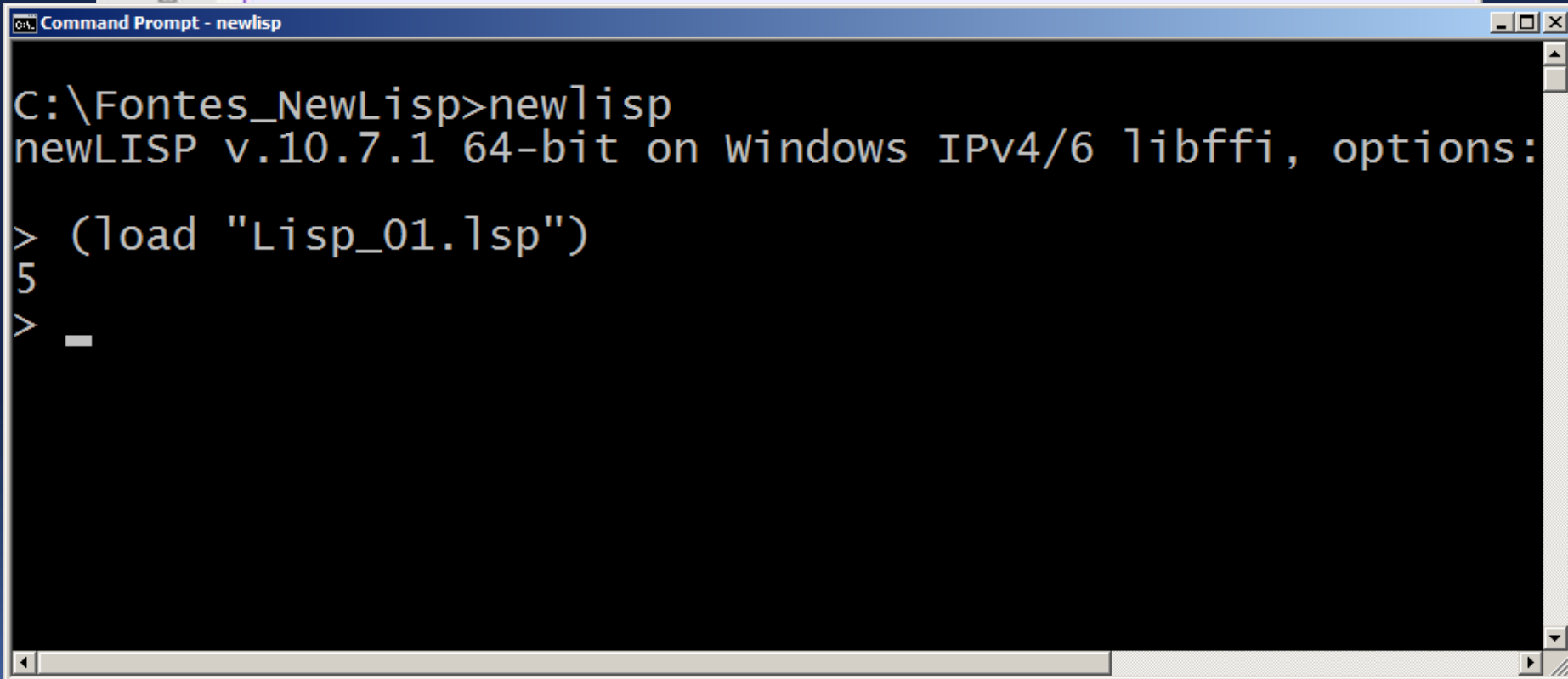
```
1 ( + 2 3 )
2
```

List length : 11 lines : 2 Ln : 2 Col : 1 Sel : 0 | 0 Windows (CR LF) ANSI INS

Exemplo – Lisp_01.lsp

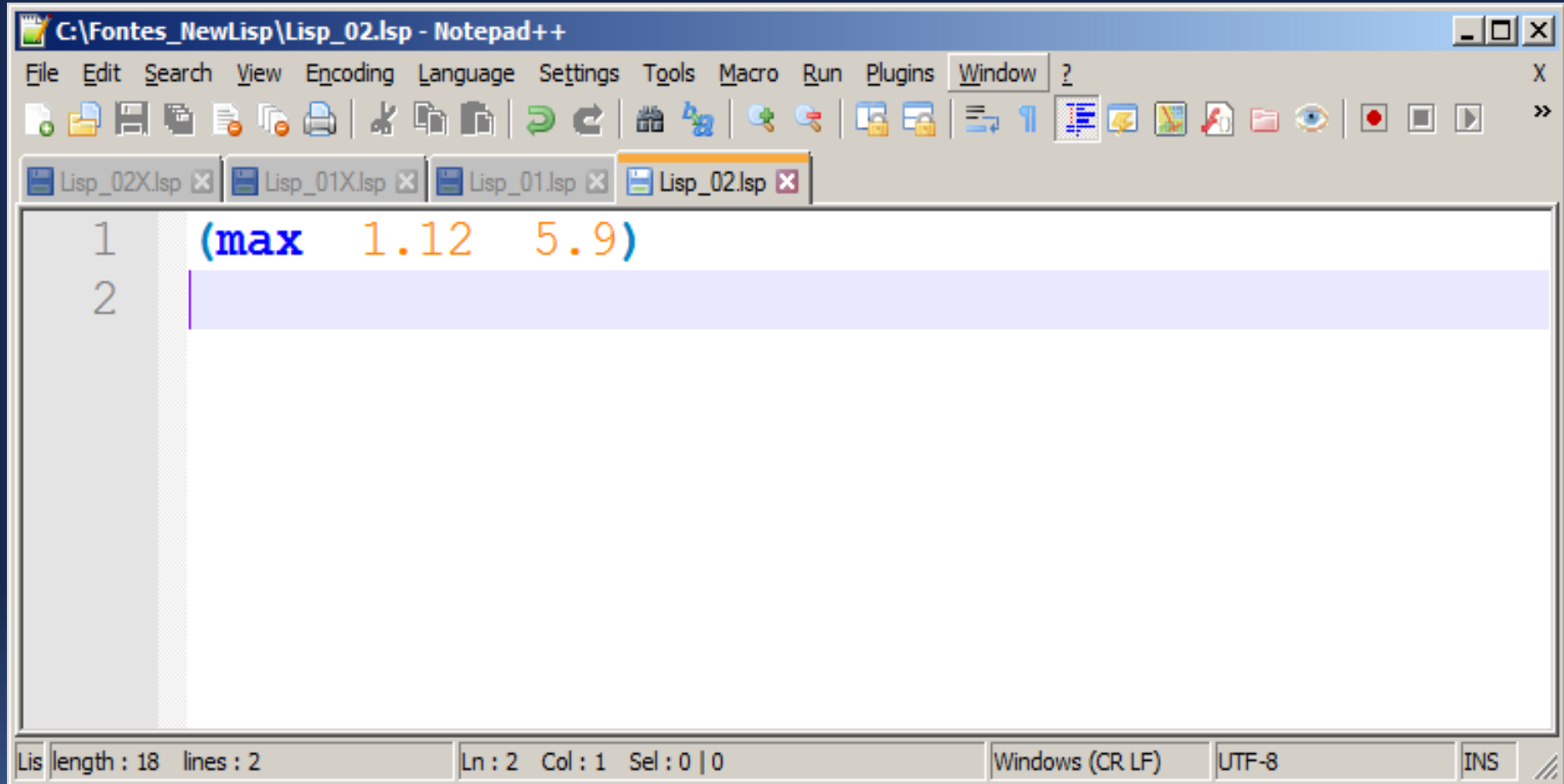


```
C:\Fontes_NewLisp\Lisp_01.lsp - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
Lisp_02.lsp Lisp_01X.lsp Lisp_01.lsp
1 ( + 2 3)
2
```



```
C:\Fontes_NewLisp>newlisp
newLISP v.10.7.1 64-bit on Windows IPv4/6 libffi, options:
> (load "Lisp_01.lsp")
5
> _
```

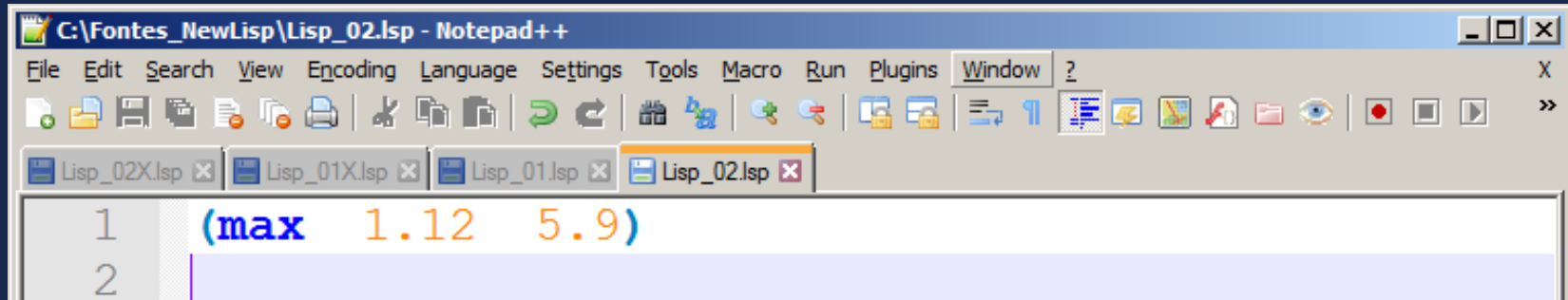
Exemplo – Lisp_02.lsp



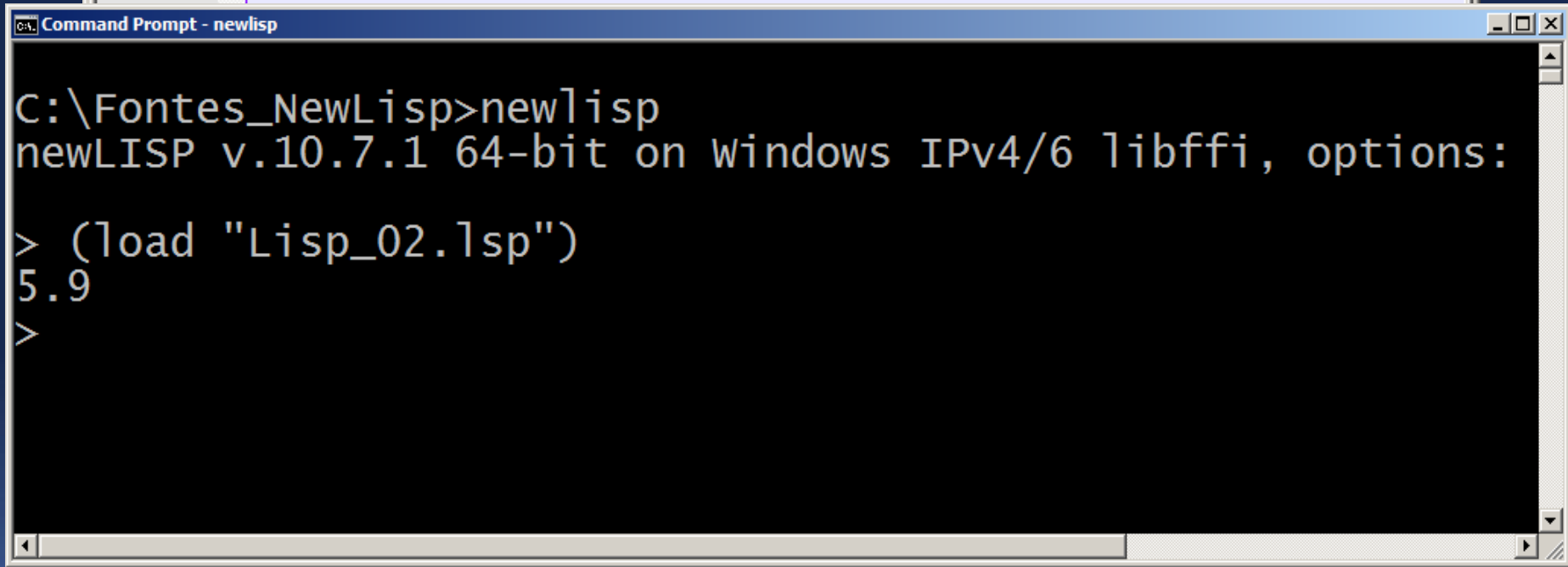
The screenshot shows a Notepad++ window titled "C:\Fontes_NewLisp\Lisp_02.lsp - Notepad++". The menu bar includes File, Edit, Search, View, Encoding, Language, Settings, Tools, Macro, Run, Plugins, Window, and Help. The toolbar contains various icons for file operations, editing, and development. The tab bar shows four open files: Lisp_02X.lsp, Lisp_01X.lsp, Lisp_01.lsp, and Lisp_02.lsp. The main text area contains two lines of code: line 1 is "(max 1.12 5.9)" and line 2 is empty. The status bar at the bottom displays "Lisp length : 18 lines : 2", "Ln : 2 Col : 1 Sel : 0 | 0", "Windows (CR LF)", "UTF-8", and "INS".

```
1 (max 1.12 5.9)
2
```

Exemplo – Lisp_02.lsp



```
C:\Fontes_NewLisp\Lisp_02.lsp - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
Lisp_02X.lsp Lisp_01X.lsp Lisp_01.lsp Lisp_02.lsp
1 (max 1.12 5.9)
2
```

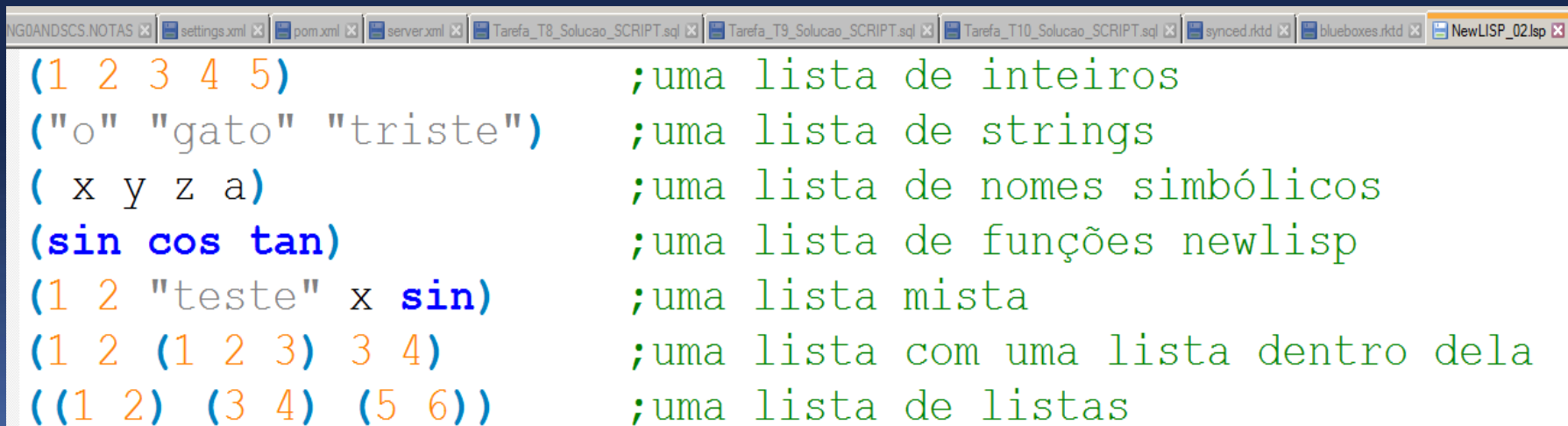


```
Command Prompt - newlisp
C:\Fontes_NewLisp>newlisp
newLISP v.10.7.1 64-bit on Windows IPv4/6 libffi, options:
> (load "Lisp_02.lsp")
5.9
>
```

As três regras básicas de newLISP

Regra 1

- ✓ Uma **lista** é uma sequência de elementos delimitados entre parênteses;
- ✓ **Lista** é a estrutura básica do newLISP;
- ✓ Exemplos:



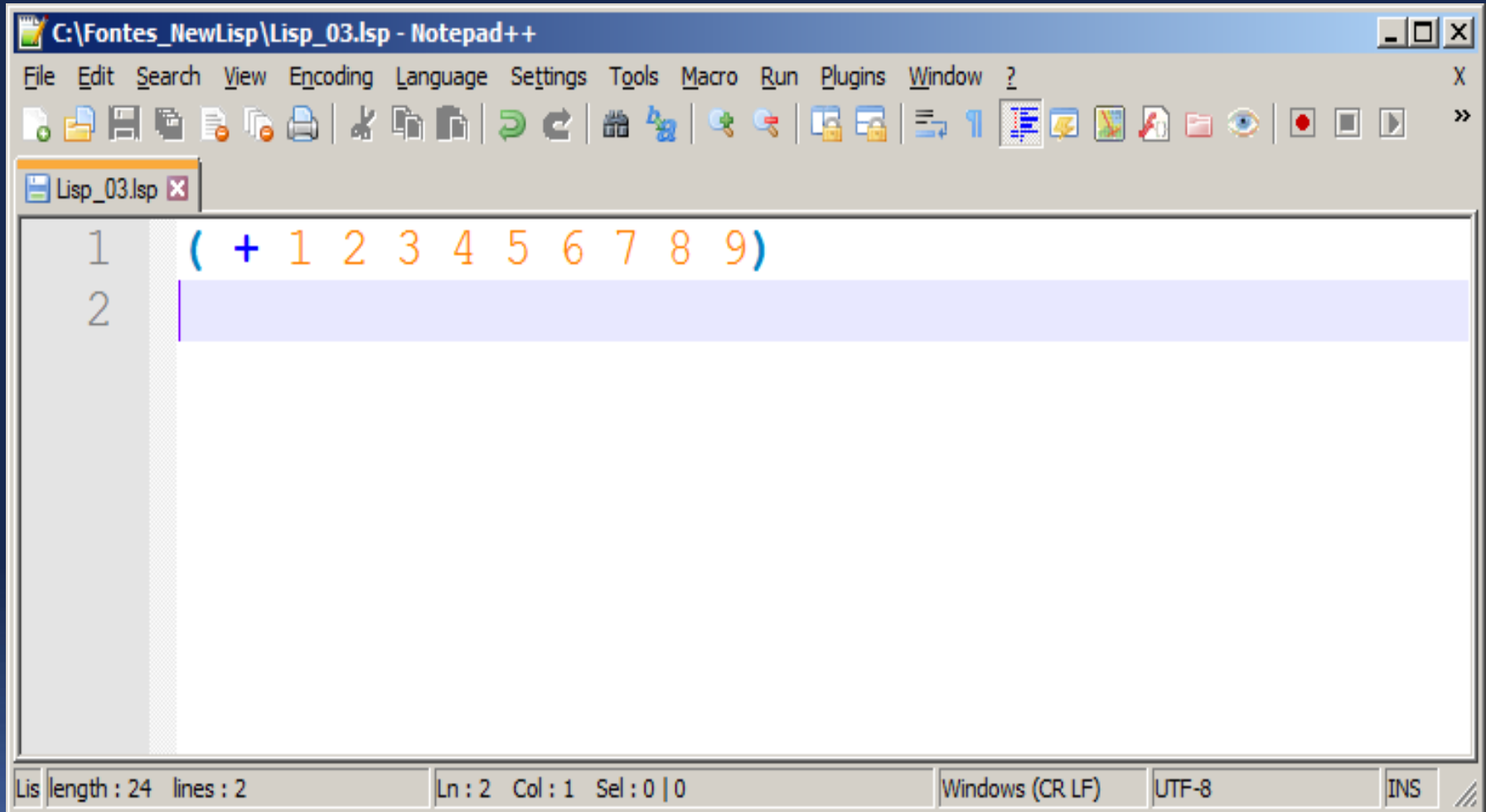
```
(1 2 3 4 5)           ;uma lista de inteiros
("o" "gato" "triste") ;uma lista de strings
(x y z a)             ;uma lista de nomes simbólicos
(sin cos tan)         ;uma lista de funções newlisp
(1 2 "teste" x sin)   ;uma lista mista
(1 2 (1 2 3) 3 4)     ;uma lista com uma lista dentro dela
((1 2) (3 4) (5 6))  ;uma lista de listas
```

As três regras básicas de newLISP

Regra 2

- ✓ Para **newLISP**, o primeiro elemento da lista tem um significado especial;
- ✓ Quando newLISP recebe uma lista, ele trata sempre o primeiro elemento como uma função e tenta utilizar os demais elementos como argumentos;
- ✓ **(+ 5 4)** é uma lista com três elementos. Uma função chamada **+**, seguida por dois valores numéricos;
- ✓ Quando newLISP recebe a lista acima, ele a avalia e retorna o valor **9**;

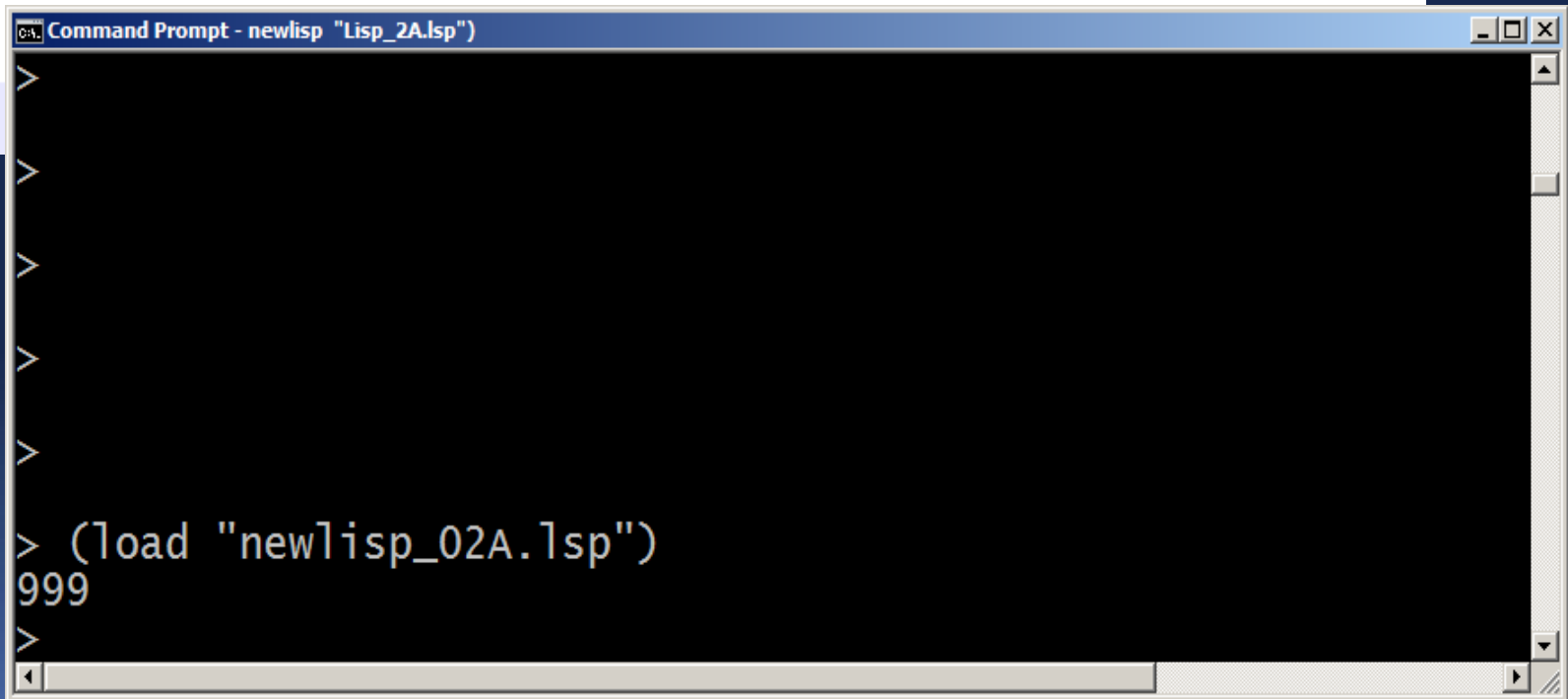
Exemplo – Lisp_03.lsp



```
C:\Fontes_NewLisp\Lisp_03.lsp - Notepad++  
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?  
Lisp_03.lsp  
1 (+ 1 2 3 4 5 6 7 8 9)  
2  
Lis length : 24 lines : 2 Ln : 2 Col : 1 Sel : 0 | 0 Windows (CR LF) UTF-8 INS
```

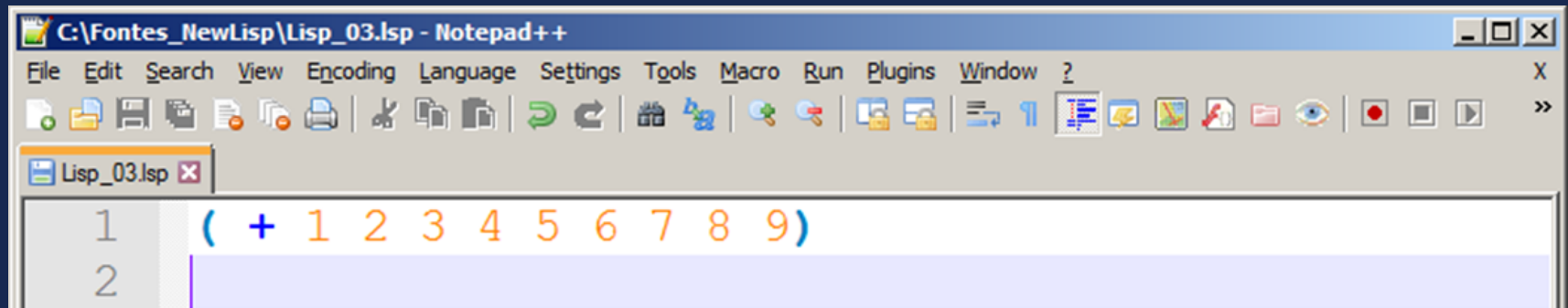
Exemplo – Lisp_02A.lsp

(max 1 4.5 9.88 12 999 45.000 65)



```
C:\> Command Prompt - newlisp "Lisp_2A.lsp")
>
>
>
>
>
>
> (load "newlisp_02A.lsp")
999
>
>
```

Exemplo – Lisp_03.lsp

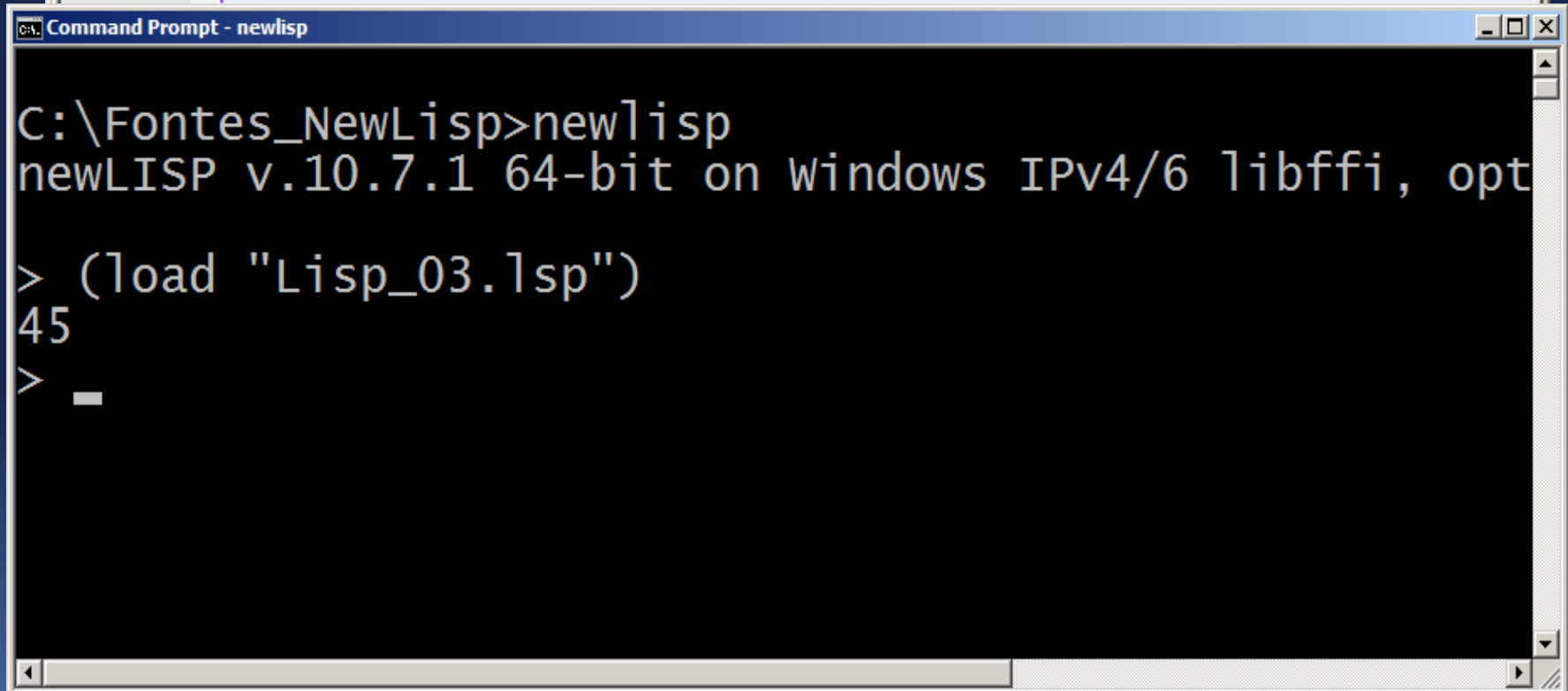


C:\Fontes_NewLisp\Lisp_03.lsp - Notepad++

File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?

Lisp_03.lsp

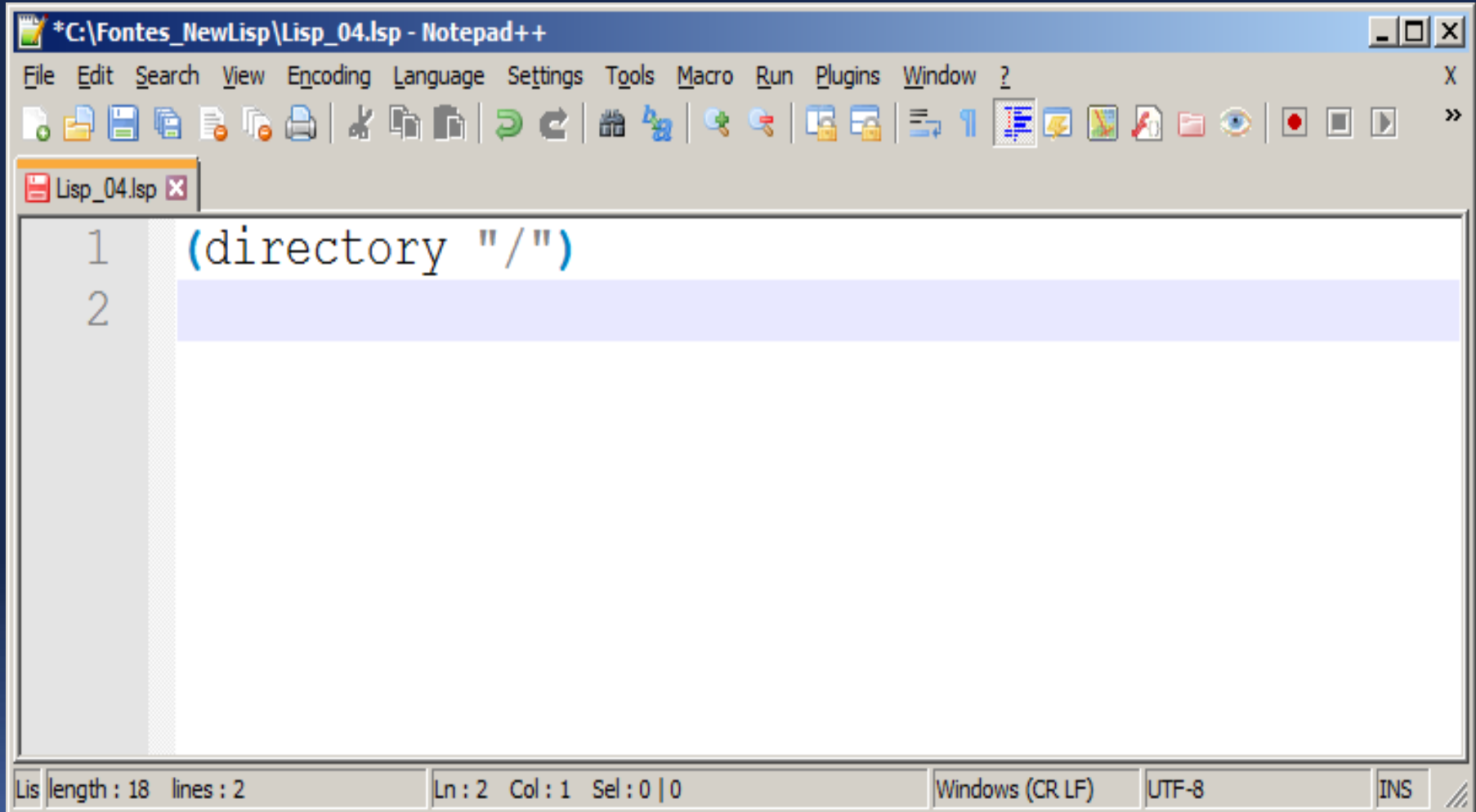
```
1  ( + 1 2 3 4 5 6 7 8 9 )
2
```



Command Prompt - newlisp

```
C:\Fontes_NewLisp>newlisp
newLISP v.10.7.1 64-bit on Windows IPv4/6 libffi, opt
> (load "Lisp_03.lsp")
45
> _
```

Exemplo – Lisp_04.lsp



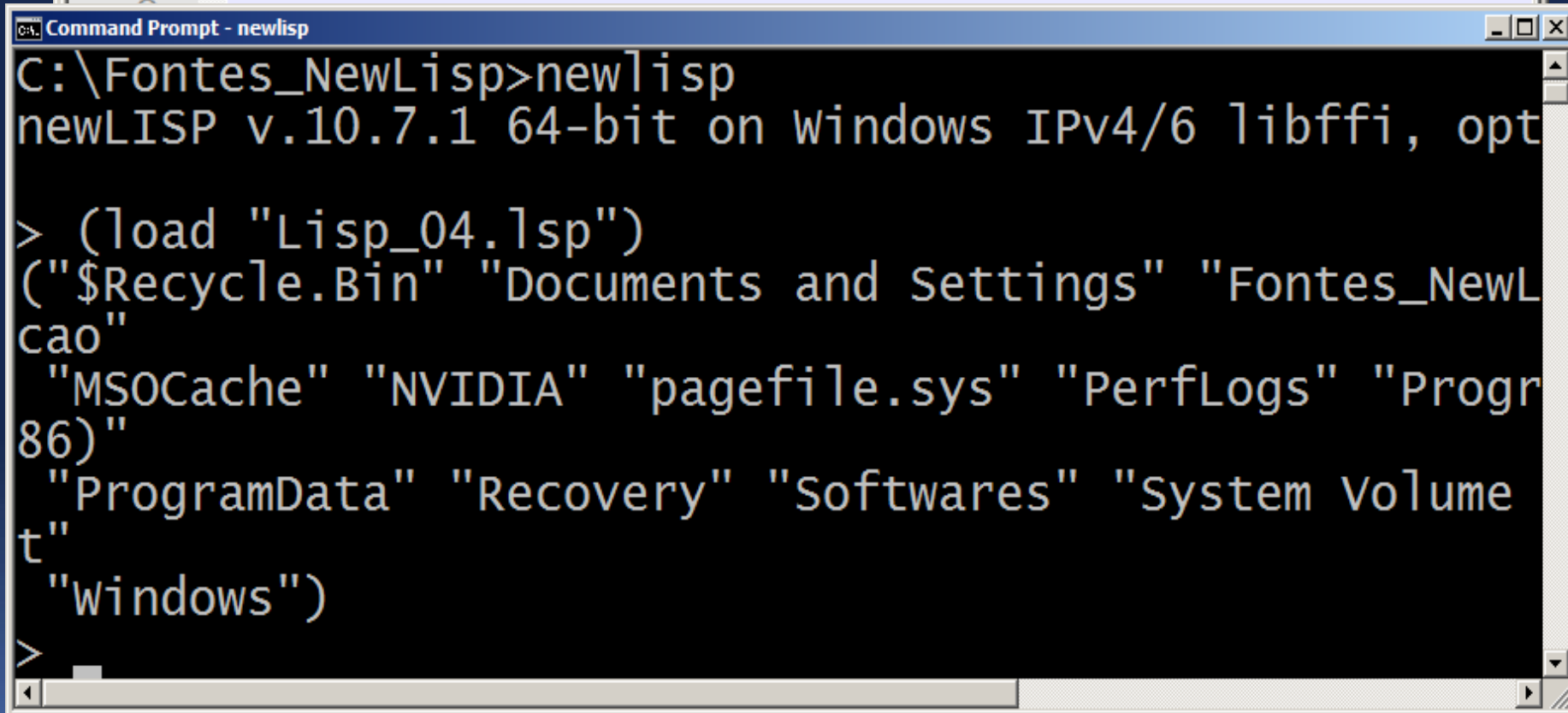
The image shows a Notepad++ window titled "*C:\Fontes_NewLisp\Lisp_04.lsp - Notepad++". The menu bar includes File, Edit, Search, View, Encoding, Language, Settings, Tools, Macro, Run, Plugins, Window, and Help. The toolbar contains various icons for file operations, editing, and development. The editor area shows a single line of code: `(directory "/")`. The status bar at the bottom displays "Ln : 2 Col : 1 Sel : 0 | 0", "Windows (CR LF)", "UTF-8", and "INS".

```
1 (directory "/")
```

Exemplo – Lisp_04.lsp

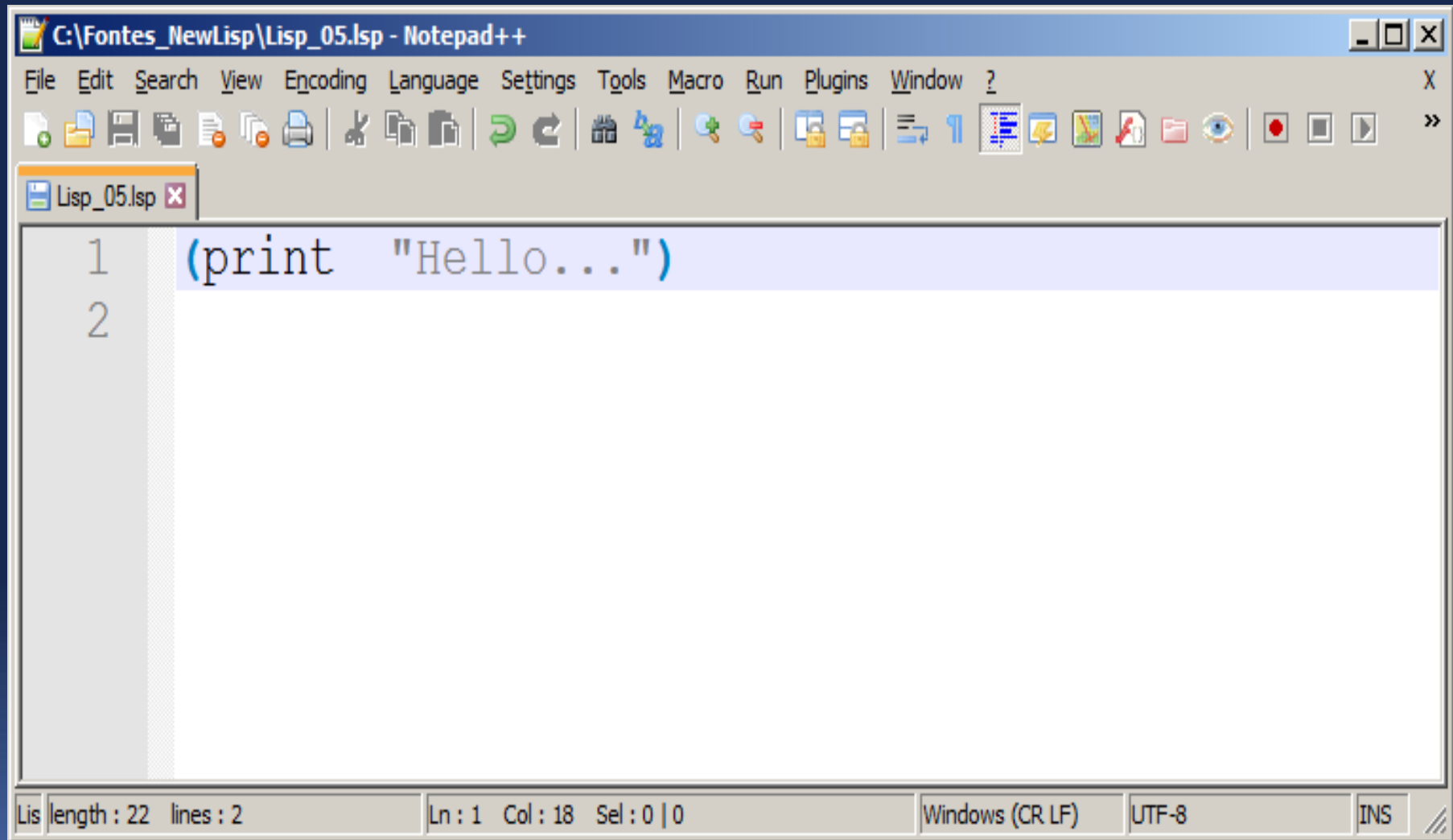
" data-bbox="56 140 969 346"/>

```
*C:\Fontes_NewLisp\Lisp_04.lsp - Notepad++  
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?  
Lisp_04.lsp  
1 (directory "/"
```



```
C:\Fontes_NewLisp>newlisp  
newLISP v.10.7.1 64-bit on Windows IPv4/6 libffi, opt  
> (load "Lisp_04.lsp")  
("$Recycle.Bin" "Documents and Settings" "Fontes_NewL  
cao"  
"MSOCache" "NVIDIA" "pagefile.sys" "PerfLogs" "Progr  
86)"  
"ProgramData" "Recovery" "Softwares" "System Volume  
t"  
"Windows")  
>
```

Exemplo – Lisp_05.lsp



C:\Fontes_NewLisp\Lisp_05.lsp - Notepad++

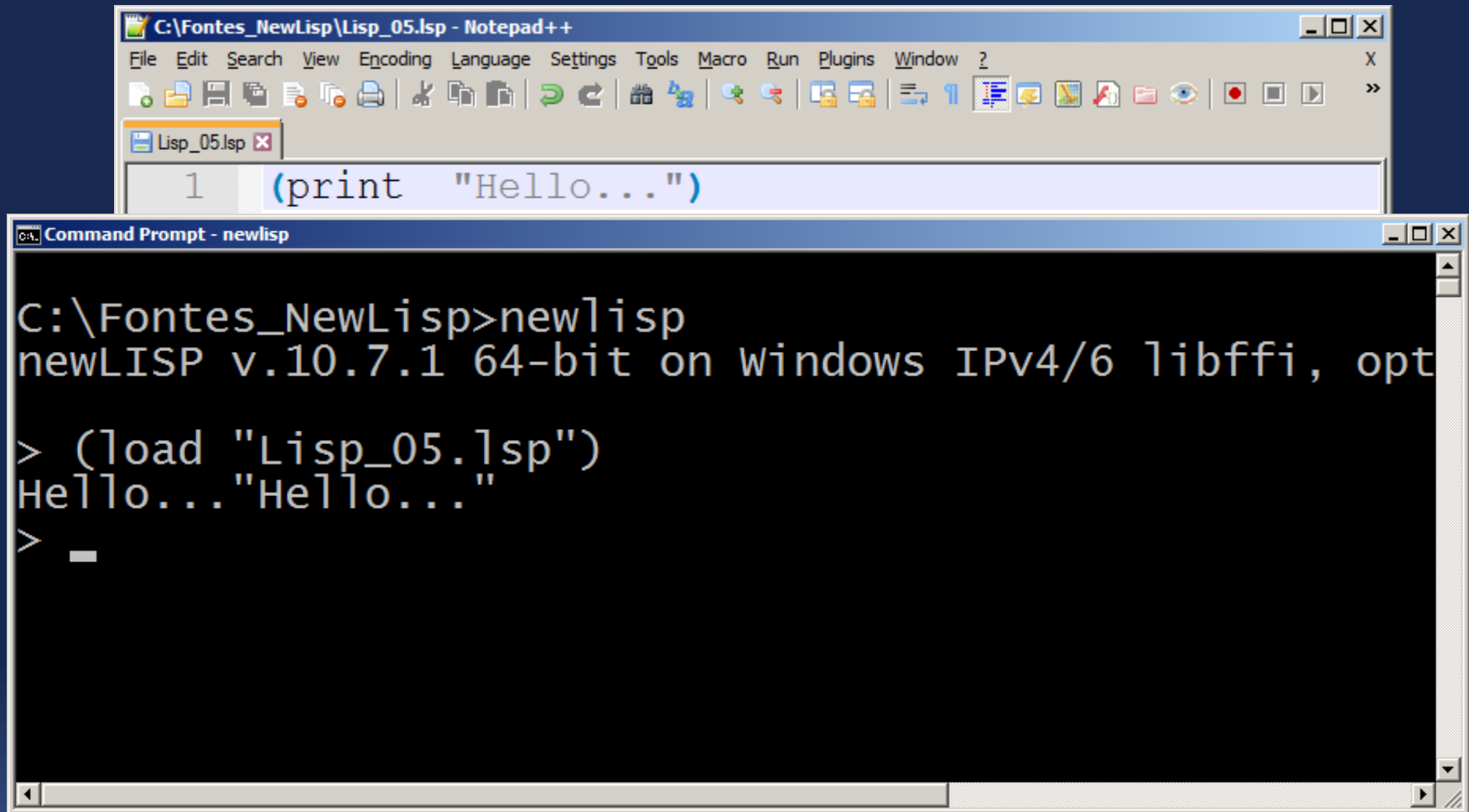
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?

Lisp_05.lsp

```
1 (print "Hello...")
2
```

Lisp length : 22 lines : 2 Ln : 1 Col : 18 Sel : 0 | 0 Windows (CR LF) UTF-8 INS

Exemplo – Lisp_05.lsp



The image shows two overlapping windows. The top window is Notepad++ with the file 'C:\Fontes_NewLisp\Lisp_05.lsp' open. The code in the editor is: `(print "Hello...")`. The bottom window is a Command Prompt titled 'newlisp'. It shows the command `newlisp` being executed, followed by the version information: `newLISP v.10.7.1 64-bit on Windows IPv4/6 libffi, opt`. Then, the command `> (load "Lisp_05.lsp")` is entered, and the output is `Hello..."Hello..."`. The prompt ends with `> _`.



Observação: Além de imprimir o string, a função também o retorna.

Exemplo – Leitura de Arquivo

(read-file "Arquivo.txt")

Ouviram do Ipiranga as margens plácidas
De um povo heroico o brado retumbante
E o sol da liberdade, em raios fúlgidos
Brilhou no céu da pátria nesse instante

Se o penhor dessa igualdade
Conseguimos conquistar com braço forte

```

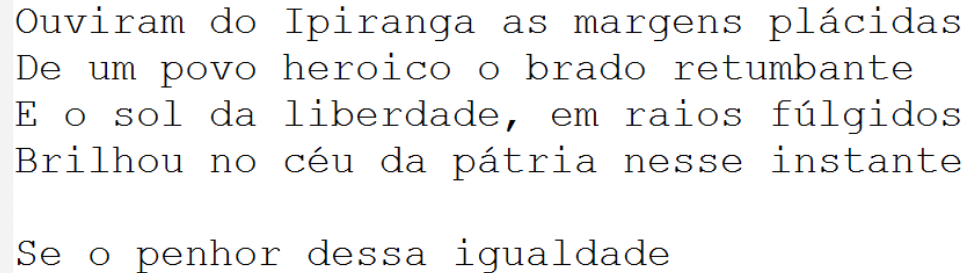
C:\> Command Prompt - newlisp

E:\NewLisp>newlisp
newLISP v.10.7.5 64-bit on windows IPv4/6 UTF-8 libffi, optio

> (read-file "Arquivo.txt")
"Ouviram do Ipiranga as margens pl|ícidas\r\nDe um povo heroi
te\r\nE o sol da liberdade, em raios f|l|gidos\r\nBrilhou no
se instante\r\n\r\nse o penhor dessa igualdade\r\nConseguimos
|ºo forte\r\nEm teu seio, | liberdade\r\nDesafia o nosso pei
!\r\n\r\n|ô P|ítria amada\r\nIdolatrada\r\nsalve! salve!\r\n\r
intenso, um raio v|ívido\r\nDe amor e de esperan|ºa |á terra
formoso c|u, risonho e l|impido\r\nA imagem do Cruzeiro res
ante pela pr|pria natureza\r\n|és belo, |s forte, imp|ívido
  
```

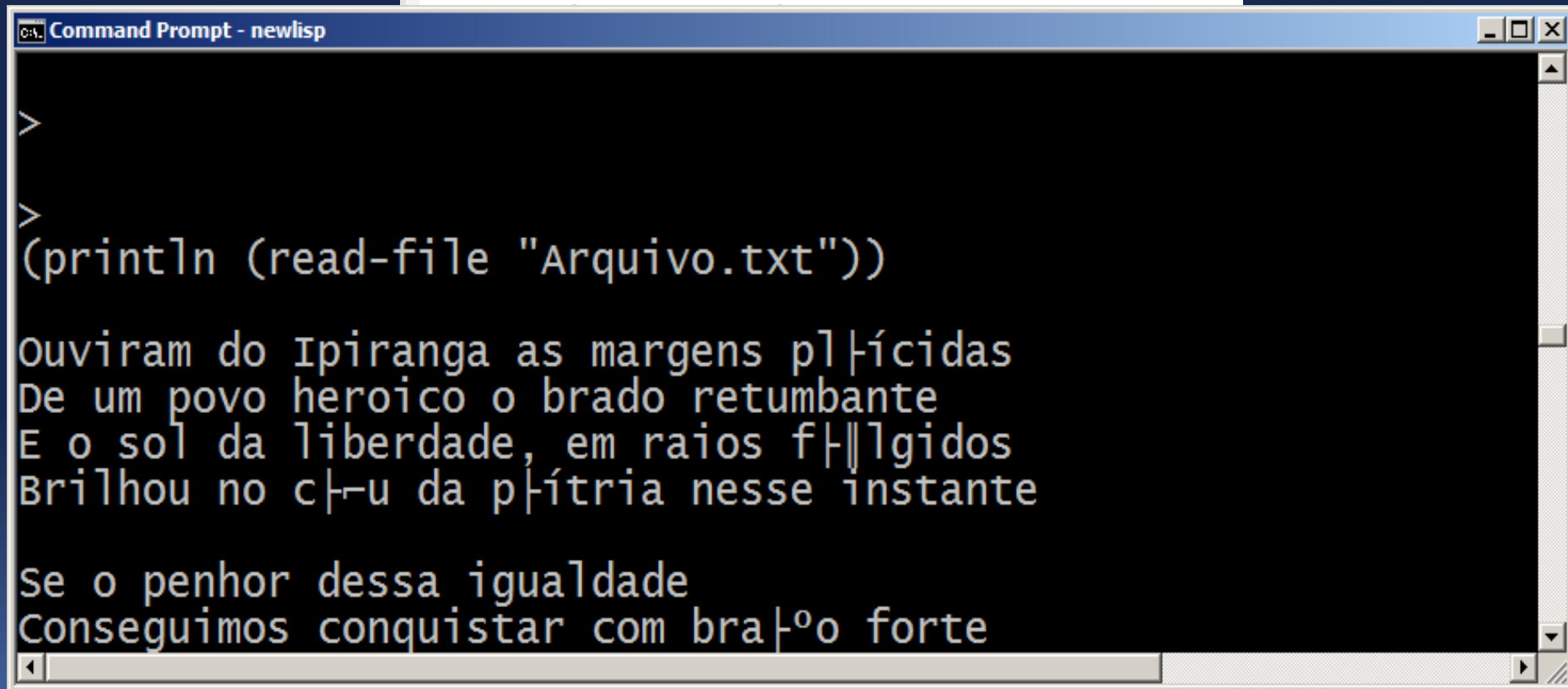
Exemplo – Leitura de Arquivo

```
( println ( read-file "Arquivo.txt") )
```



Ouviram do Ipiranga as margens plácidas
De um povo heroico o brado retumbante
E o sol da liberdade, em raios fúlgidos
Brilhou no céu da pátria nesse instante

Se o penhor dessa igualdade



```
Command Prompt - newlisp

>
>
(println (read-file "Arquivo.txt"))

Ouviram do Ipiranga as margens plácidas
De um povo heroico o brado retumbante
E o sol da liberdade, em raios fúlgidos
Brilhou no céu da pátria nesse instante

Se o penhor dessa igualdade
Conseguimos conquistar com braço forte
```

Funções newLISP

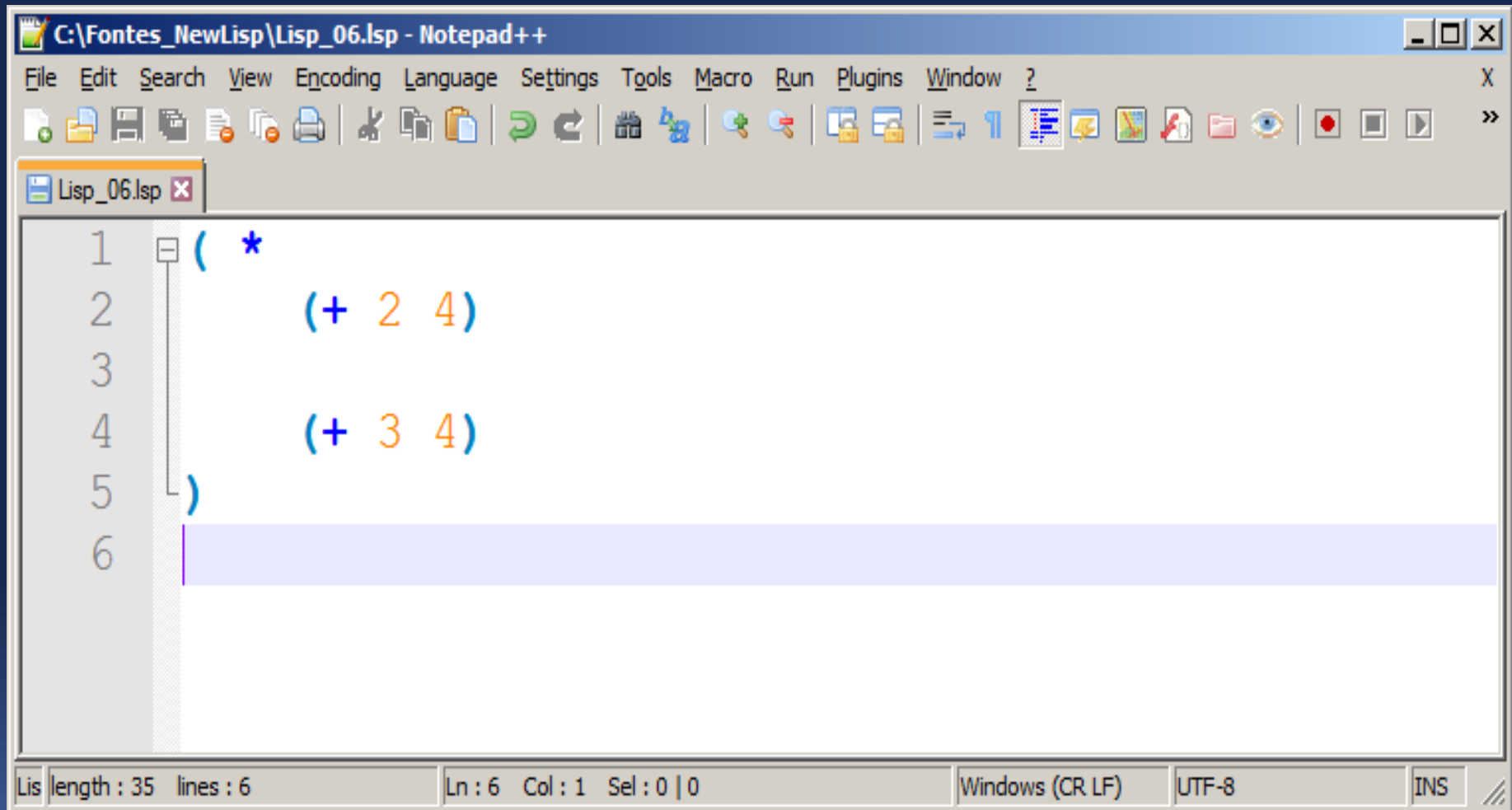
- ✓ Há cerca de 380 funções newLISP;
- ✓ Toda função newLISP retorna um valor;
- ✓ Mesmo um print retorna um valor;
- ✓ Diz-se que a ação de imprimir um valor é um “**side-effect**”;
- ✓ Assim, quando se executa a função print serão vistos dois valores: um que é o valor impresso e outro o valor retornado.



Listas Aninhadas

- ✓ Listas podem conter elementos que também são listas;
- ✓ Exemplo: $(* (+ 2 4) (+ 3 4))$
- ✓ newLISP avalia inicialmente a função $(+ 2 4)$ o qual retorna 6;
- ✓ Em seguida avalia a função $(+ 3 4)$ o qual retorna 7;
- ✓ Finalmente a expressão $(* 6 7)$ é avaliada, retornando o valor 42.

Exemplo – Lisp_06.lsp

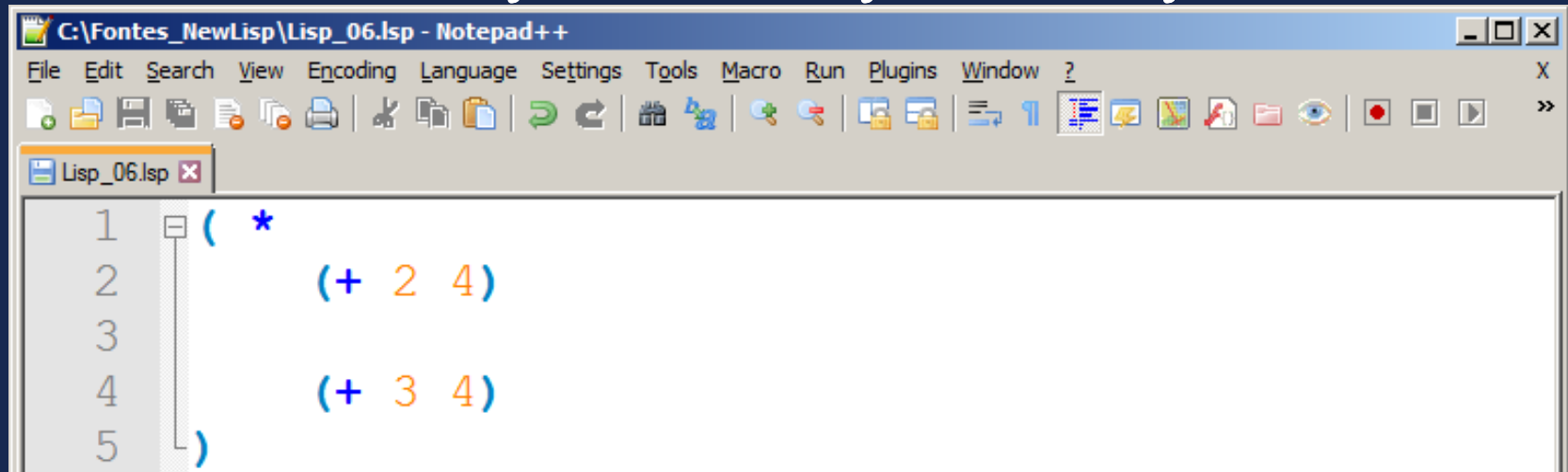


The screenshot shows a Notepad++ window titled "C:\Fontes_NewLisp\Lisp_06.lsp - Notepad++". The menu bar includes File, Edit, Search, View, Encoding, Language, Settings, Tools, Macro, Run, Plugins, Window, and ?. The toolbar contains various icons for file operations, editing, and development. The editor has a single tab for "Lisp_06.lsp". The code is as follows:

```
1  ( *  
2      (+ 2 4)  
3  
4      (+ 3 4)  
5  )  
6
```

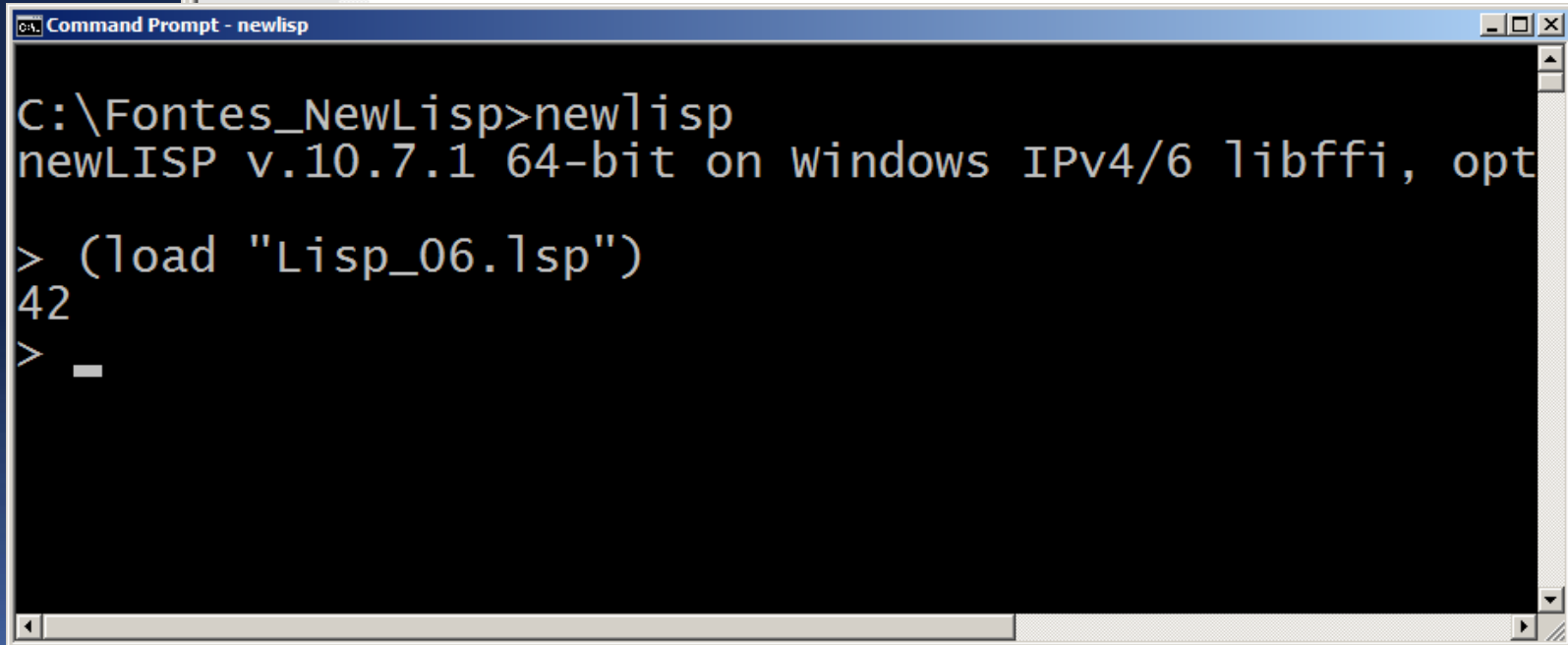
The status bar at the bottom displays: "Lis length : 35 lines : 6", "Ln : 6 Col : 1 Sel : 0 | 0", "Windows (CR LF)", "UTF-8", and "INS".

Exemplo – Lisp_06.lsp



C:\Fontes_NewLisp\Lisp_06.lsp - Notepad++

```
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
Lisp_06.lsp
1 ( *
2   (+ 2 4)
3
4   (+ 3 4)
5 )
```



Command Prompt - newlisp

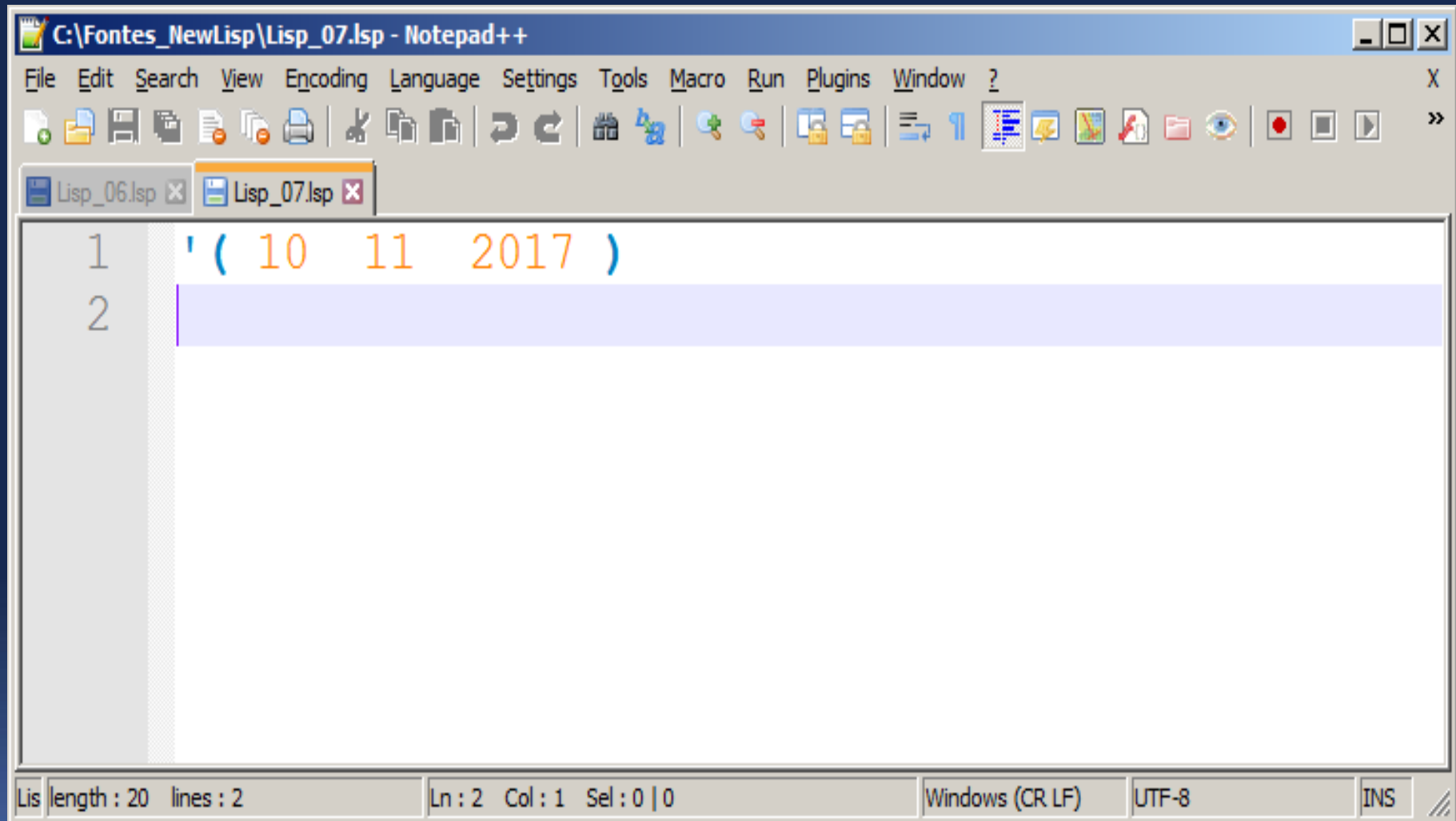
```
C:\Fontes_NewLisp>newlisp
newLISP v.10.7.1 64-bit on windows IPv4/6 libffi, opt
> (load "Lisp_06.lsp")
42
> _
```

As três regras básicas de newLISP

Regra 3

- ✓ Apóstrofe inibe a avaliação da expressão;
- ✓ Assim, `(+ 2 3)` será avaliada pelo interpretador newLISP retornando o valor 5;
- ✓ A expressão `'(+ 2 3)` é precedida por `'` o qual previne a avaliação. Nesse caso, a expressão será retornada sem processar a avaliação;
- ✓ Esse procedimento permite que a **lista** seja **tratada** como **dado** e não como chamada de função para ser avaliada;
- ✓ Exemplo: `(31 11 2017)` representa uma data e portanto deve ser escrita `'(31 11 2017)`.

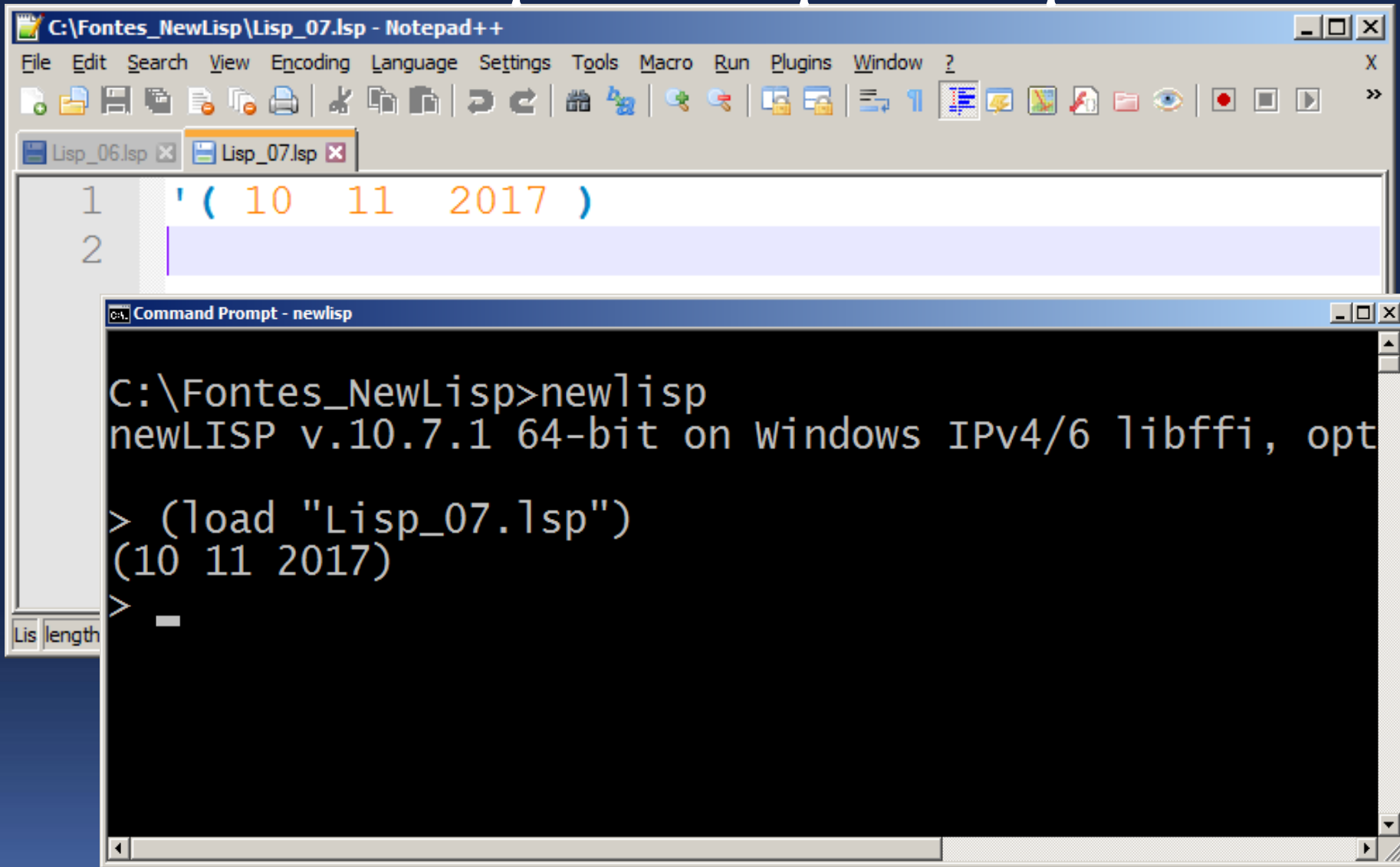
Exemplo – Lisp_07.lsp



The screenshot shows a Notepad++ window titled "C:\Fontes_NewLisp\Lisp_07.lsp - Notepad++". The menu bar includes File, Edit, Search, View, Encoding, Language, Settings, Tools, Macro, Run, Plugins, Window, and ?. The toolbar contains various icons for file operations and editing. The tab bar shows two tabs: "Lisp_06.lsp" and "Lisp_07.lsp", with the latter being the active tab. The editor area displays two lines of code: line 1 contains the expression `' ( 10 11 2017 )` and line 2 is empty. The status bar at the bottom shows "Lis length : 20 lines : 2", "Ln : 2 Col : 1 Sel : 0 | 0", "Windows (CR LF)", "UTF-8", and "INS".

```
1 ' ( 10 11 2017 )
2
```

Exemplo – Lisp_07.lsp



The image shows a Notepad++ window titled "C:\Fontes_NewLisp\Lisp_07.lsp - Notepad++" with the following code:

```
1 ' ( 10 11 2017 )
2
```

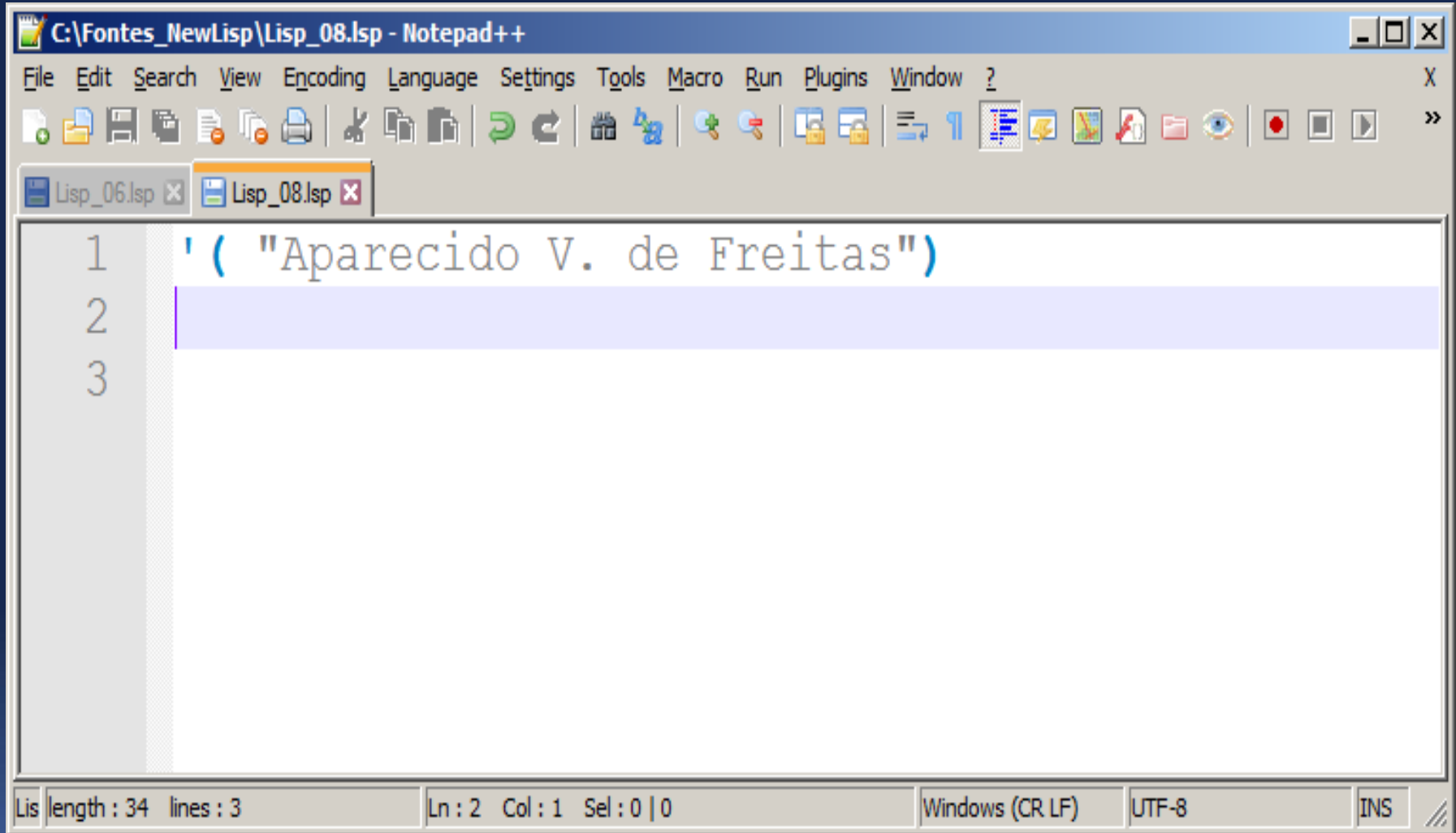
Below the Notepad++ window is a Command Prompt window titled "Command Prompt - newlisp". It shows the following commands and output:

```
C:\Fontes_NewLisp>newlisp
newLISP v.10.7.1 64-bit on windows IPv4/6 libffi, opt

> (load "Lisp_07.lsp")
(10 11 2017)
> _
```

At the bottom left of the Command Prompt window, there is a small text box containing the text "Lis | length".

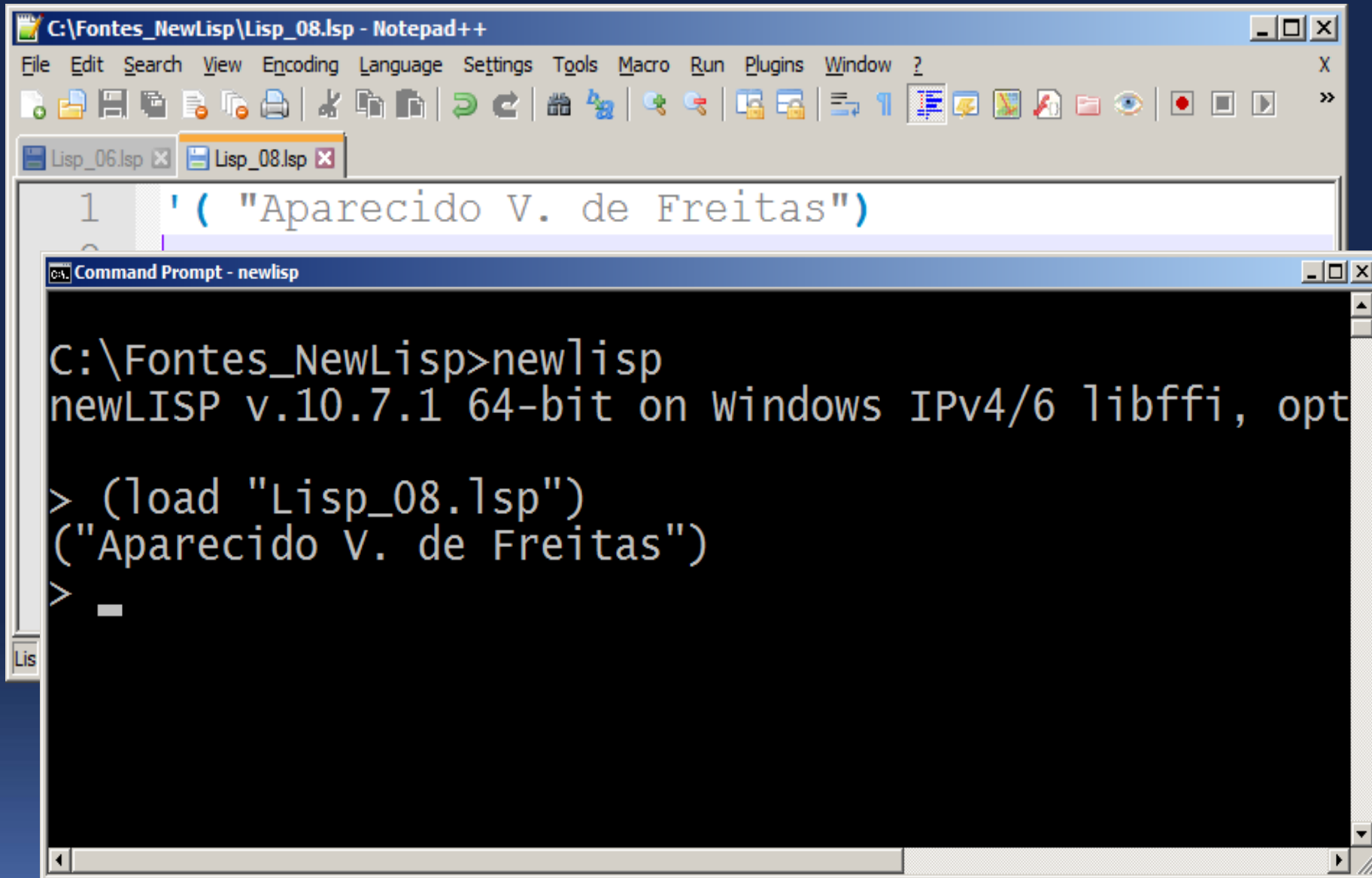
Exemplo – Lisp_08.lsp



The image shows a Notepad++ window titled "C:\Fontes_NewLisp\Lisp_08.lsp - Notepad++". The menu bar includes File, Edit, Search, View, Encoding, Language, Settings, Tools, Macro, Run, Plugins, Window, and ?. The toolbar contains various icons for file operations, editing, and development. The tab bar shows two tabs: "Lisp_06.lsp" and "Lisp_08.lsp", with the latter being the active tab. The editor area contains three lines of code: line 1 has the text `' ( "Aparecido V. de Freitas")`, line 2 is empty, and line 3 is empty. The status bar at the bottom displays "Lis length : 34 lines : 3", "Ln : 2 Col : 1 Sel : 0 | 0", "Windows (CR LF)", "UTF-8", and "INS".

```
1 ' ( "Aparecido V. de Freitas")
2
3
```

Exemplo – Lisp_08.lsp



```
C:\Fontes_NewLisp\Lisp_08.lsp - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
Lisp_06.lsp x Lisp_08.lsp x
1 ( "Aparecido V. de Freitas")

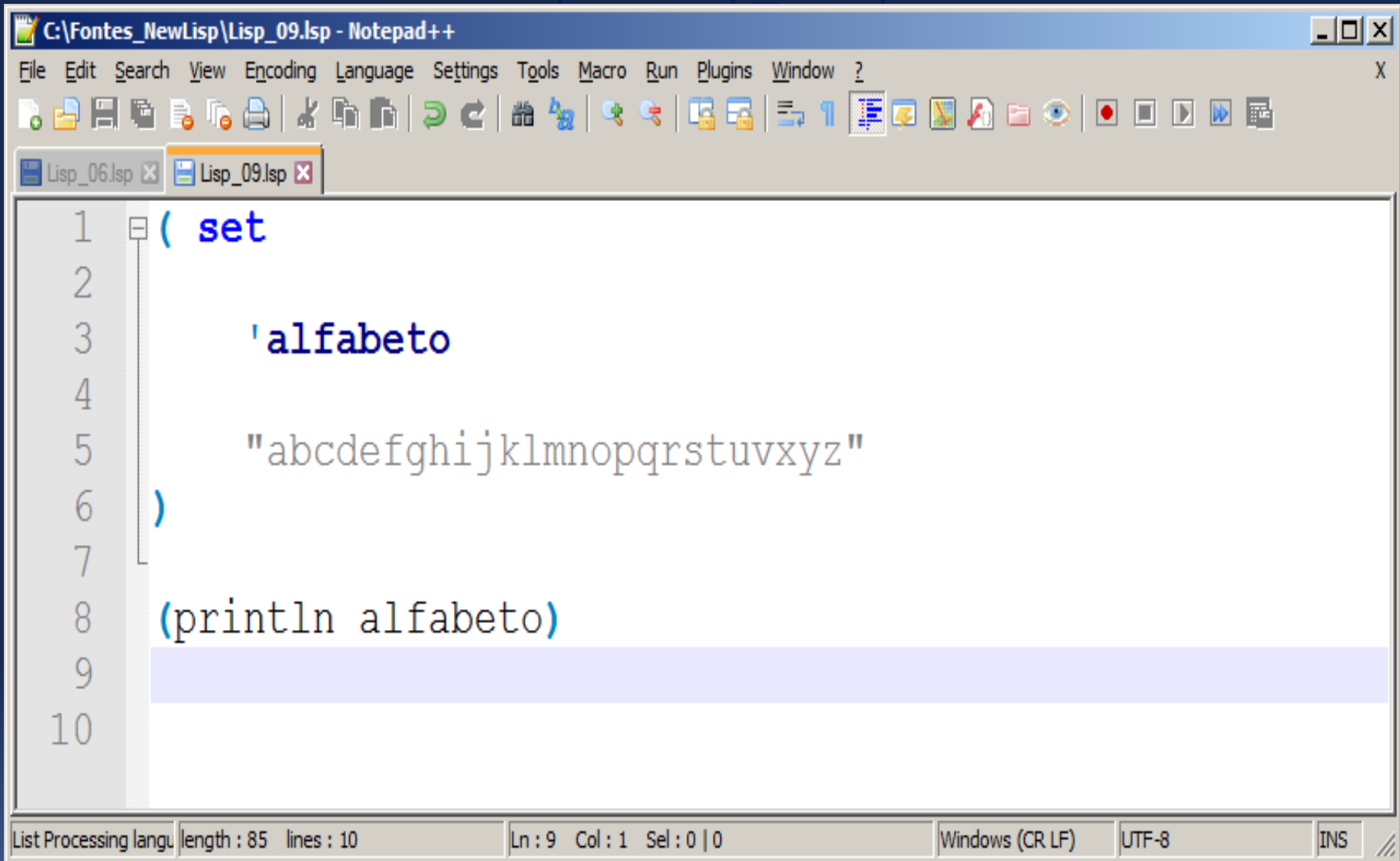
C:\Fontes_NewLisp>newlisp
newLISP v.10.7.1 64-bit on Windows IPv4/6 libffi, opt

> (load "Lisp_08.lsp")
("Aparecido V. de Freitas")
> _
```

Símbolos

- ✓ Em **newLISP**, um **símbolo** representa algo associado a um **nome**;
- ✓ Pode-se definir esse **nome** representando alguma coisa no código e, posteriormente, usar-se o nome ao invés de seu conteúdo;
- ✓ Por exemplo: (set 'alfabeto "abcdefghijklmnopqrstuvwxyz")
- ✓ No exemplo, definiu-se por meio da função set um nome chamado 'alfabeto cujo valor é o string que contém as 26 letras do alfabeto;
- ✓ A partir dessa definição, o símbolo alfabeto pode ser usado em qualquer ponto do programa e será avaliado sempre que for chamado;

Exemplo – Lisp 09.lsp

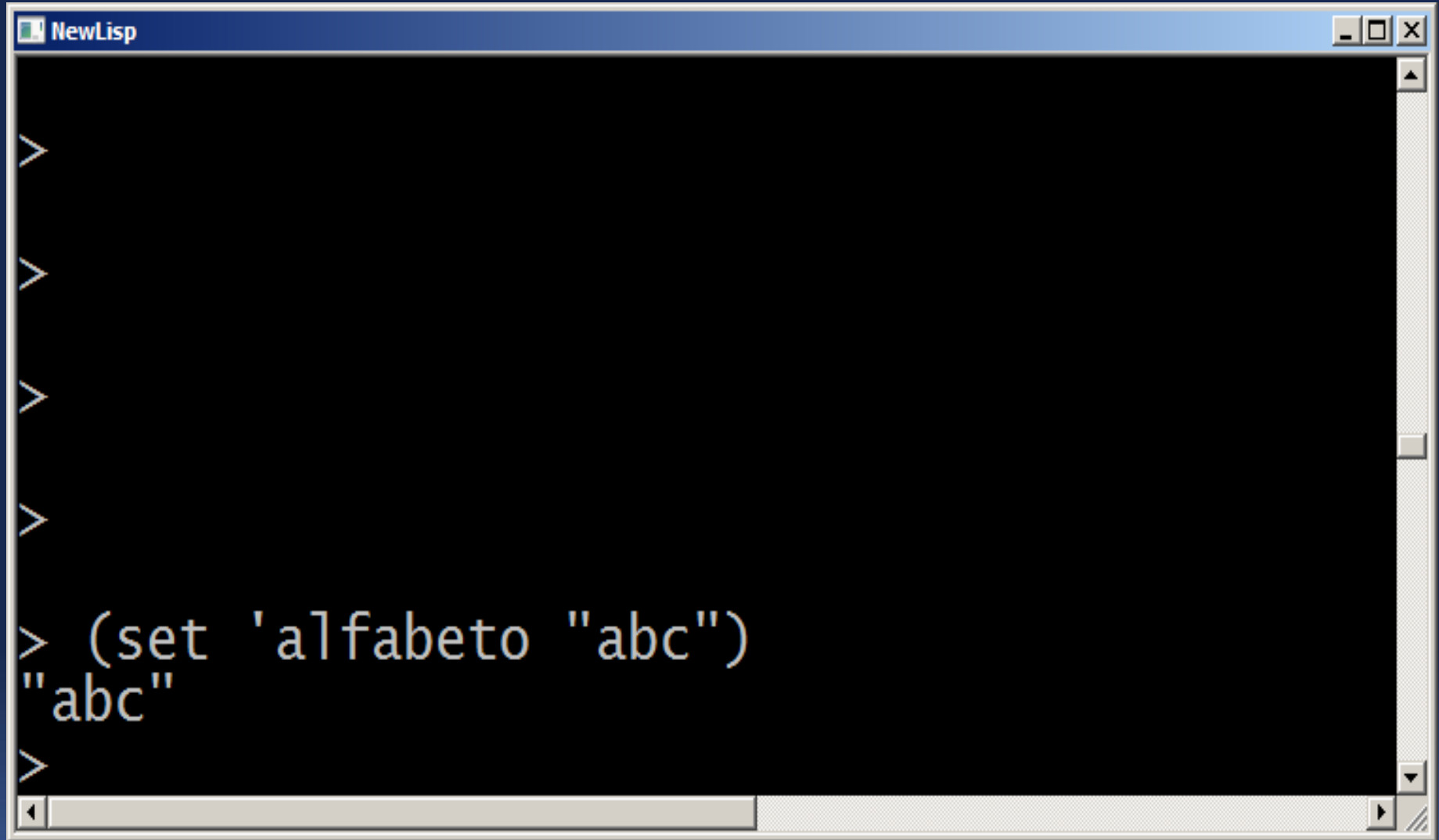


The screenshot shows a Notepad++ window titled "C:\Fontes_NewLisp\Lisp_09.lsp - Notepad++". The menu bar includes File, Edit, Search, View, Encoding, Language, Settings, Tools, Macro, Run, Plugins, Window, and ?. The toolbar contains various icons for file operations, editing, and running. The tab bar shows two open files: "Lisp_06.lsp" and "Lisp_09.lsp". The main text area contains the following Lisp code:

```
1 ( set
2
3     'alfabeto
4
5     "abcdefghijklmnopqrstuvxyz"
6 )
7
8 (println alfabeto)
9
10
```

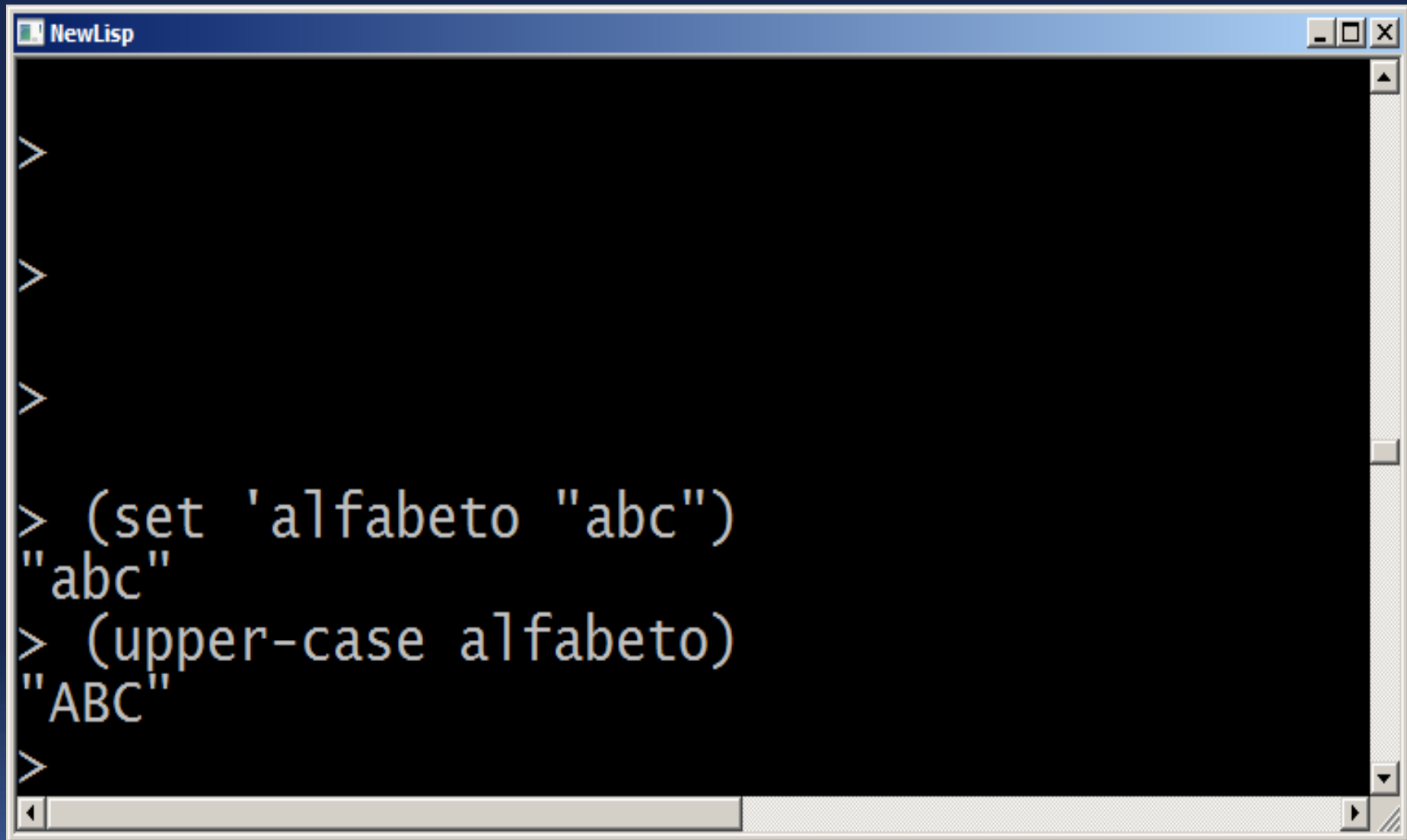
The status bar at the bottom displays: "List Processing langu", "length : 85 lines : 10", "Ln : 9 Col : 1 Sel : 0 | 0", "Windows (CR LF)", "UTF-8", and "INS".

Exemplo – Lisp_10.lsp

A screenshot of a NewLisp window. The window has a title bar with the text "NewLisp" and standard window controls (minimize, maximize, close). The main area is black with white text. On the left side, there are four right-pointing chevrons (>) stacked vertically. Below them, the text "(set 'alfabeto \"abc\")" is displayed on one line, and "\"abc\"" is displayed on the line below it. At the bottom left, there is another right-pointing chevron (>). A horizontal scrollbar is visible at the bottom of the window.

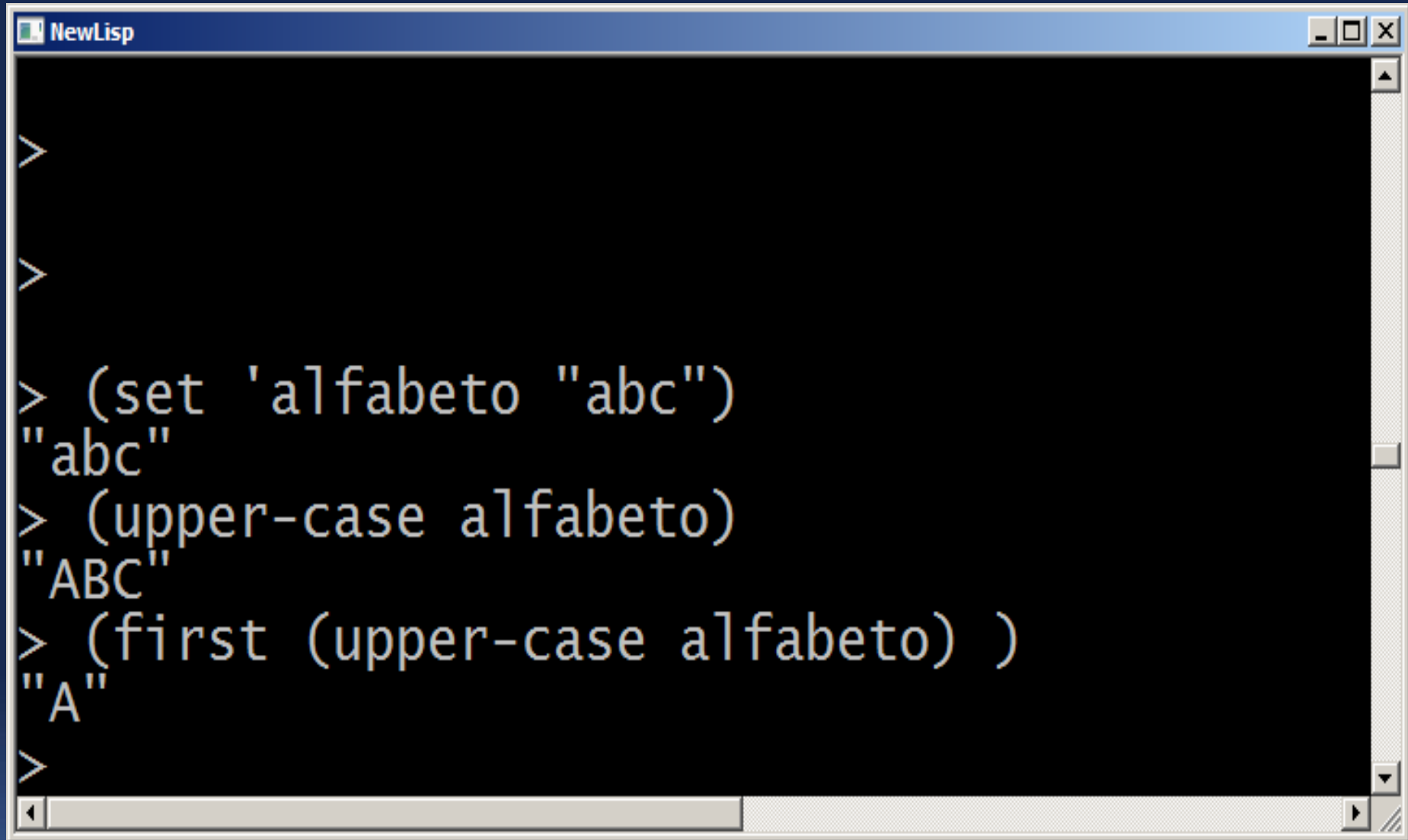
```
>  
>  
>  
>  
> (set 'alfabeto "abc")  
"abc"  
>
```

Exemplo – Lisp_10.lsp

A screenshot of a NewLisp window. The window has a title bar with the text "NewLisp" and standard window controls (minimize, maximize, close). The main area is a black text editor with white text. It shows a series of prompts ">" followed by Lisp code and its output. The code defines a variable 'alfabeto' with the value "abc" and then uses the 'upper-case' function to convert it to "ABC".

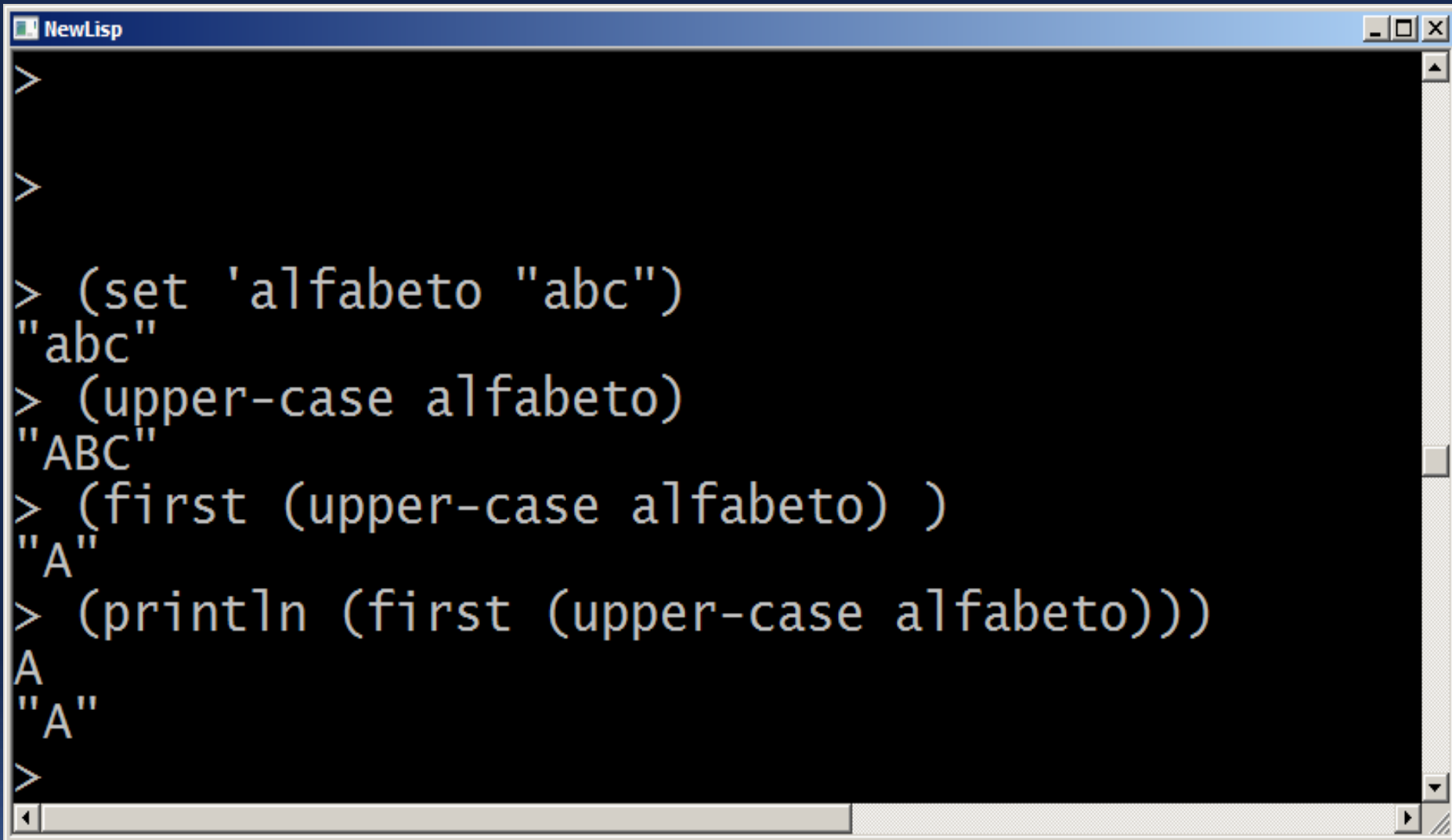
```
>  
>  
>  
> (set 'alfabeto "abc")  
"abc"  
> (upper-case alfabeto)  
"ABC"  
>
```

Exemplo – Lisp_10.lsp

A screenshot of a NewLisp window titled "NewLisp". The window has a blue title bar with standard Windows window controls (minimize, maximize, close) on the right. The main area is a black text editor with white text. The text shows a series of Lisp commands and their results, with each command on a new line starting with a prompt character ">". The commands are: 1. A blank line. 2. A blank line. 3. (set 'alfabeto "abc") followed by the result "abc" on the next line. 4. (upper-case alfabeto) followed by the result "ABC" on the next line. 5. (first (upper-case alfabeto)) followed by the result "A" on the next line. 6. A blank line. The window has a horizontal scrollbar at the bottom and a vertical scrollbar on the right side.

```
>  
>  
> (set 'alfabeto "abc")  
"abc"  
> (upper-case alfabeto)  
"ABC"  
> (first (upper-case alfabeto) )  
"A"  
>
```

Exemplo – Lisp_11.lsp

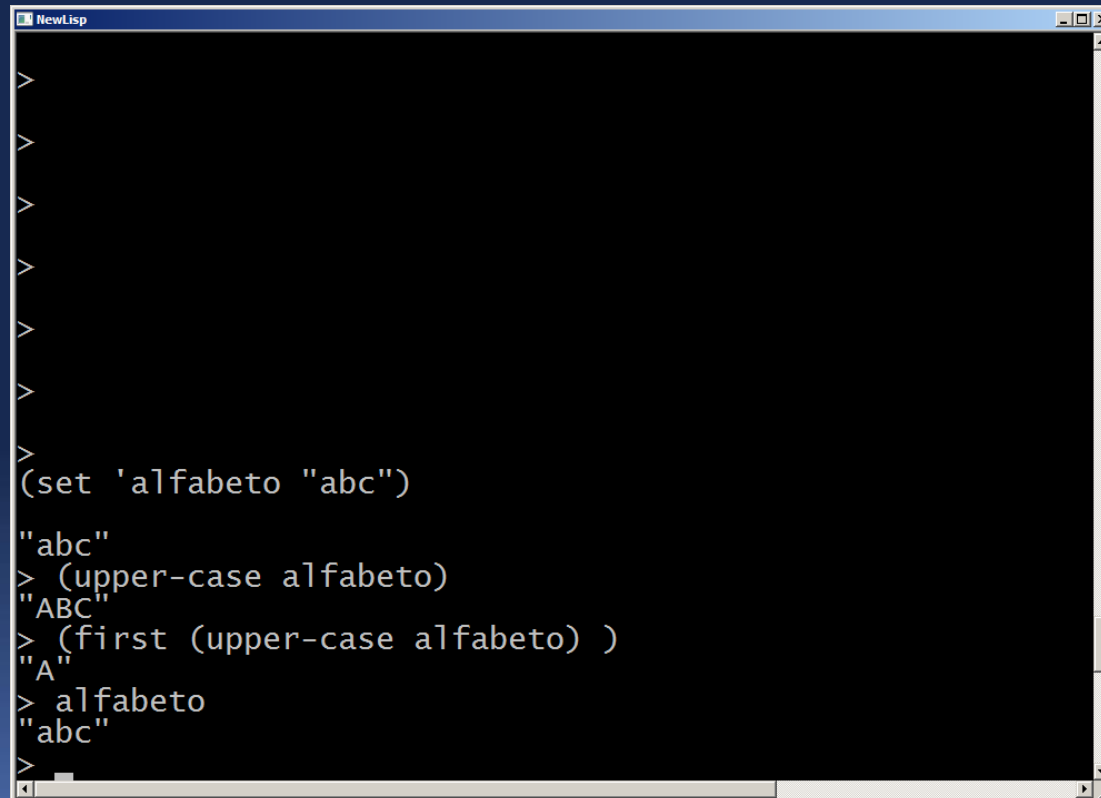


```
NewLisp
>
>
> (set 'alfabeto "abc")
"abc"
> (upper-case alfabeto)
"ABC"
> (first (upper-case alfabeto) )
"A"
> (println (first (upper-case alfabeto)))
A
"A"
>
```

Exemplo – Lisp 12.lisp

Observação

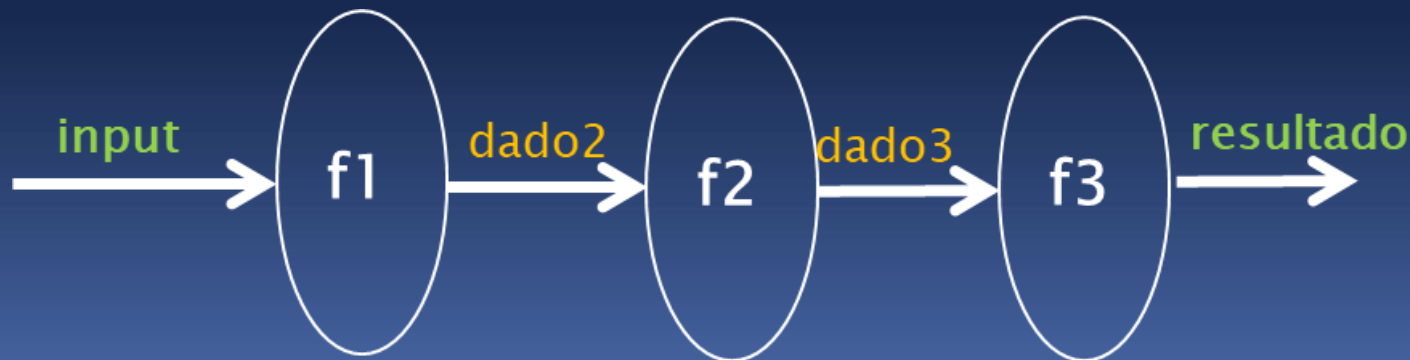
O valor do símbolo 'alfabeto não foi permanentemente alterado, ou seja permaneceu com os valores em minúsculo, pois a função upper-case criou e retornou um novo string com os valores em maiúsculo. (O símbolo 'alfabeto permaneceu inalterado)



```
NewLisp
>
>
>
>
>
>
>
>
> (set 'alfabeto "abc")
"abc"
> (upper-case alfabeto)
"ABC"
> (first (upper-case alfabeto) )
"A"
> alfabeto
"abc"
>
```

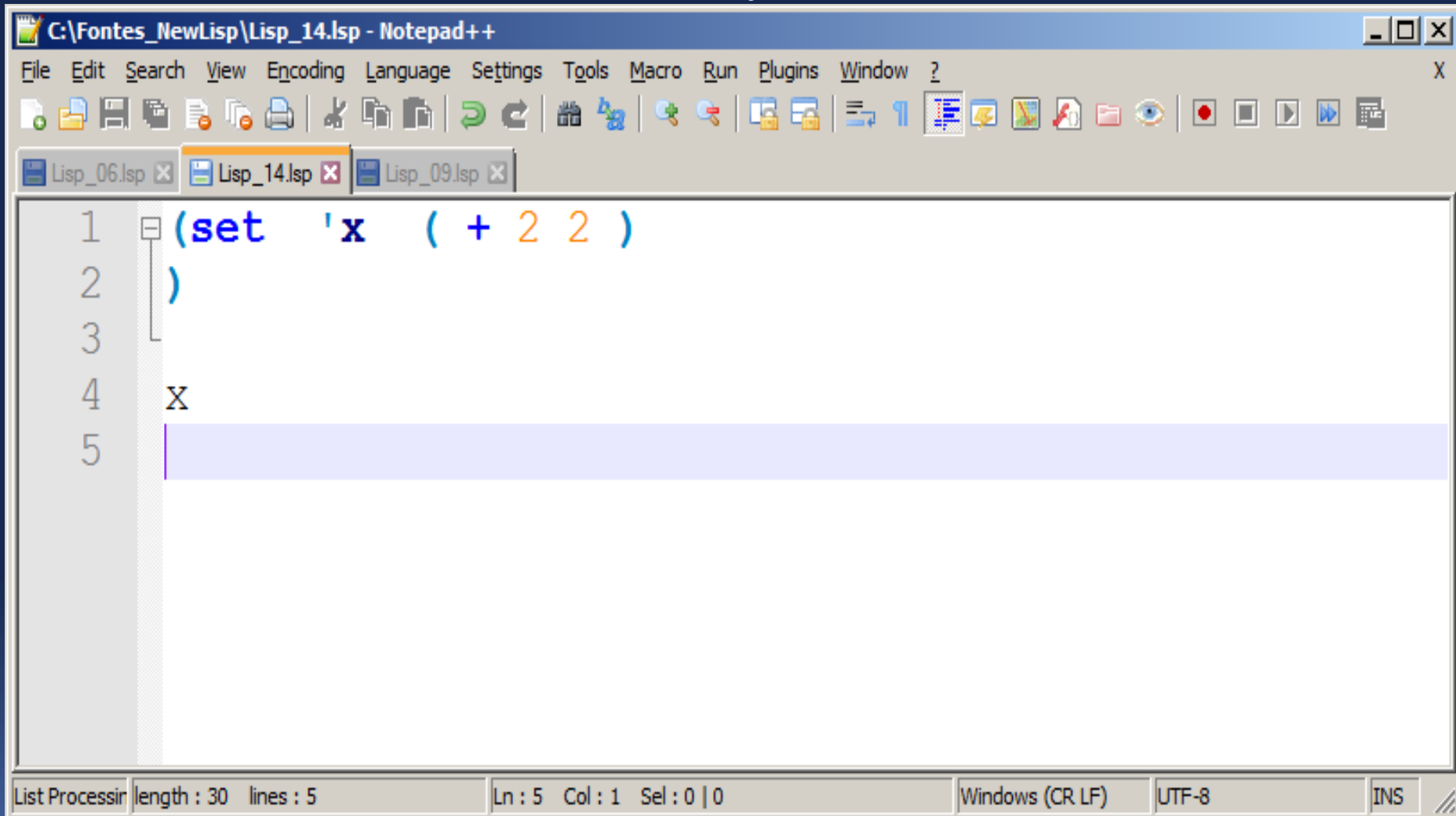
Símbolos – Mais observações

- ✓ **Símbolos** correspondem à "variáveis" em outras linguagens de programação;
- ✓ Na verdade, **newLISP** não usa símbolos de forma extensiva como se usa em outras linguagens de programação;
- ✓ A razão disto é que valores retornados por uma função estarão sendo continuamente entrados para outras funções, sem portanto a necessidade de armazená-los;
- ✓ No paradigma funcional o foco está na **transformação dos dados** de forma sucessiva até atingir-se o resultado final esperado.



Exemplo

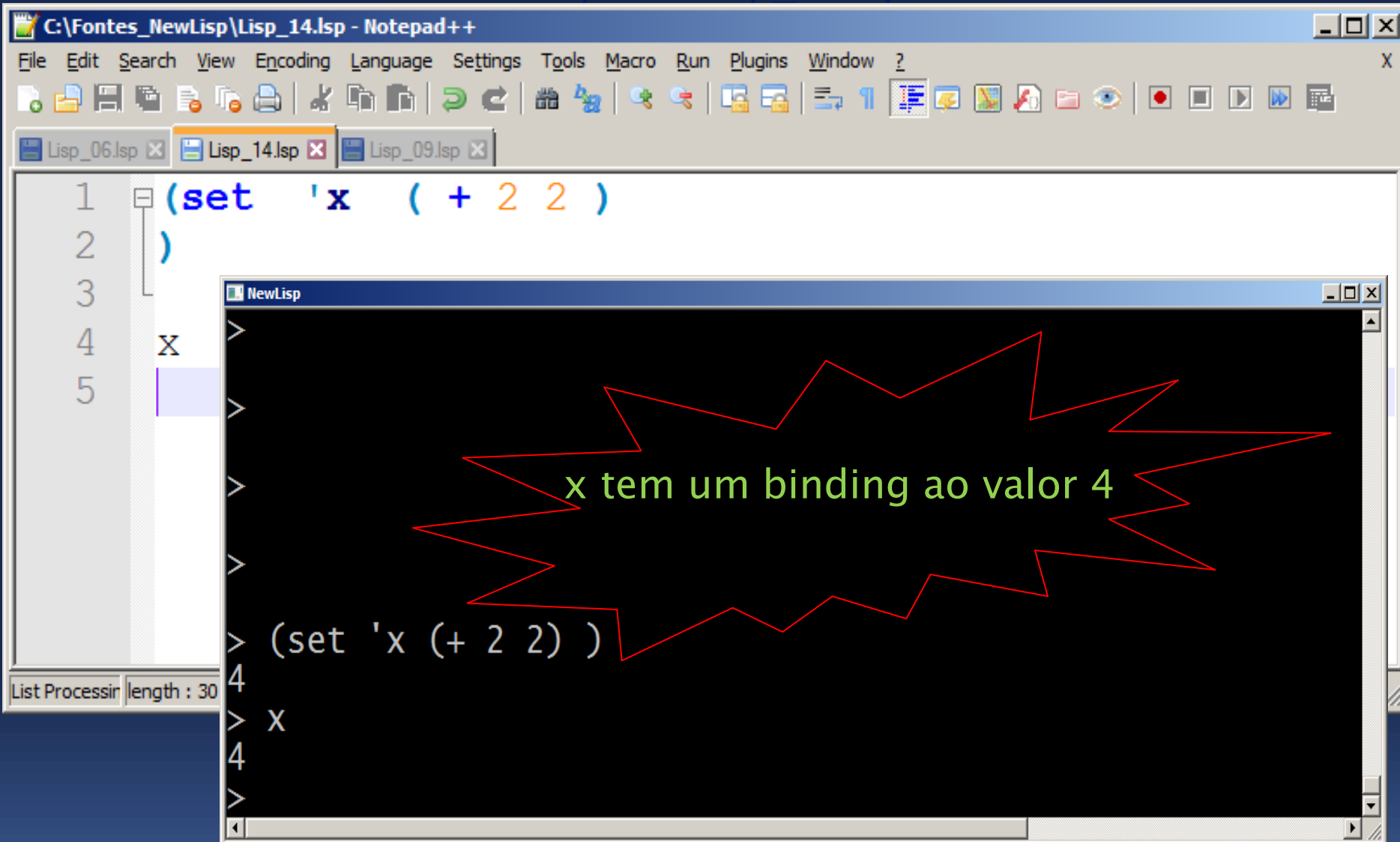
Exemplo – Lisp_14.lsp



```
1 (set 'x ( + 2 2 )  
2 )  
3  
4 x  
5
```

List Processor | length : 30 lines : 5 | Ln : 5 Col : 1 Sel : 0 | 0 | Windows (CR LF) UTF-8 INS

Exemplo – Lisp 14.lsp



The image shows a Notepad++ window titled "C:\Fontes_NewLisp\Lisp_14.lsp - Notepad++". The script contains the following code:

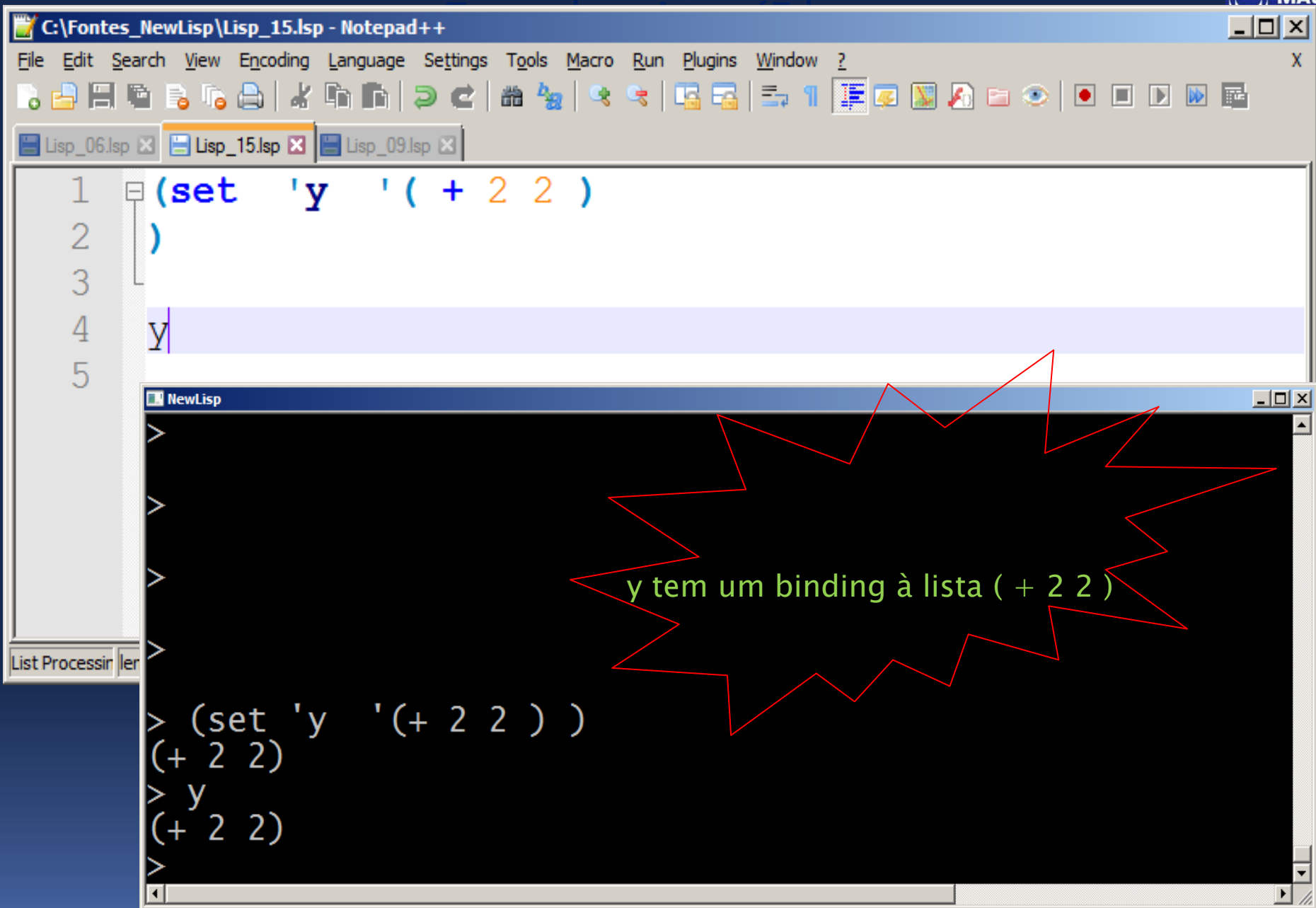
```
1 (set 'x (+ 2 2))  
2 )  
3  
4 x  
5
```

The variable `x` is highlighted in the editor. Below the editor, a status bar shows "List Processor length : 30".

Overlaid on the Notepad++ window is a NewLisp console window titled "NewLisp". It displays the following interaction:

```
>  
>  
>  
>  
>  
> (set 'x (+ 2 2))  
4  
> x  
4  
>
```

A red jagged starburst shape is drawn around the text "x tem um binding ao valor 4" in green, which is positioned between the two console prompts.



C:\Fontes_NewLisp\Lisp_15.lsp - Notepad++

File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?

Lisp_06.lsp x Lisp_15.lsp x Lisp_09.lsp x

```
1 (set 'y '( + 2 2 )
2 )
3
4 y
5
```

NewLisp

>

>

>

>

>

> (set 'y '(+ 2 2))

(+ 2 2)

> y

(+ 2 2)

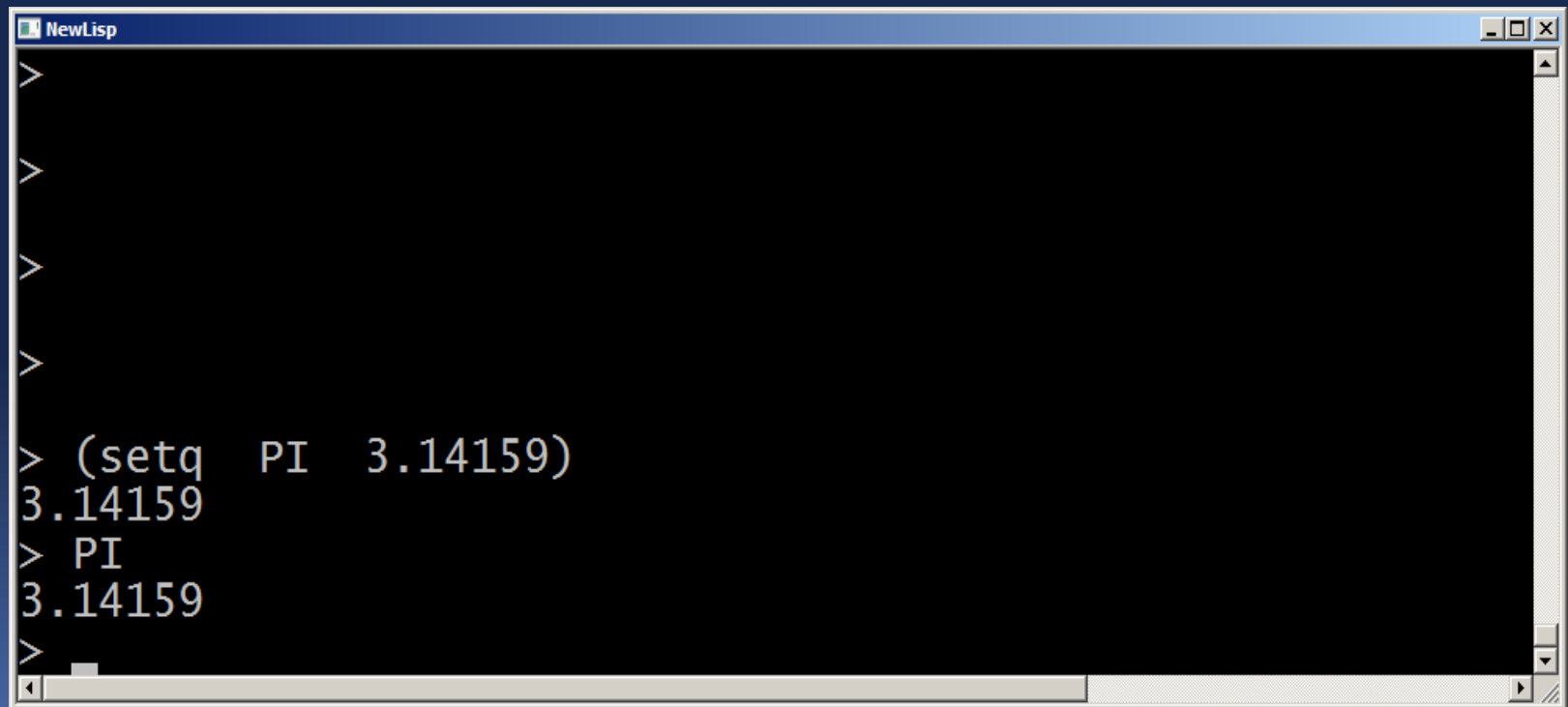
>

y tem um binding à lista (+ 2 2)

List Processor ler

Definindo símbolos

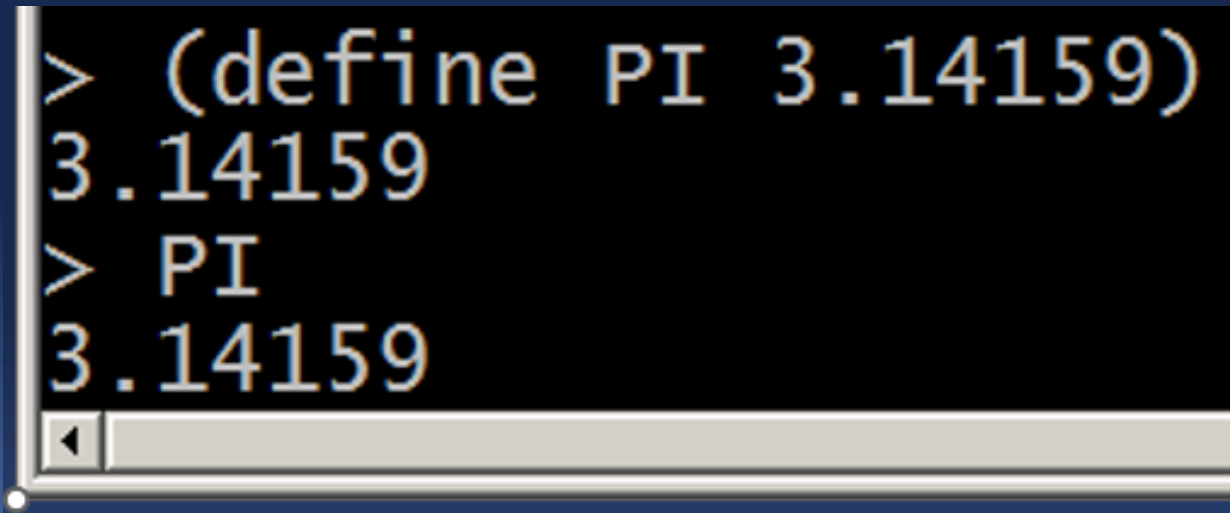
- ✓ Símbolos também podem ser definidos pela função `setq`;
- ✓ A função `setq` considera que primeiro argumento é um símbolo, e dessa forma, não se exige que o argumento seja `quoted` (com apóstrofe).



```
NewLisp
>
>
>
>
> (setq PI 3.14159)
3.14159
> PI
3.14159
>
```

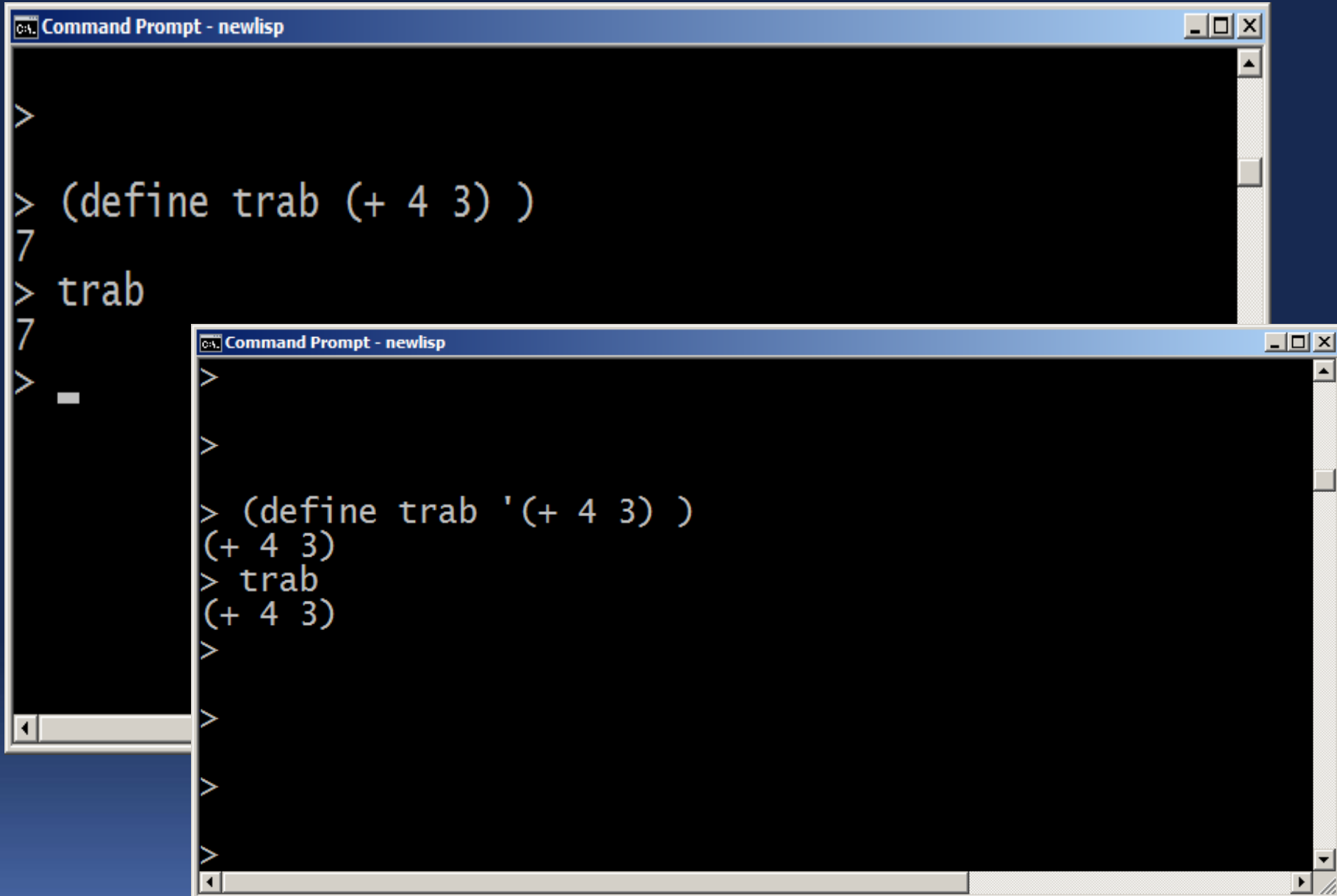
Definindo símbolos

- ✓ **Símbolos** também podem ser definidos pela função **define**;
- ✓ A função **set** vista anteriormente exige que o símbolo seja definido com **'**;
- ✓ Mas, **define** não exige que o argumento seja **quoted**;
- ✓ **define** também é usado para definição de funções.



```
> (define PI 3.14159)
3.14159
> PI
3.14159
```

Definindo símbolos com define



The image shows two overlapping Command Prompt windows titled "Command Prompt - newlisp".

The top window shows the following commands and output:

```
>  
> (define trab (+ 4 3) )  
7  
> trab  
7  
> _
```

The bottom window shows the following commands and output:

```
>  
>  
> (define trab '(+ 4 3) )  
(+ 4 3)  
> trab  
(+ 4 3)  
>  
>  
>
```

Controlando o Fluxo de Execução

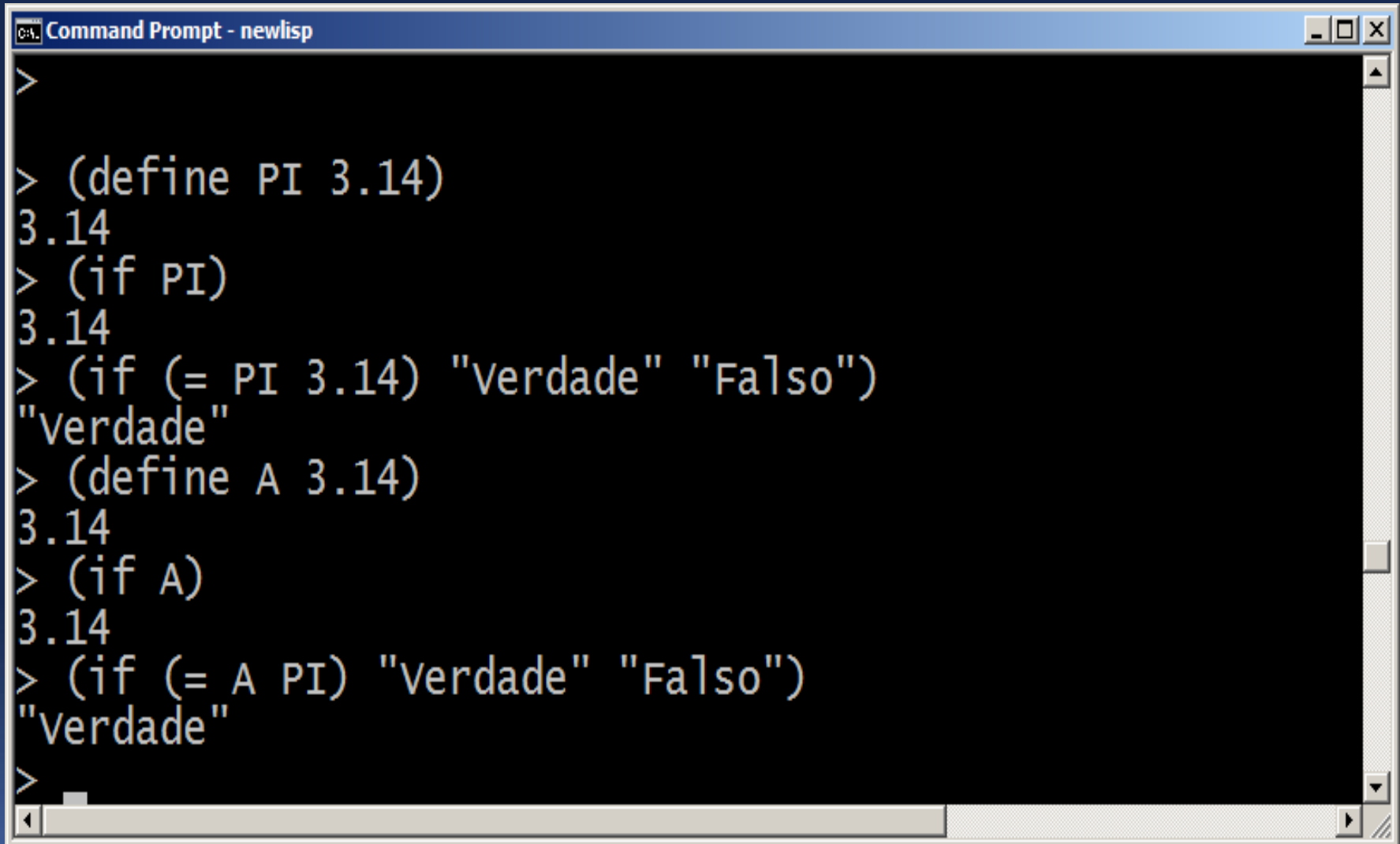
- ✓ Há diferentes formas de se controlar o Fluxo de Execução de nosso código;
- ✓ Todo o controle de fluxo em newLISP seguem determinadas regras da linguagem;
- ✓ A forma geral de definir o controle de fluxo é por meio de uma lista no qual o primeiro elemento é uma **keyword** (palavra reservada) seguida por uma ou mais expressões para serem avaliadas.

```
( keyword      expressao2      expressao3 ... )
```

if

- ✓ Consiste de um teste e uma ação;
- ✓ A grande maioria das coisas já conhecidas em **newLISP** são **true**;
- ✓ Há duas importantes coisas em **newLISP** que são falsas; **nil** e a **lista vazia ()**.
- ✓ Qualquer coisa que newLISP não conhece também é retornado como falso.

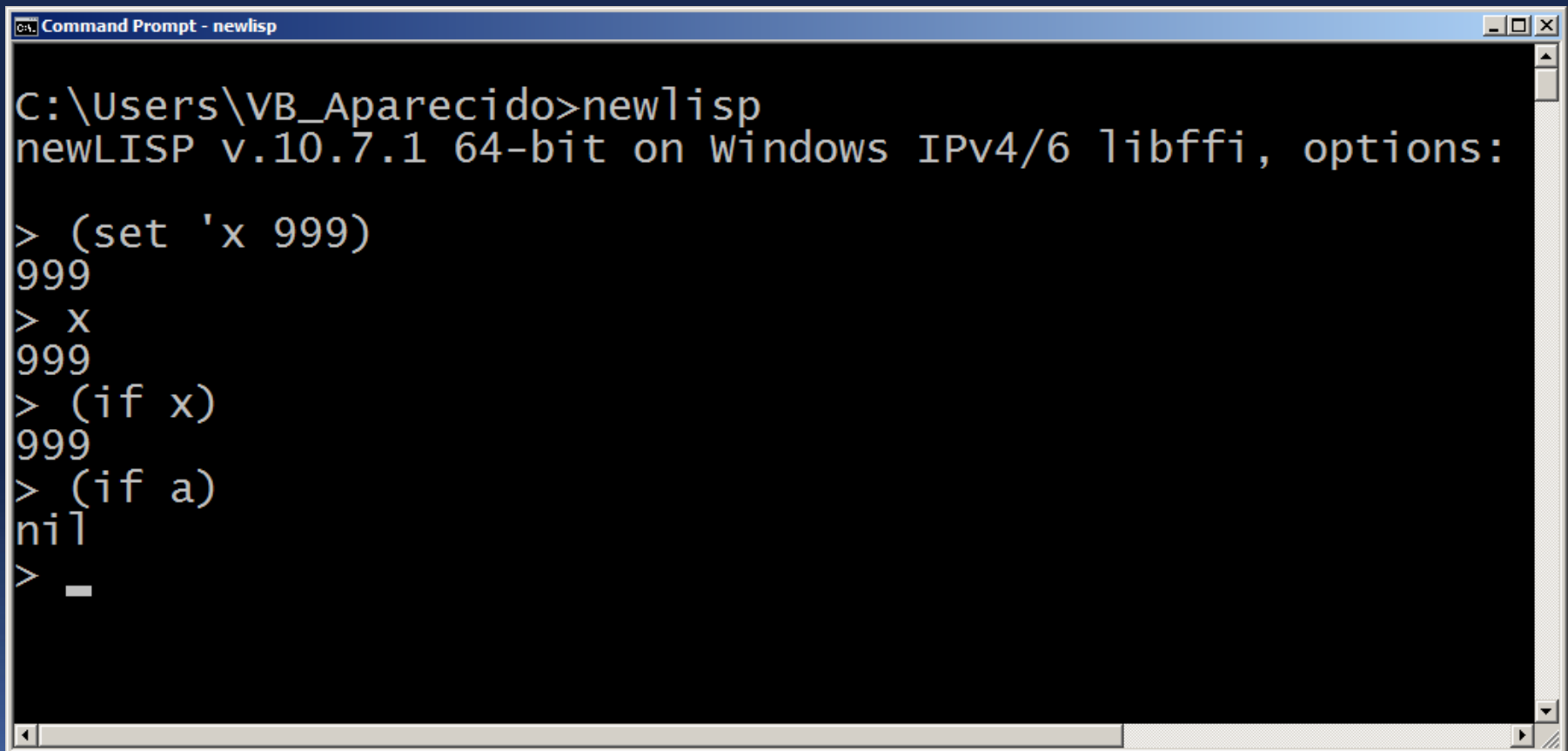
if



```
Command Prompt - newlisp
>
> (define PI 3.14)
3.14
> (if PI)
3.14
> (if (= PI 3.14) "Verdade" "Falso")
"Verdade"
> (define A 3.14)
3.14
> (if A)
3.14
> (if (= A PI) "Verdade" "Falso")
"Verdade"
>
```

if

- ✓ O símbolo **x** é conhecido, portanto (if x) retorna **999** que é **true**;
- ✓ O símbolo **y** é desconhecido, portanto (if y) retorna **nil**, que corresponde a **false**.

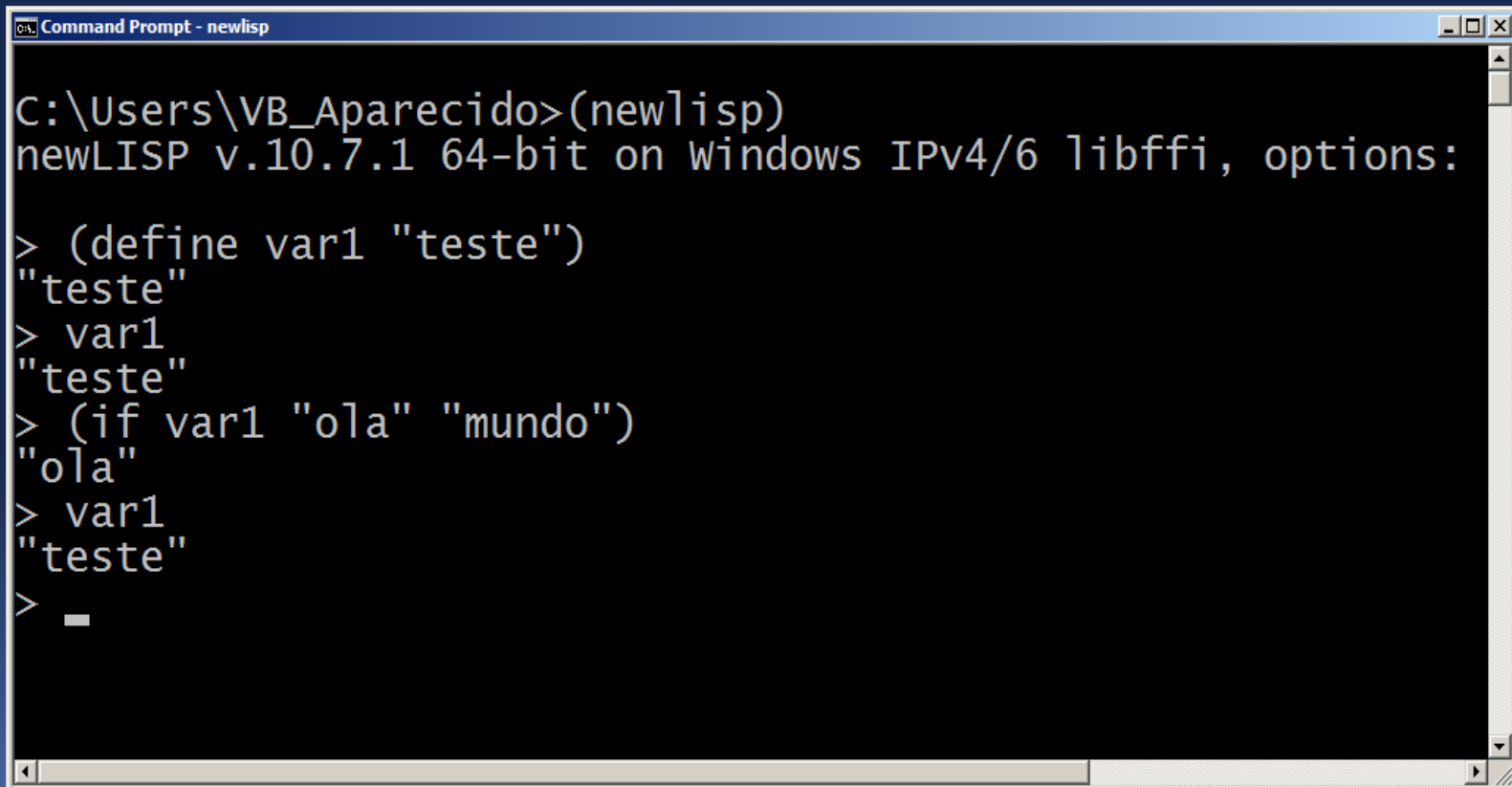


```
Command Prompt - newlisp
C:\Users\VB_Aparecido>newlisp
newLISP v.10.7.1 64-bit on Windows IPv4/6 libffi, options:

> (set 'x 999)
999
> x
999
> (if x)
999
> (if a)
nil
>
```

if

- ✓ O símbolo **var1** é conhecido, portanto (**if var1 "ola" "mundo"**) retorna **"ola"** que é **true**;
- ✓ Mas, o símbolo **var1** após o **if**, continua com o valor original, **"teste"**
- ✓ A terceira expressão, representa uma ação **else**.



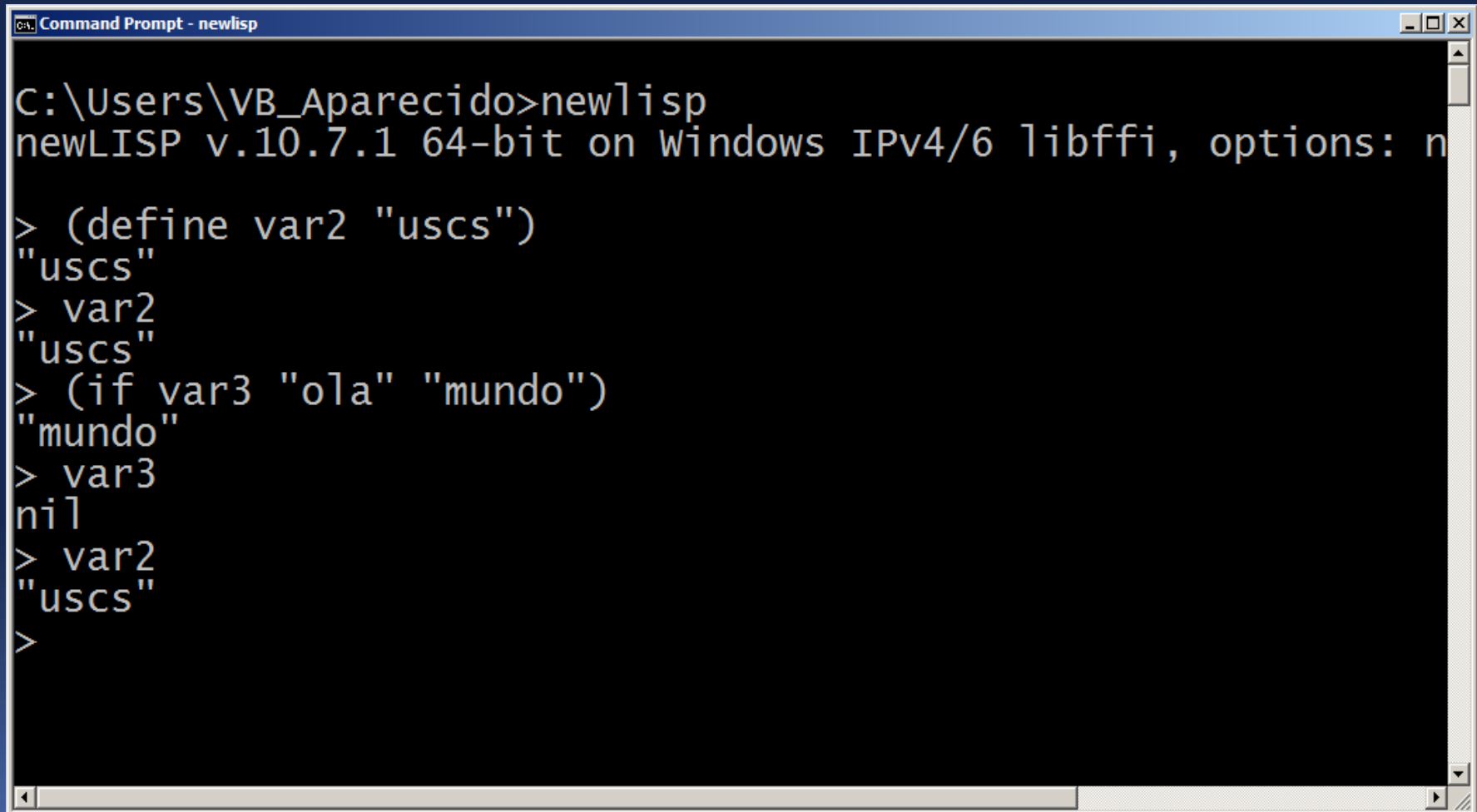
```
Command Prompt - newlisp

C:\Users\VB_Aparecido>(newlisp)
newLISP v.10.7.1 64-bit on Windows IPv4/6 libffi, options:

> (define var1 "teste")
"teste"
> var1
"teste"
> (if var1 "ola" "mundo")
"ola"
> var1
"teste"
> _
```

if

- ✓ O símbolo **var3** não é conhecido, portanto (if var3 "ola" "mundo") retorna "mundo" ;
- ✓ O símbolo **var2** após o if, continua com o valor original, "uscs";
- ✓ O símbolo **var3** não está definido e, portanto, ao acessá-lo será retornado **nil**.

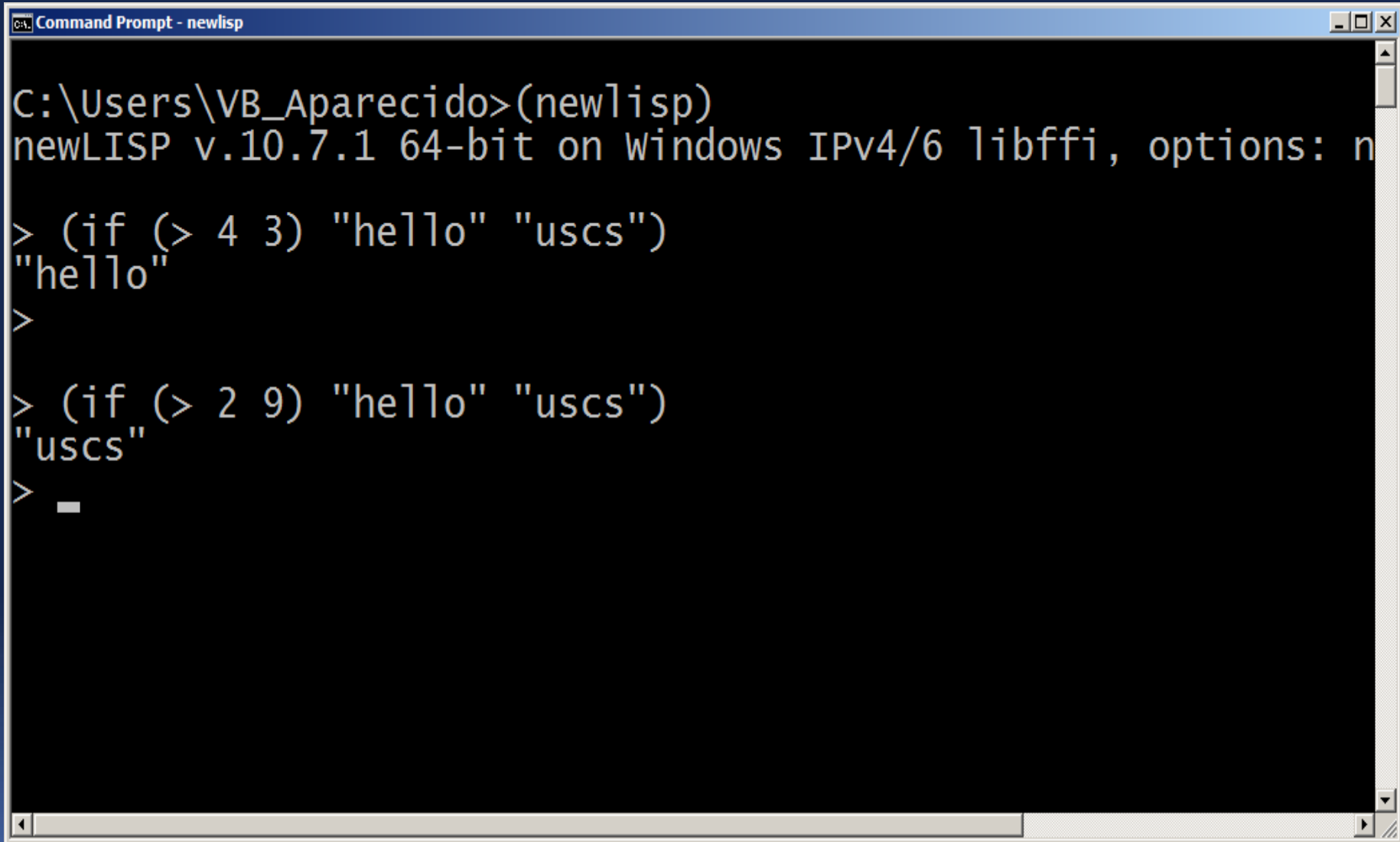


```
Command Prompt - newlisp

C:\Users\VB_Aparecido>newlisp
newLISP v.10.7.1 64-bit on Windows IPv4/6 libffi, options: n

> (define var2 "uscs")
"uscs"
> var2
"uscs"
> (if var3 "ola" "mundo")
"mundo"
> var3
nil
> var2
"uscs"
>
```

Testes com if



```
Command Prompt - newlisp

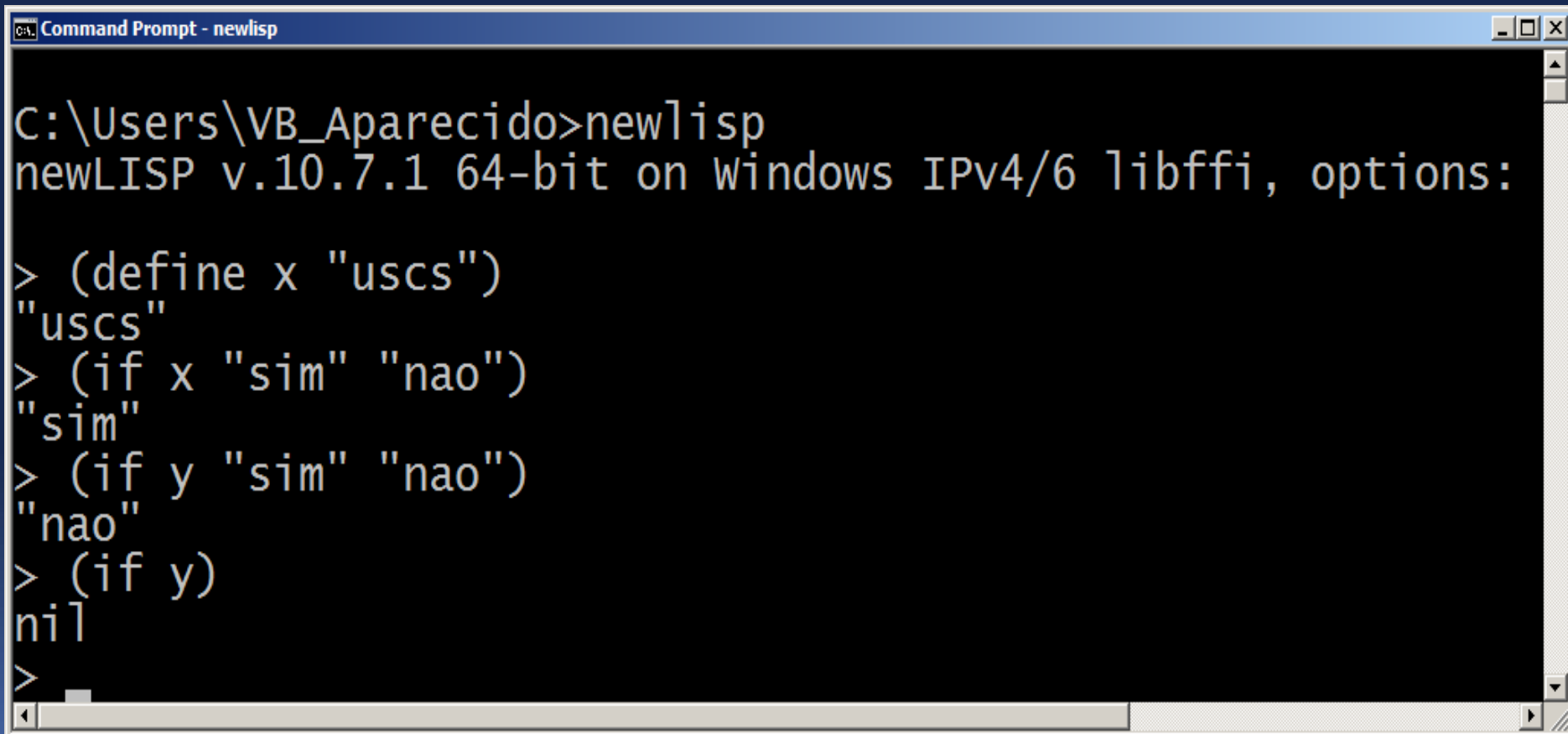
C:\Users\VB_Aparecido>(newlisp)
newLISP v.10.7.1 64-bit on Windows IPv4/6 libffi, options: n

> (if (> 4 3) "hello" "uscs")
"hello"
>

> (if (> 2 9) "hello" "uscs")
"uscs"
> _
```

Testes com if

- ✓ Se um símbolo é avaliado como **nil** (talvez porque ele não exista ou não tenha sido assinalado a algum valor) newLISP o considera **falso** e o teste retorna **nil**.

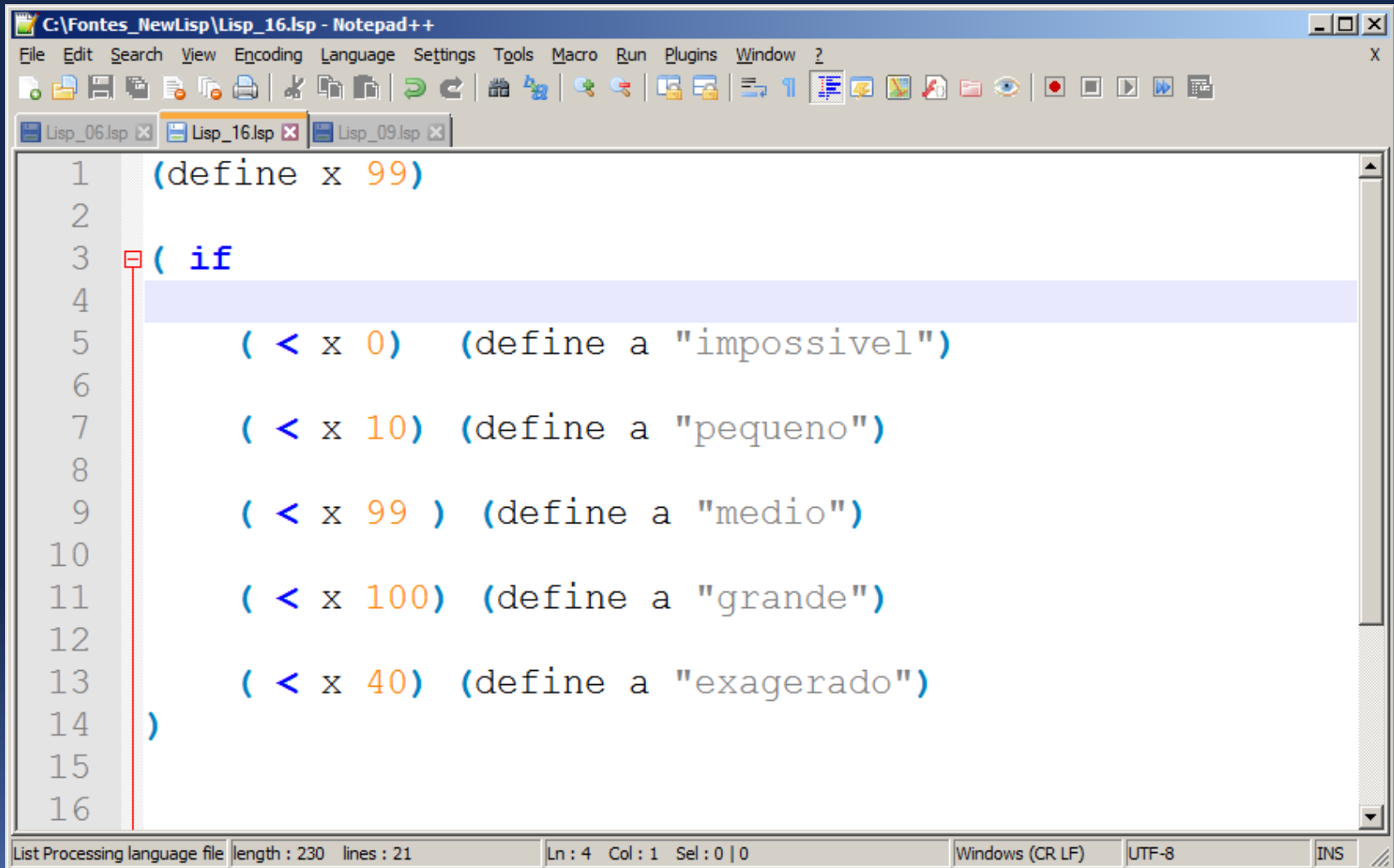


```
C:\Users\VB_Aparecido>newlisp
newLISP v.10.7.1 64-bit on Windows IPv4/6 libffi, options:

> (define x "uscs")
"uscs"
> (if x "sim" "nao")
"sim"
> (if y "sim" "nao")
"nao"
> (if y)
nil
>
```

If com diversos testes

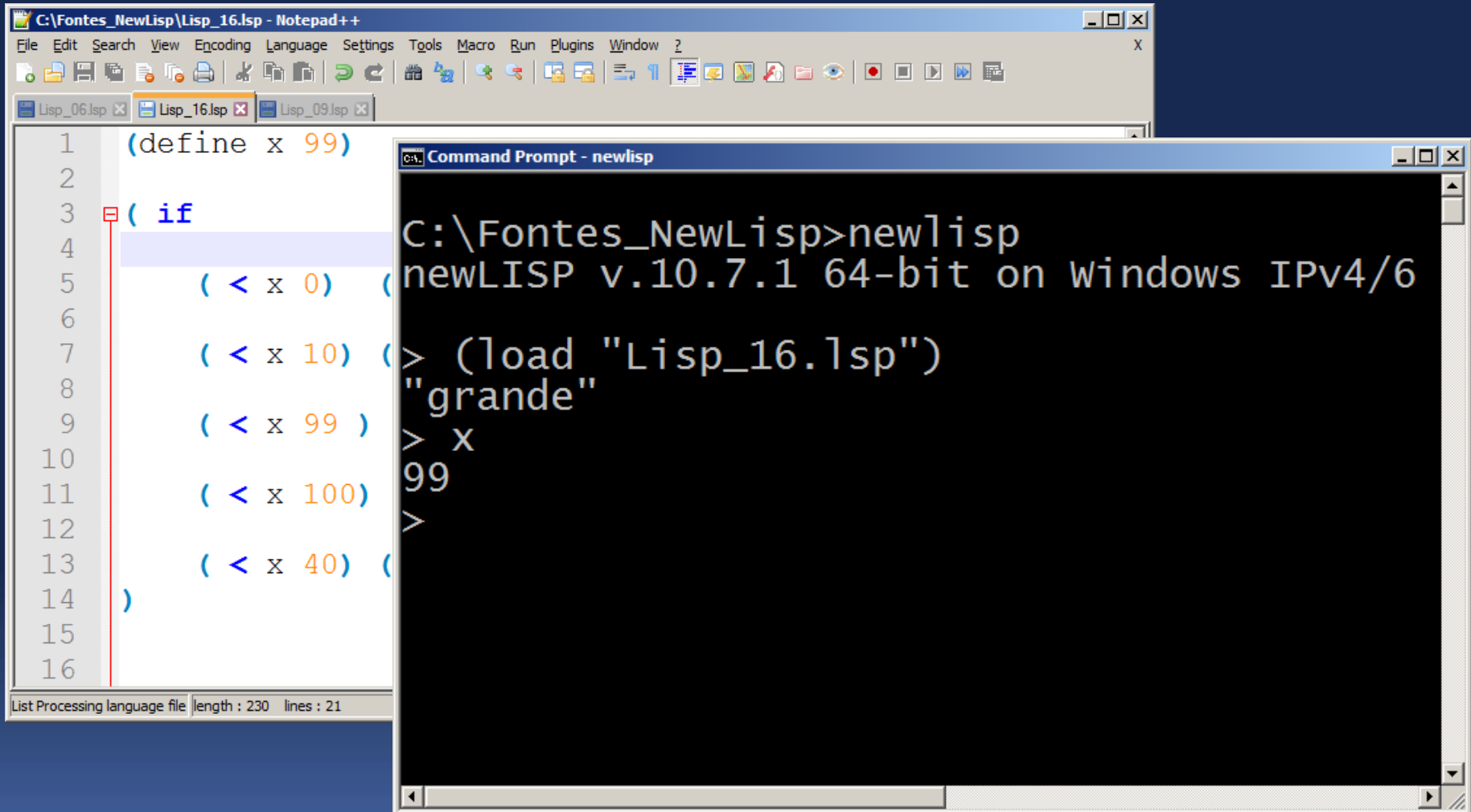
- ✓ Pode-se empregar if com um número ilimitado de testes e ações;
- ✓ Neste caso, a lista de if's consiste de uma série de pares teste-ação;



```
C:\Fontes_NewLisp\Lisp_16.lsp - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
Lisp_06.lsp x Lisp_16.lsp x Lisp_09.lsp x
1 (define x 99)
2
3 ( if
4
5     ( < x 0) (define a "impossivel")
6
7     ( < x 10) (define a "pequeno")
8
9     ( < x 99 ) (define a "medio")
10
11     ( < x 100) (define a "grande")
12
13     ( < x 40) (define a "exagerado")
14 )
15
16
List Processing language file length : 230 lines : 21 Ln : 4 Col : 1 Sel : 0 | 0 Windows (CR LF) UTF-8 INS
```

If com diversos testes

- ✓ Pode-se empregar if com um número ilimitado de testes e ações;
- ✓ Neste caso, a lista de if's consiste de uma série de pares teste-ação;



The image shows a Notepad++ window editing a Lisp file and a Command Prompt window running the code.

Notepad++ - C:\Fontes_NewLisp\Lisp_16.lisp

```
1 (define x 99)
2
3 (if
4
5   (< x 0) (
6
7   (< x 10) (> (load "Lisp_16.lisp")
8             "grande"
9             > x
10            99
11            (< x 100)
12            (< x 40) (
13
14 )
15
16
```

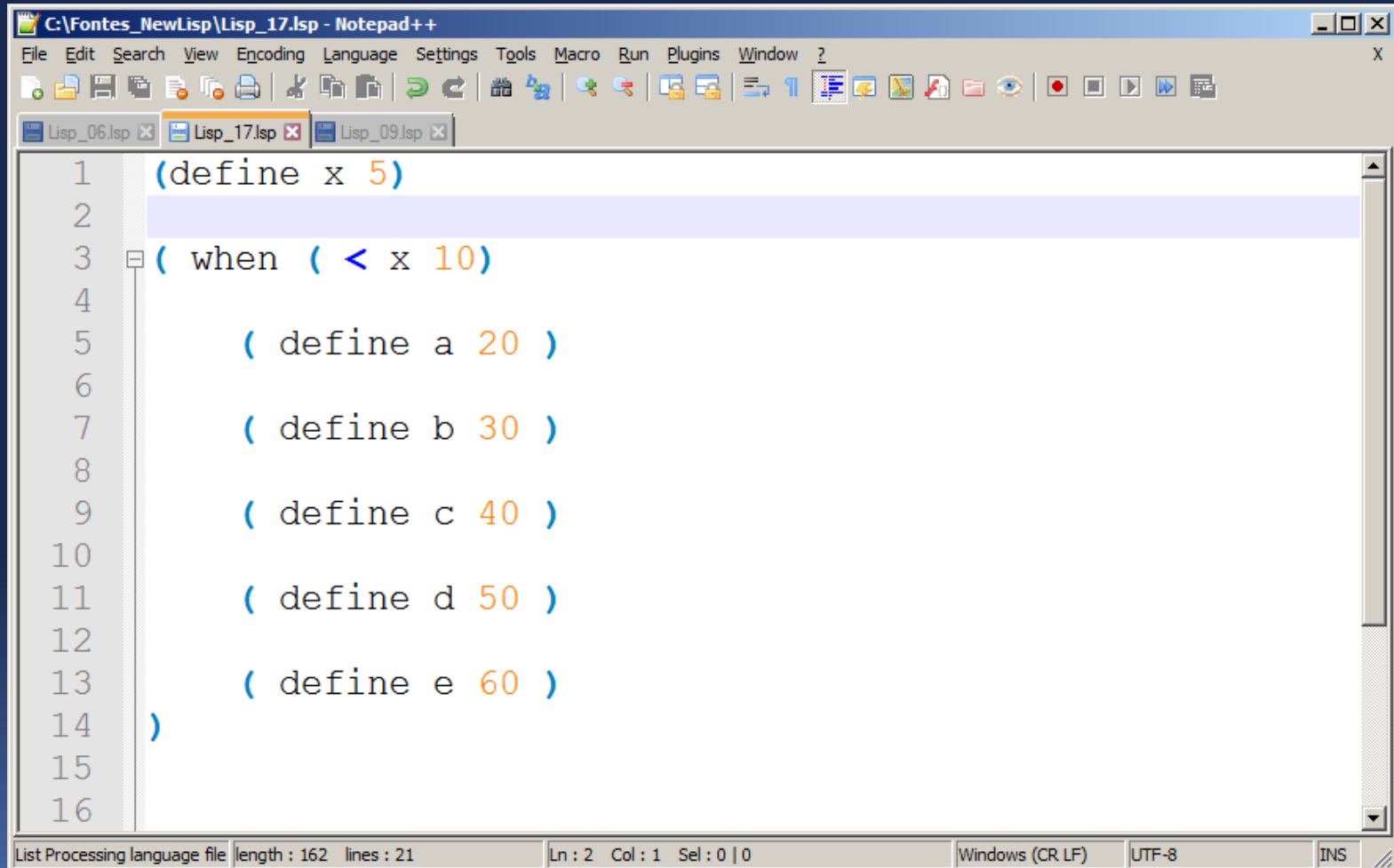
Command Prompt - newlisp

```
C:\Fontes_NewLisp>newlisp
newLISP v.10.7.1 64-bit on windows IPv4/6
> (load "Lisp_16.lisp")
"grande"
> x
99
>
```

At the bottom of the Notepad++ window, it says: List Processing language file | length : 230 | lines : 21

Função when

- ✓ A função **when** permite que duas ou mais ações sejam processadas se um teste for satisfeito;
- ✓ Pode-se usar **when** como um if sem a parte 'else'.

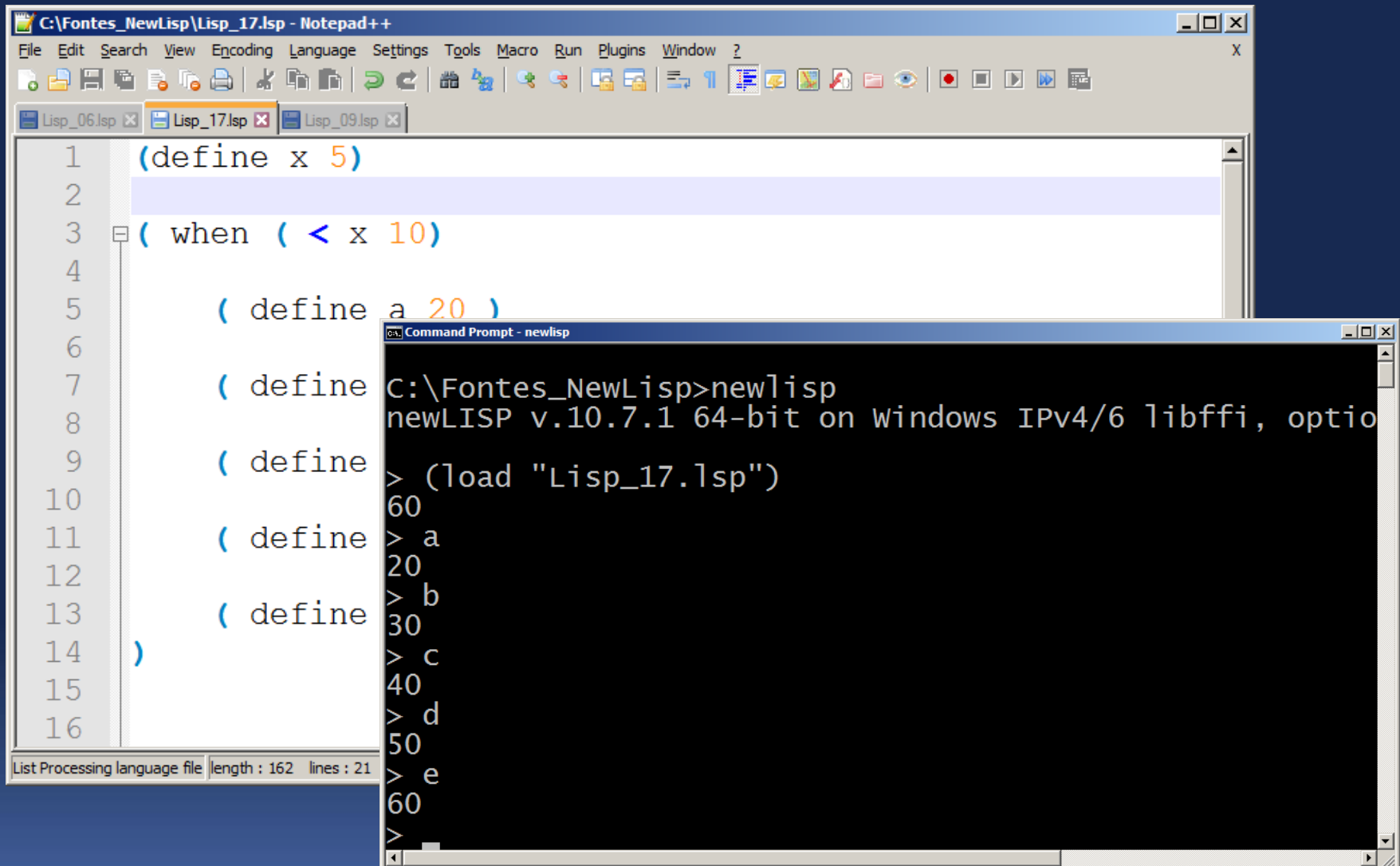


```
C:\Fontes_NewLisp\Lisp_17.lsp - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
Lisp_06.lsp x Lisp_17.lsp x Lisp_09.lsp x

1 (define x 5)
2
3 ( when ( < x 10)
4
5     ( define a 20 )
6
7     ( define b 30 )
8
9     ( define c 40 )
10
11    ( define d 50 )
12
13    ( define e 60 )
14 )
15
16

List Processing language file length : 162 lines : 21 Ln : 2 Col : 1 Sel : 0 | 0 Windows (CR LF) UTF-8 INS
```

Função when



The image shows a Notepad++ window editing a Lisp file named `Lisp_17.lsp`. The code defines a variable `x` with the value 5, and then uses the `when` function to execute a block of code if `x` is less than 10. The code block contains several `define` statements for variables `a`, `b`, `c`, `d`, and `e`, each followed by a value. The Command Prompt window shows the execution of the `newlisp` command, loading the `Lisp_17.lsp` file, and the resulting output of the `when` function, which prints the values of `a` through `e`.

```
C:\Fontes_NewLisp\Lisp_17.lsp - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
Lisp_06.lsp x Lisp_17.lsp x Lisp_09.lsp x

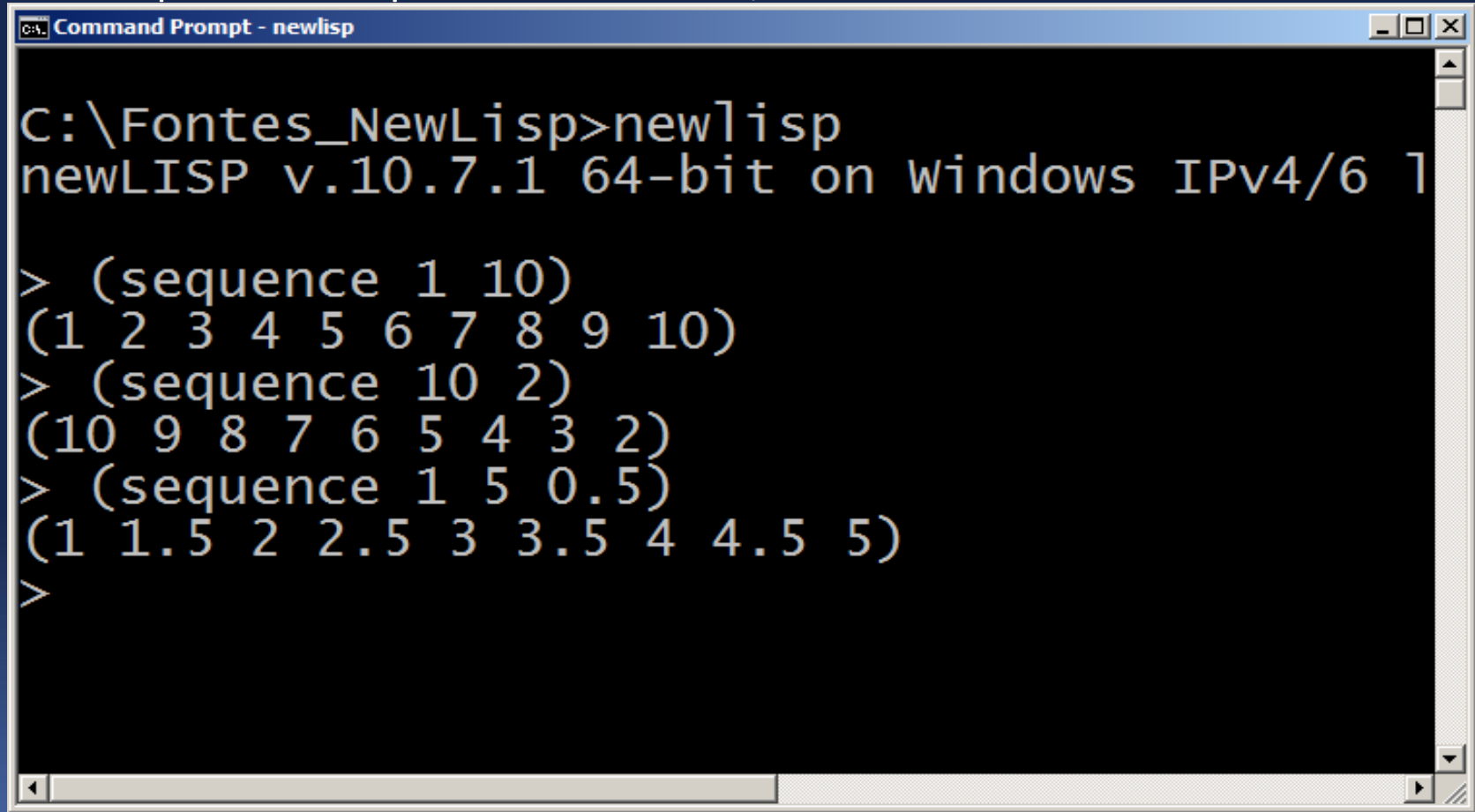
1 (define x 5)
2
3 (when (< x 10)
4
5   (define a 20)
6
7   (define
8
9   (define
10
11   (define
12
13   (define
14 )
15
16

List Processing language file length : 162 lines : 21

Command Prompt - newlisp
C:\Fontes_NewLisp>newlisp
newLISP v.10.7.1 64-bit on Windows IPv4/6 libffi, optio
> (load "Lisp_17.lsp")
60
> a
20
> b
30
> c
40
> d
50
> e
60
>
```

Trabalhando com Listas

- ✓ A função **sequence** gera uma **sequência** de números em um intervalo, com um valor opcional definindo o passo da sequência.
- ✓ Quando o passo da sequência for omitido, o valor 1 é assumido.

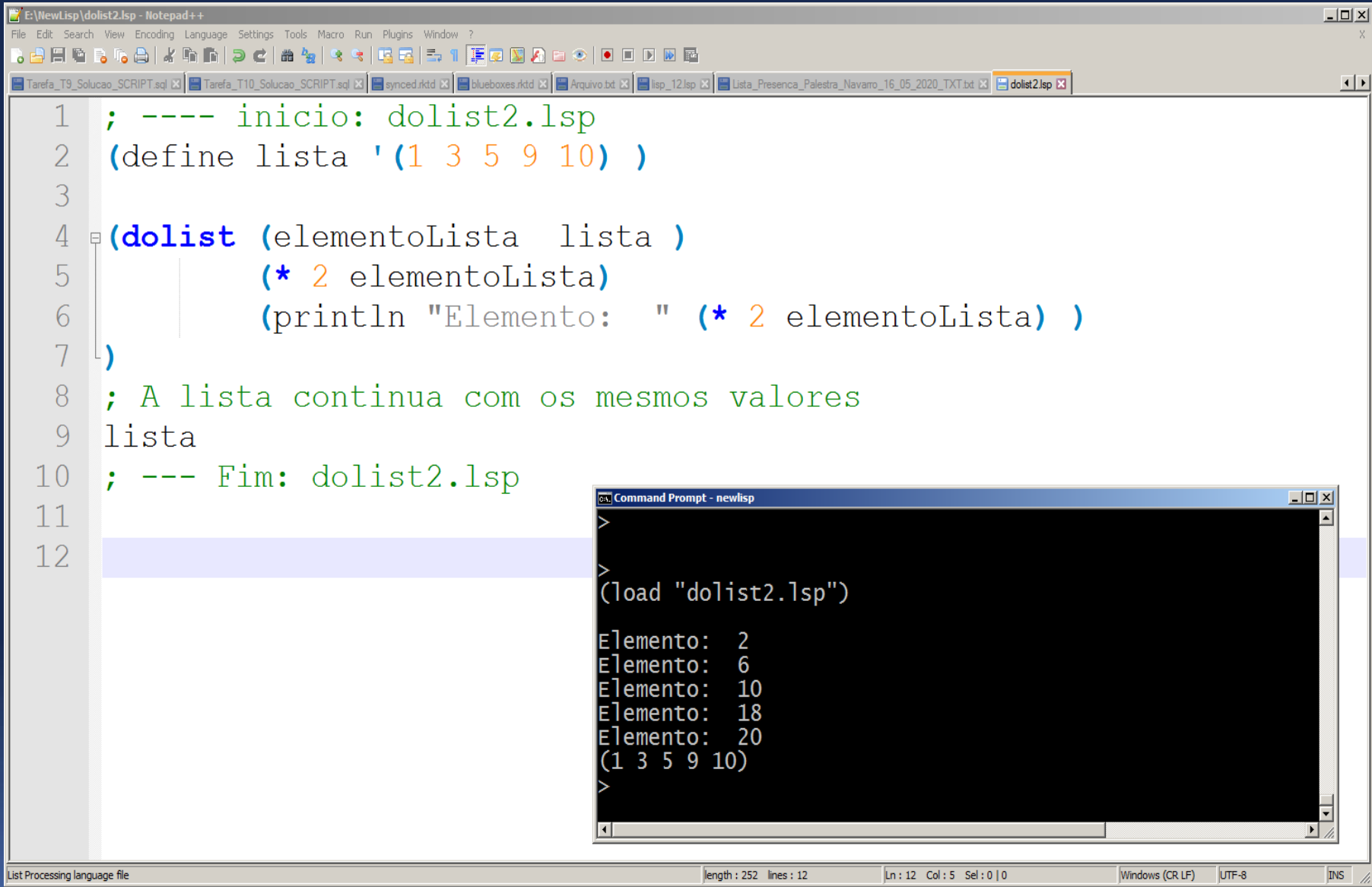


```
C:\Fontes_NewLisp>newlisp
newLISP v.10.7.1 64-bit on Windows IPv4/6 1

> (sequence 1 10)
(1 2 3 4 5 6 7 8 9 10)
> (sequence 10 2)
(10 9 8 7 6 5 4 3 2)
> (sequence 1 5 0.5)
(1 1.5 2 2.5 3 3.5 4 4.5 5)
>
```

- ✓ A função **dolist** seta um símbolo local (“variável”) para cada item da lista e permite que se executem ações em

Função dolist



```
E:\NewLisp\dolist2.lsp - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?

Tarefa_T9_Solucao_SCRIPT.sql Tarefa_T10_Solucao_SCRIPT.sql synced.rkt blueboxes.rkt Arquivo.txt lsp_12.lsp Lista_Presenca_Palestra_Navarro_16_05_2020_TXT.txt dolist2.lsp

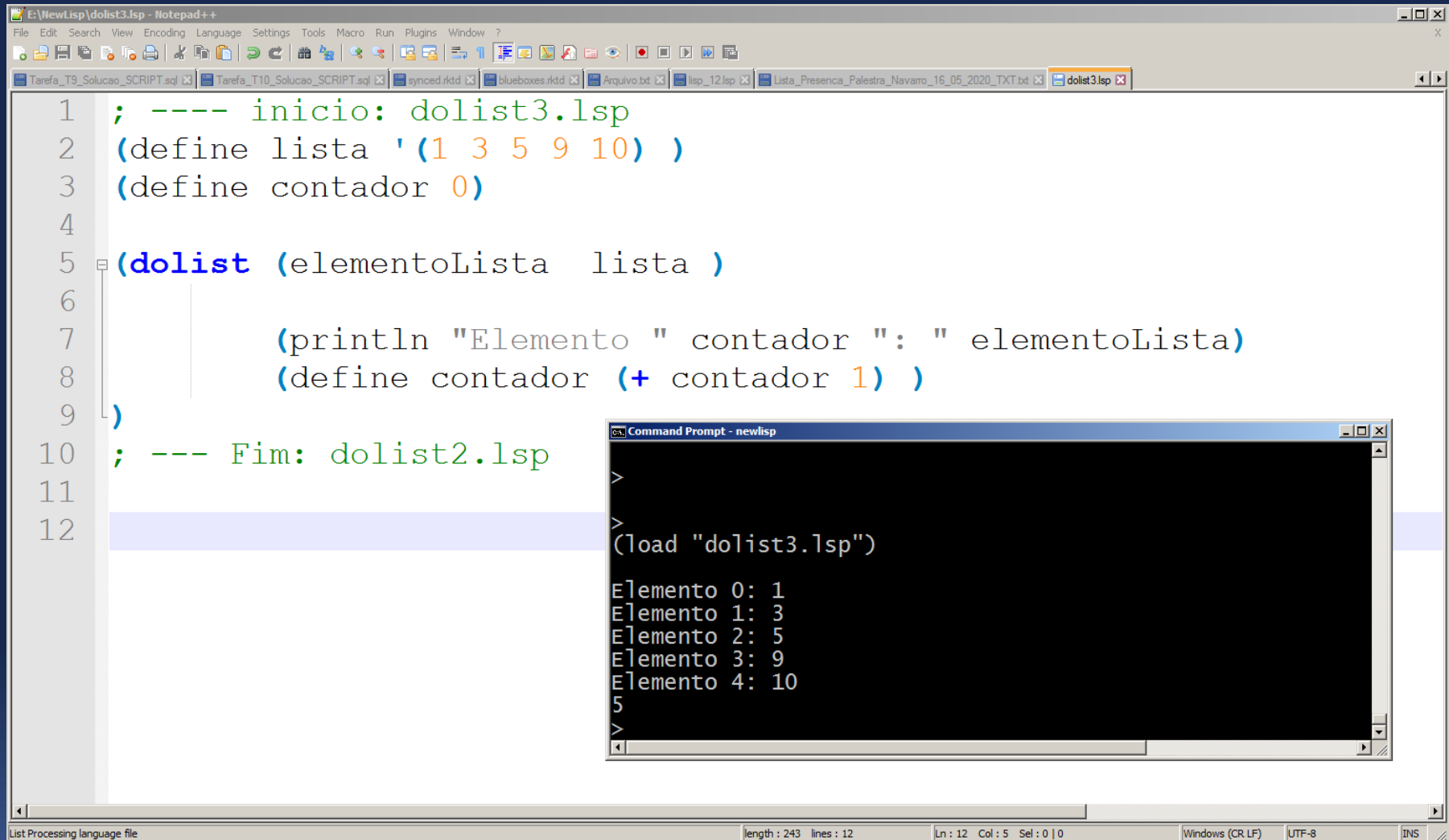
1 ; ---- inicio: dolist2.lsp
2 (define lista '(1 3 5 9 10) )
3
4 (dolist (elementoLista lista )
5       (* 2 elementoLista)
6       (println "Elemento: " (* 2 elementoLista) )
7 )
8 ; A lista continua com os mesmos valores
9 lista
10 ; --- Fim: dolist2.lsp
11
12
```

```
Command Prompt - newlisp
>
>
(load "dolist2.lsp")

Elemento: 2
Elemento: 6
Elemento: 10
Elemento: 18
Elemento: 20
(1 3 5 9 10)
>
```

List Processing language file length : 252 lines : 12 Ln : 12 Col : 5 Sel : 0 | 0 Windows (CR LF) UTF-8 INS

Função dolist



```
1 ; ---- inicio: dolist3.lsp
2 (define lista '(1 3 5 9 10) )
3 (define contador 0)
4
5 (dolist (elementoLista lista)
6
7     (println "Elemento " contador ": " elementoLista)
8     (define contador (+ contador 1) )
9 )
10 ; --- Fim: dolist2.lsp
11
12
```

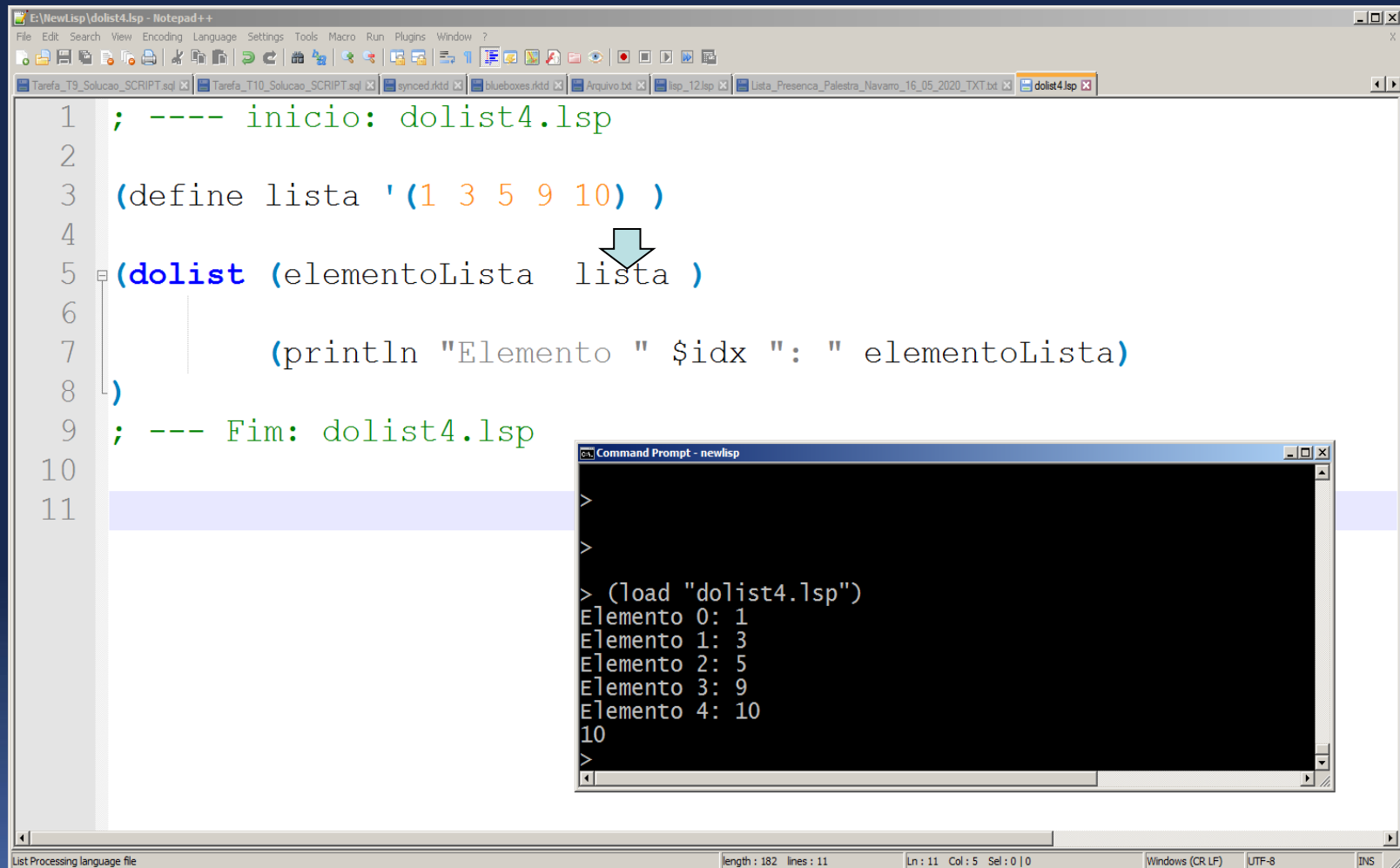
```
Command Prompt - newlisp
>
>
(load "dolist3.lsp")

Elemento 0: 1
Elemento 1: 3
Elemento 2: 5
Elemento 3: 9
Elemento 4: 10
5
>
```

List Processing language file | length : 243 lines : 12 | Ln : 12 Col : 5 Sel : 0 | 0 | Windows (CR LF) | UTF-8 | INS

Contador de Loop mantido por newLISP

- ✓ newLISP automaticamente mantém um contador de loop chamado **\$idx**



The screenshot displays a Notepad++ editor window titled "E:\NewLisp\dolist4.lsp - Notepad++". The script content is as follows:

```
1 ; ---- inicio: dolist4.lsp
2
3 (define lista '(1 3 5 9 10) )
4
5 (dolist (elementoLista lista )
6
7     (println "Elemento " $idx ": " elementoLista)
8 )
9 ; --- Fim: dolist4.lsp
10
11
```

A blue arrow points from the `lista` argument in the `dolist` function call on line 5 to the `lista` argument in the `(define lista ...)` definition on line 3.

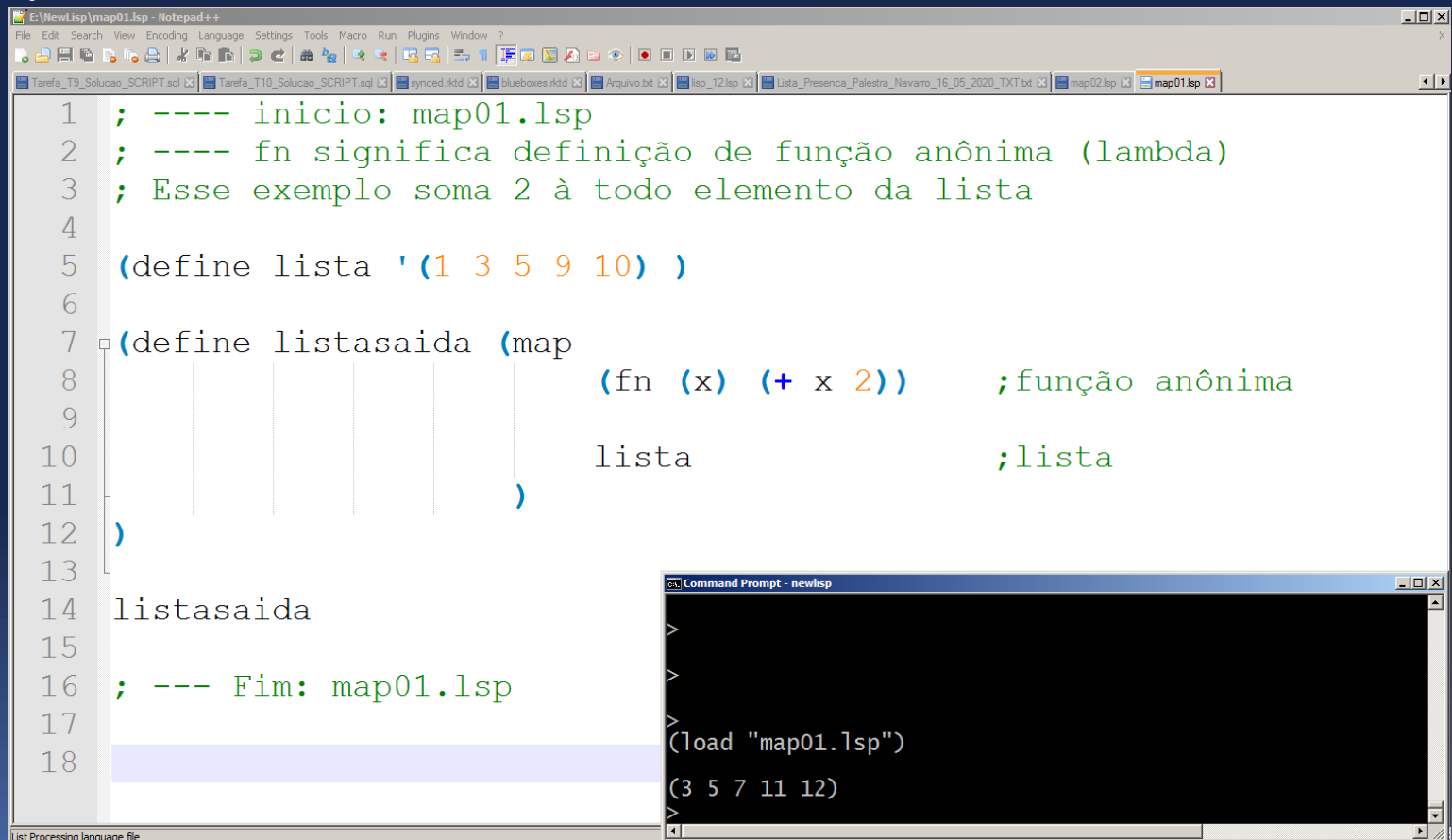
Overlaid on the bottom right is a "Command Prompt - newlisp" window showing the execution output:

```
>
>
> (load "dolist4.lsp")
Elemento 0: 1
Elemento 1: 3
Elemento 2: 5
Elemento 3: 9
Elemento 4: 10
10
>
>
```

The status bar at the bottom of the Notepad++ window indicates: "List Processing language file", "length: 182 lines: 11", "Ln: 11 Col: 5 Sel: 0 | 0", "Windows (CR LF)", "UTF-8", and "JNS".

Função map

- ✓ A função **map** pode ser usada para se aplicar uma função à todo elemento de uma lista;
- ✓ A **função** a ser aplicada aos elementos da lista é definida de forma anônima por **fn**.

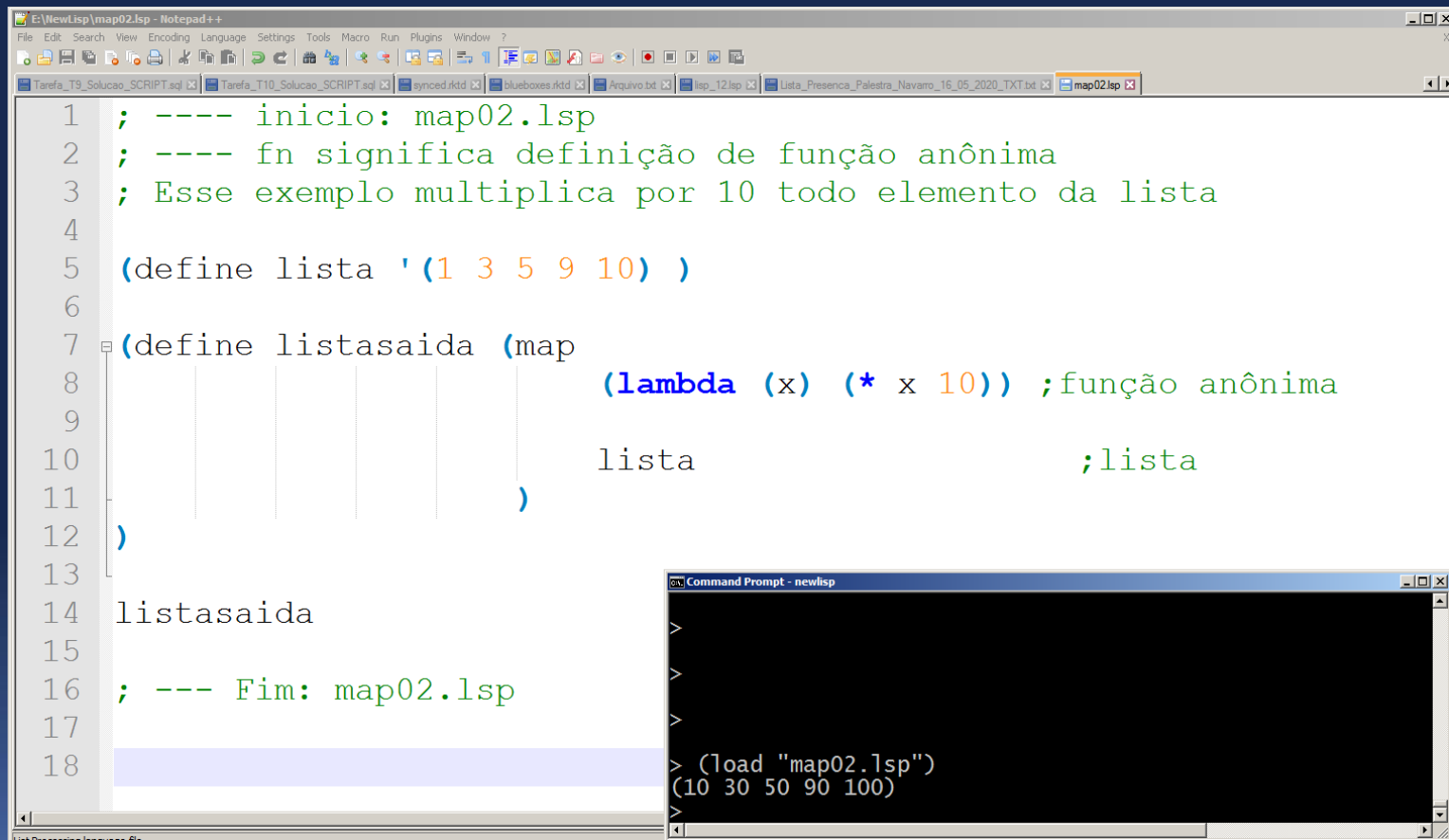


```
1 ; ---- inicio: map01.lsp
2 ; ---- fn significa definição de função anônima (lambda)
3 ; Esse exemplo soma 2 à todo elemento da lista
4
5 (define lista '(1 3 5 9 10) )
6
7 (define listasaida (map
8   (fn (x) (+ x 2)) ;função anônima
9   lista             ;lista
10 )
11 )
12
13 listasaida
14
15
16 ; --- Fim: map01.lsp
17
18
```

```
>
>
> (load "map01.lsp")
(3 5 7 11 12)
>
```

Função map

- ✓ A função **map** pode ser usada para se aplicar uma função à todo elemento de uma lista;
- ✓ A **função** a ser aplicada aos elementos da lista pode ser anonimamente definida por expressão **lambda** (que equivale ao fn)



```

1 ; ---- inicio: map02.lsp
2 ; ---- fn significa definição de função anônima
3 ; Esse exemplo multiplica por 10 todo elemento da lista
4
5 (define lista '(1 3 5 9 10) )
6
7 (define listasaida (map
8   (lambda (x) (* x 10)) ;função anônima
9   lista
10  )
11 )
12
13 listasaida
14
15
16 ; --- Fim: map02.lsp
17
18

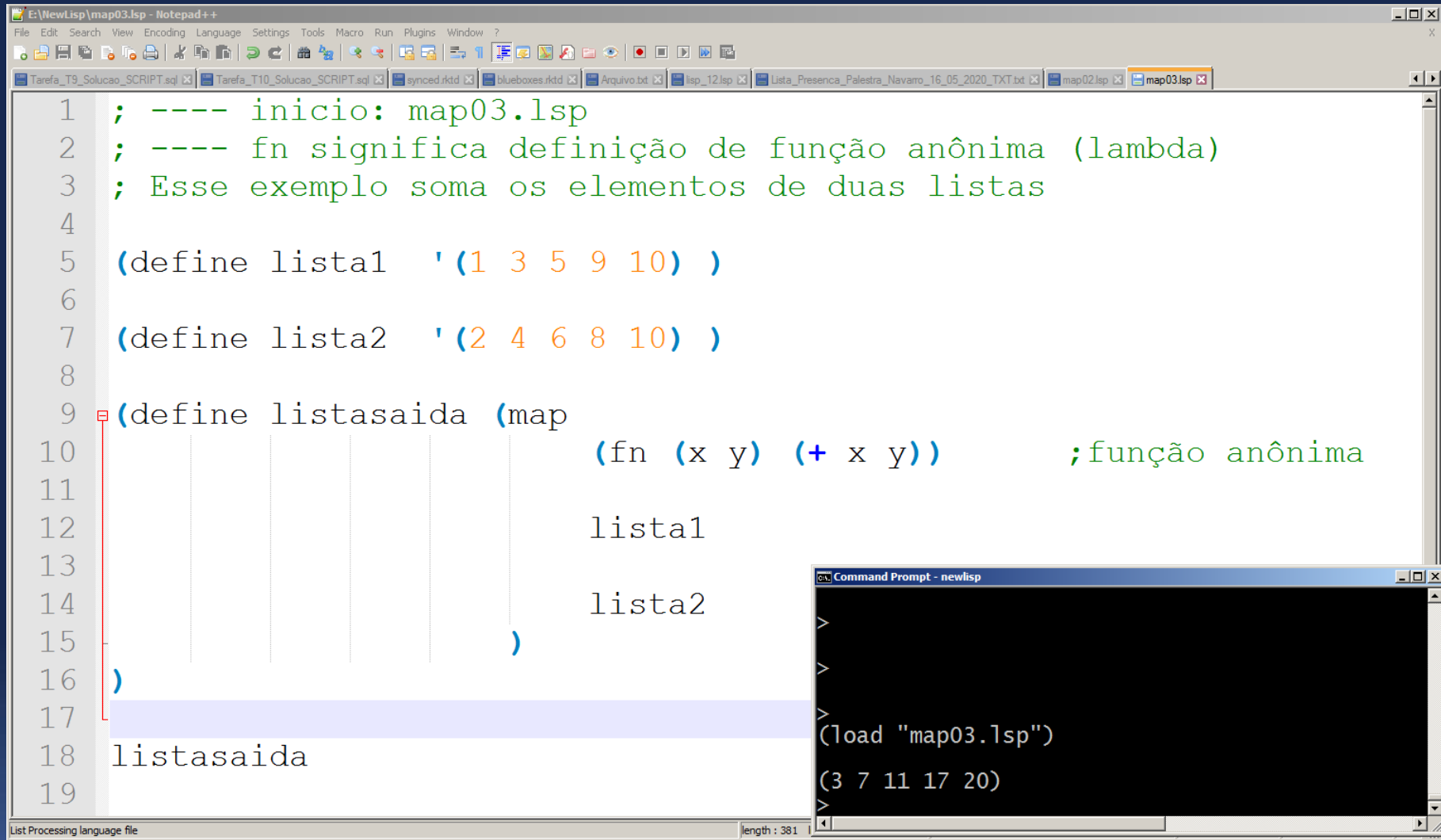
```

```

>
>
>
> (load "map02.lsp")
> (10 30 50 90 100)
>

```

Função map



```
E:\NewLisp\map03.lsp - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
Tarefa_T9_Solucao_SCRIPT.sql Tarefa_T10_Solucao_SCRIPT.sql synced.rkt.d blueboxes.rkt.d Arquivo.txt lsp_12.lsp Lista_Presenca_Palestra_Navarro_16_05_2020_TXT.txt map02.lsp map03.lsp
1 ; ---- inicio: map03.lsp
2 ; ---- fn significa definição de função anônima (lambda)
3 ; Esse exemplo soma os elementos de duas listas
4
5 (define lista1 '(1 3 5 9 10) )
6
7 (define lista2 '(2 4 6 8 10) )
8
9 (define listasaida (map
10                     (fn (x y) (+ x y))           ;função anônima
11                     lista1
12                     lista2
13                     )
14 )
15
16
17
18 listasaida
19
```

```
Command Prompt - newlisp
>
>
> (load "map03.lsp")
(3 7 11 17 20)
>
```

List Processing language file length : 381